Planar Line-Query Decision Trees: Approximation and Strong NP-Completeness

October 13, 2026

Abstract

We identify a hidden planar point using only a supplied separating family of line queries. For maximum depth and arbitrary-prior expected depth, respectively, we give deterministic polynomial approximation factors below 4.464966 and 5.182976, and quasipolynomial $(2 + \varepsilon)$ approximations for both objectives. The shared geometric reduction turns difficult states into circular-interval discovery followed by pattern decoding. Restriction of an optimal tree pays for discovery; decreasing count or entropy potentials pay for decoding. Exact scheduling and short lookahead exploit these credits.

Both exact objectives are strongly NP-complete, including expected depth under a uniform prior. Distinguished all-miss hypotheses yield stronger restricted algorithms: exact maximum depth for fixed-size caps, an EPTAS for weighted singleton/pair paths and cycles, and a PTAS for uniform fixed-size convex-position caps without singleton tests. Constant-factor inapproximability remains open. All constructed queries are supplied lines; an on-line third answer admits a factor-two transfer. For the distinct randomized objective $\max_h \mathbb{E}_T d_T(h)$, an expected-depth oracle and a finite mixture LP give the same polynomial and quasipolynomial factors, with at most n sampled trees.

Contents

1 Introduction **3**
- 1.1 Results and the main ideas . . . 4

2 Preliminaries and basic accounting **5**
- 2.1 Line-query model and objectives . . . 5
- 2.2 Binary codes and ordered comparisons . . . 9
- 2.3 Entropy and information inequalities . . . 10
- 2.4 Restriction and recursive accounting . . . 12
- 2.5 Proofs of the coding and accounting facts . . . 13
- 2.6 Convex geometry and finite set systems . . . 15
- 2.7 Query orderings and first-cover costs . . . 16
- 2.8 Normalized queries and hull patterns . . . 16

3 Common-witness geometry and interval tools **17**
- 3.1 A common majority witness . . . 18
- 3.2 Adjacent supports and circular patterns . . . 18
- 3.3 Interval hitting and discovery length . . . 19
- 3.4 Recovering the interval endpoints . . . 20
- 3.5 Proofs of the geometric and interval facts . . . 21

4 Maximum-depth scheduling ingredients **24**
4.1 Local credits and the priority fallback . . . 24
4.2 Technical proofs of the local ingredients . . . 26

5 Scheduling the first positive answer **28**
5.1 The batch-ordering guarantee . . . 30
5.2 Proofs of the batch-ordering propositions . . . 31

6 A polynomial approximation for expected depth **34**
6.1 The algorithm and its supporting propositions . . . 34
6.2 Proofs of the supporting propositions . . . 38
6.3 Calibration of the expected-depth bound . . . 43

7 Quasipolynomial approximations approaching two **44**
7.1 Algorithm and the three supporting propositions . . . 44
7.2 Proofs of the supporting propositions . . . 46
7.3 A simple calibration; no optimization is needed . . . 50

8 Polynomial maximum depth with rank-subset scheduling **50**
8.1 The algorithm and its guarantee . . . 50
8.2 Technical proofs of the propositions . . . 52
8.3 Parameter calibration . . . 55

9 Stronger algorithms for small caps **56**
9.1 Exact maximum depth with bounded caps . . . 56
9.2 An EPTAS for weighted singleton and pair caps . . . 58

10 A PTAS for uniform shallow caps in convex position **62**
10.1 Supporting propositions and the algorithm . . . 63
10.2 Technical proofs of the supporting propositions . . . 64
10.3 Calibration of the block length . . . 66

11 Randomized per-hypothesis depth and optimal mixing **66**
11.1 One algorithm and its supporting propositions . . . 67
11.2 Proofs of the supporting propositions . . . 69
11.3 Calibration and the strongest resulting guarantees . . . 72

12 Allowing hypotheses on supplied lines **72**

13 Strong NP-completeness **73**
13.1 The geometric realization and the matching choice . . . 74
13.2 Maximum depth: bin loads become branch-height counts . . . 76
13.3 Expected depth: an exact matching formula . . . 80
13.4 Expected depth: bin loads become a quadratic penalty . . . 82

14 Uniform expected-depth hardness **87**
14.1 Exact cloud replacement: normalization before accounting . . . 87
14.2 Uniform-prior strong NP-completeness . . . 91

15 Conclusion **92**

A Fixed certificate for the two initial batches **93**

References **95**

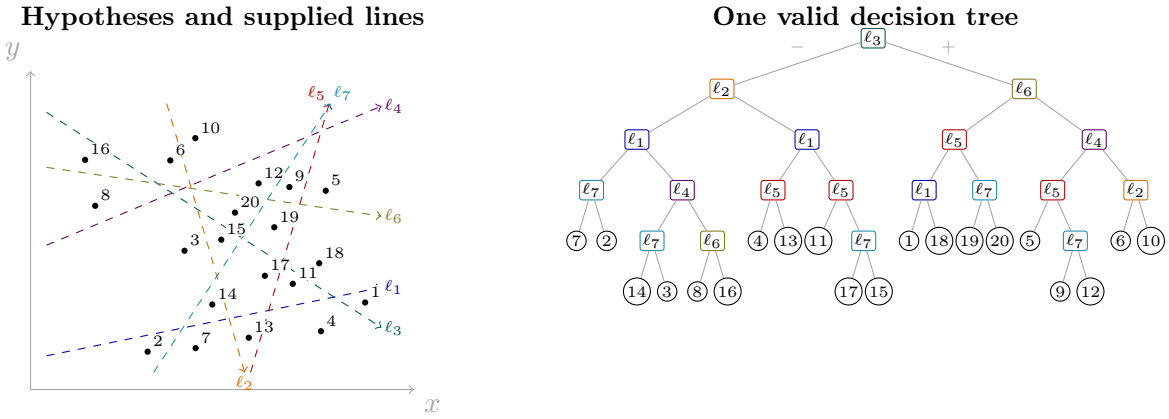


Figure 1: Seven randomly oriented supplied lines are drawn first, then twenty hypotheses are placed in distinct cells of their arrangement. The arrows orient the lines: the positive side is to the left of the arrow. Each internal node queries the correspondingly colored line. At every node the left branch is the negative answer and the right branch is the positive answer. The leaf numbers match the point labels. This valid tree has maximum depth five.

1 Introduction

An unknown point is promised to belong to a finite set of planar hypotheses. A query asks on which side of a specified line the point lies. The line family is part of the input: a strategy may choose the next query after seeing previous answers, but it may not invent a separator that was not supplied. Such a strategy is a decision tree; each leaf names the hypothesis identified by the answers on its path. Every query has unit cost.

We study two objectives for these trees. The *maximum depth* is the largest number of queries used on any hypothesis. The *expected depth* is the average under a given prior. These objectives value the same tree differently: maximum depth is governed by its hardest branch, whereas expected depth can spend more queries on low-probability hypotheses. Our expected-depth guarantees allow arbitrary strictly positive probabilities.

The supplied-query restriction is what makes the problem interesting. If arbitrary lines were allowed, a projection in a direction giving distinct projected coordinates would order the points, and repeated median splits would give height $\lceil \log_2 n \rceil$. A supplied family need not contain any balanced split. For example, if the only useful tests isolate individual points, the worst-case strategy may have to test them one at a time and use depth $n - 1$. Thus the information bound $\log_2 n$ is not by itself an algorithm: the challenge is to find a good tree using the separators that are actually available.

Figure 1 makes the input and output concrete. The left panel has seven supplied lines with varied orientations and twenty hypotheses placed in distinct cells of their arrangement. The right panel is one legal decision tree for that instance: successive line-side answers isolate a single hypothesis. Each internal node uses a line drawn on the left, and its leaves identify the same hypotheses marked by the numbered points.

1.1 Results and the main ideas

Setup and objective	Guarantee	Running time
General maximum depth	< 4.464966	Polynomial
General arbitrary-prior expected depth	< 5.182976	Polynomial
Either general objective	$2 + \varepsilon$	Quasipolynomial
Randomized $\max_h \mathbb{E}_T d_T(h)$	< 5.182976	Polynomial
Same randomized objective	$2 + \varepsilon$	Quasipolynomial
All-miss, fixed-size caps: maximum depth	Exact	Polynomial
All-miss, weighted singleton/pair paths and cycles: expected depth	$1 + \varepsilon$	EPTAS
All-miss, uniform fixed-size convex caps: expected depth	$1 + \varepsilon$	PTAS

Only the strongest algorithm in each setup is presented. Scheduling and coding procedures are supporting components of those algorithms. The expected-depth guarantee permits arbitrary strictly positive rational priors. For $0 < \varepsilon \leq 1$, the general quasipolynomial algorithms run in $(mn)^{O(\varepsilon^{-1} \log n)}$ time.

Both exact objectives are strongly NP-complete, including expected depth under a uniform prior. The reductions use integer-coordinate points on a parabola and supplied line queries. Constant-factor inapproximability remains open.

The geometric structure behind the algorithms is easiest to see in a state (the current set of hypotheses consistent with all answers so far) where no line splits much of the remaining mass. Every test then has a majority answer. The opposite, minority half-planes are small, and Helly's theorem gives a point lying on all majority sides. After translating this common witness to the origin, each query is represented by a point in a normalized plane. The vertices of their convex hull (the smallest convex set containing those points) suffice to describe the relevant caps: each original cap is covered by at most two adjacent vertex caps, and the vertex answers of any one hypothesis form a *circular interval*: a consecutive block in the cyclic order in which the hull vertices appear around the polygon, with wrap-around allowed. This turns the geometric part of the problem into scheduling and searching over circular intervals.

Maximum depth: schedule discovery against child heights. In a flat state, the algorithm first builds the recursive codes for the hull-pattern classes. It then searches for a positive hull answer using a schedule whose deadlines are derived from those codes' actual heights. An optimal-tree path through the common witness supplies a comparison schedule: classes that remain negative on a long prefix have already saved that many queries when the optimal tree is restricted to the class. Each original query has at most two adjacent hull supports, so the comparison schedule consists of singletons and adjacent pairs rather than arbitrary subsets. Enumerating its first pair turns the remaining circular intervals into ordinary intervals. The priority fallback preserves the resulting credit simultaneously for every deadline. A second, exact scheduler stores subsets of available ranks in a spatial Cartesian tree. It is polynomial under a logarithmic total cap; simultaneous recursive bounds for small and large optima make that cap sufficient for the stronger general factor. Endpoint searches are charged separately, with a smaller allowance when the discovery path already supplies a negative neighbor. This is how the analysis improves the constant without adding artificial height to a child code.

Expected depth: pay once for learning a pattern. For expected depth, the algorithm again discovers the complete hull-answer pattern before recursing. The analysis separates the work into two parts. First, a class's constant prefix on an optimal witness path disappears when the optimal

tree is restricted to that class; the saved queries pay for finding the class's first positive hull answer. Second, the remaining endpoint information is paid for by the entropy of the pattern that was not already revealed during discovery. Exact weighted coverage at each integer budget, together with a deterministic choice among shifted geometric batches, gives a sharper first-cover bound. Adaptive block modes also credit the information from a second endpoint answer, measured against a coarsened discovery transcript whose cells retain the individual cap bound. A mass cap on each adjacent support block forces some targets into later blocks, where the discrete bound is better; the waiting time of the all-zero pattern covers the complementary case. Both the optimal-depth savings and the entropy decrease telescope over the recursive partition, so no part of the optimal cost is charged repeatedly.

Strong hardness: encode bin loads by matching choices. The reductions start from 3-Partition. Singleton and adjacent-pair queries on disjoint odd paths make each item choose one of the path's even-position vertices. A distinguished hypothesis forces every optimal tree to make a maximum matching on these paths, so an unmatched position represents a bin choice. For maximum depth, clouds of points turn the resulting bin loads into counts of branches with specified remaining heights. For expected depth, carefully chosen weights turn them into a quadratic penalty for unequal loads. The parabola construction realizes every query by an explicit line. Replacing weight w by a cloud of w uniform points with private singleton tests adds the same internal identification cost to every matching invoice; this exact identity preserves the reduction.

Proof strategy.

Reading order.

1. Sections 2–3 explain the model, restriction credits, and the circular-interval geometry.
2. Sections 4–5 build the deadline and weighted-coverage primitives.
3. Sections 6, 7, and 8 assemble the best general approximation algorithms.
4. Sections 9–10 exploit bounded caps. The randomized mixture setup follows, in Section 11, connecting expected-depth oracles to mathematical programming.
5. Sections 12–14 treat boundary answers and strong NP-completeness.

Each long result first states its supporting propositions and gives the overall proof. The next subsection proves those propositions. Construction parameters are calibrated only after the structural argument.

2 Preliminaries and basic accounting

We collect the model, coding and information inequalities, and geometric terminology used by the algorithms. Reduction-specific graph and complexity definitions are given in Section 13. For a positive integer r, write $[r] = \{1, \dots, r\}$. The notation $|S|$ denotes set cardinality, while $|Q|$ denotes the length of a word, vector, or sequence when Q is such an object. The indicator $\mathbf{1}[P]$ is one if statement P holds and zero otherwise. We also write $\mathbf{1}_A(x) = \mathbf{1}[x \in A]$ for the indicator function of a set A. The floor $\lfloor x \rfloor$ and ceiling $\lceil x \rceil$ are the greatest integer at most x and the least integer at least x, respectively. For planar vectors, $x \cdot y = x_1 y_1 + x_2 y_2$ is the dot product.

2.1 Line-query model and objectives

Let $\mathcal{H} \subset \mathbb{R}^2$ be $n \geq 2$ distinct points. A finite family $\mathcal{L}$ of m supplied lines separates every pair of hypotheses, meaning that every two distinct points receive different answers to at least one line.

Write a line as the zero set of an affine function $f_\ell(x) = a_\ell \cdot x + b_\ell$, where $a_\ell \in \mathbb{R}^2 \setminus \{0\}$ and $b_\ell \in \mathbb{R}$. Querying ℓ returns the sign of $f_\ell(h)$ for the hidden point h. Every query contributes one to the depth. Unless explicitly stated otherwise, no hypothesis lies on a supplied line, so the answer is binary. Section 12 treats a third, on-line outcome.

After any sequence of queries and answers, the current *state* is the set of hypotheses whose signs on all queried lines agree with the answers received. For a state S and a supplied line ℓ, write

$$S_\ell^- = \{h \in S : f_\ell(h) < 0\}, \qquad S_\ell^+ = \{h \in S : f_\ell(h) > 0\}.$$

These are the two answer classes; the line *splits* S when both classes are nonempty.

Definition 2.1 (Line-query decision trees)**.** A binary line-query decision tree is a finite rooted tree whose internal nodes are labeled by supplied lines and whose two outgoing edges are labeled by their negative and positive answers. A hypothesis follows the edge matching its answer at each query. An *identifying* tree for S has hypothesis labels at its leaves and sends every $h \in S$ to a leaf labeled h. The root has depth zero; a leaf's depth is the number of queries on its root-to-leaf path, and the tree's *height* is its largest leaf depth.

A *partial decision tree*, or *stage*, may instead stop at leaves labeled by nonempty subsets of S. Each leaf is labeled by exactly the hypotheses compatible with its path, so these output classes partition S. Attaching an identifying tree for each class produces a complete identifying tree.

We suppress a query whenever its answer is already determined by the current candidate set. In particular, a line is never queried twice on one path. The resulting tree is *proper*: both children of every internal node contain hypotheses. A proper binary tree on n hypotheses has $n-1$ internal nodes and depth at most $n-1$: if I is its number of internal nodes, counting edges gives $2I = I+n-1$, so $I = n-1$ and every path has at most that many queries. This convention does not restrict the optimum: a query with only one nonempty outcome can be deleted, and repeated queries have already known answers.

A *transcript* records the queries and answers observed along a path. In a fixed binary tree the previous answers determine the next query, so the transcript can also be represented by its answer word, encoding negative as 0 and positive as 1. A *cached answer* is an answer already learned on that path; reusing it or inferring a constant answer costs no additional query. For nonempty $A \subseteq S$, the *restriction* $T|_A$ is obtained by deleting leaves outside A, deleting empty branches, and contracting queries with only one surviving outcome. It is a valid tree on A, and no surviving hypothesis's depth increases.

Definition 2.2 (Membership answers and spines)**.** A *membership interpretation* designates one answer class A of a supplied binary query. Its answer is a *hit* when $h \in A$ and a *miss* otherwise. A hit may be either affine sign; interpreting the answer changes neither the supplied line nor its cost.

The *spine* of a distinguished hypothesis is its root-to-leaf path. Another hypothesis *exits* that path at its first different answer; the attached subtree is its local exit code. If this occurs at query position j, a local depth d contributes total depth $j+d$. If every designated membership set excludes the distinguished hypothesis, its spine is an all-miss route and exits occur at first hits. A point p on no supplied line similarly determines a *witness path*, even when $p \notin \mathcal{H}$.

Let $d_T(h)$ be the number of queries on the path followed by h, stopping at its output leaf; for an identifying tree that leaf names h. For a tree T identifying a candidate set $S \subseteq \mathcal{H}$, define

$$c_{\mathrm{W}}(T) = \max_{h \in S} d_T(h).$$

For expected depth, let $\pi_h > 0$ with $\sum_h \pi_h = 1$, write $w(S) = \sum_{h \in S} \pi_h$, and define

$$c_{\mathrm{E}}(T) = \sum_{h \in S} \pi_h d_T(h).$$

The weights in $c_{\mathrm{E}}(T)$ are the original probabilities; they are not renormalized when a tree identifies a restricted state. Thus $c_{\mathrm{E}}(T)/w(S)$ is the conditional expected cost, whereas $c_{\mathrm{E}}(T)$ is that state's contribution to the original expectation. At the root, where $w(\mathcal{H}) = 1$, the two agree. Define the corresponding optima by

$$\mathrm{OPT}_{\mathrm{W}}(S) = \min_{T \text{ identifying } S} c_{\mathrm{W}}(T), \qquad \mathrm{OPT}_{\mathrm{E}}(S) = \min_{T \text{ identifying } S} c_{\mathrm{E}}(T).$$

We set both optima to zero on the empty set and on singletons. Hypothesis weights used later in the hardness proof specify these probabilities; they do not alter how queries are counted. Keeping weights unnormalized means that the expected costs of disjoint recursive states add directly.

More explicitly, if positive raw weights ω_h are supplied, put $\Omega = \sum_{h \in \mathcal{H}} \omega_h$ and $\pi_h = \omega_h/\Omega$. Then, also on a restricted state,

$$\sum_{h \in S} \omega_h d_T(h) = \Omega c_{\mathrm{E}}(T).$$

Thus normalizing weights divides both a weighted cost and its decision threshold by the same Ω, without changing which tree is optimal. Raw weights are not costs of individual queries: every query still costs one.

If S_ℓ^- and S_ℓ^+ are the two answer classes in S, the optimality recurrences on nonsingleton states are

$$\mathrm{OPT}_{\mathrm{W}}(S) = 1 + \min_{\ell \text{ splitting } S} \max\{\mathrm{OPT}_{\mathrm{W}}(S_\ell^-), \mathrm{OPT}_{\mathrm{W}}(S_\ell^+)\},$$
$$\mathrm{OPT}_{\mathrm{E}}(S) = w(S) + \min_{\ell \text{ splitting } S} \{\mathrm{OPT}_{\mathrm{E}}(S_\ell^-) + \mathrm{OPT}_{\mathrm{E}}(S_\ell^+)\}.$$

Algorithm 1 implements these recurrences on the small local states used later. These recurrences define the quantities used in the analysis; the approximation algorithms do not evaluate them. The first recurrence charges one query on every path and then takes the harder child. In the second, that query contributes the total state mass $w(S)$, and the two child contributions add because their hypothesis sets are disjoint.

Algorithm 1: Exact supplied-query Bellman recursion (small local states)

Input: State S; objective $o \in \{\mathrm{W}, \mathrm{E}\}$; original weights π_h.
Output: An optimal supplied-query tree and its cost.

if $|S| \le 1$ **then**
  return the leaf of cost zero
if *the memoized value for* (S, o) *exists* **then**
  return that value and tree
foreach *supplied line* ℓ *splitting* S **do**
  $(T_-, v_-) \leftarrow \textsc{ExactTree}(S_\ell^-, o)$; $(T_+, v_+) \leftarrow \textsc{ExactTree}(S_\ell^+, o)$
  $v_\ell \leftarrow 1 + \max\{v_-, v_+\}$ if $o = \mathrm{W}$, and $v_\ell \leftarrow w(S) + v_- + v_+$ otherwise
Memoize the minimum v_ℓ and the tree joining its stored children
return the memoized cost and tree

Algorithm 1 takes $2^{O(|S|)}\,\mathrm{poly}(m, L)$ time with bit-mask states. It is used only on bounded local sets; the general approximation algorithms never call it on the full input.

All logarithms are natural unless a base is displayed, and we use $0 \ln 0 = 0$, the continuous extension at zero. Coordinates, line coefficients, and probabilities are rational and explicitly given by binary-encoded numerators and denominators. Polynomial time means polynomial in this input's bit length, not in the magnitudes of its numerical values. The *decision version* of either objective asks, given an instance and a threshold K, whether a legal identifying tree has cost at most K. For maximum depth the threshold can be an integer; for expected depth it is rational and the prior is part of the input. An *α-approximation* for a minimization objective returns a feasible tree whose cost is at most α times the optimum on every valid input. A deterministic guarantee concerns the one tree returned, not an average over randomly chosen trees. Whenever an algorithm permits several choices, fix a deterministic tie-breaking rule, such as the input order.

Symbol	Meaning
$\mathcal{H}, \mathcal{L};\ n, m$	Hypothesis points, supplied lines, and their counts.
$S,\ S_\ell^-, S_\ell^+$	Current state and its negative and positive answer classes for line ℓ.
$d_T(h)$	Number of queries on the path for hypothesis h in tree T.
$c_{\mathrm{W}}(T),\ \mathrm{OPT}_{\mathrm{W}}(S)$	Maximum path depth and its minimum over identifying trees for S.
$\pi_h,\ w(S)$	Root prior probability of h and the original probability mass of state S.
$c_{\mathrm{E}}(T),\ \mathrm{OPT}_{\mathrm{E}}(S)$	Expected depth using the original probabilities and its minimum over identifying trees for S.
$\mathcal{E}(S)$	State-mass-weighted entropy of the conditional distribution on S.
$\Pi,\ \Delta_{\mathrm{E}}(S,\Pi),\ I(S,\Pi)$	A partition of S, the decrease in optimal expected depth, and the information gained from its class label.
$\omega_h,\ \Omega$	Optional raw positive weights and their total; $\pi_h = \omega_h/\Omega$.

Definition 2.3 (Balanced and flat states)**.** Fix $0 < \delta < 1/3$. A nonsingleton state S is *count-balanced* if some supplied line has at least $\delta|S|$ hypotheses on each side. Otherwise it is *count-flat*. In the flat case, every line has a unique *majority answer*, on the side containing more than half of S. We call this its *frozen answer* and write C_ℓ for the open half-plane on the opposite side. Its induced answer class $C_\ell \cap S$ is the *minority cap*, of size less than $\delta|S|$. We also use "cap" for the corresponding half-plane when discussing its geometry.

The expected-depth algorithm uses probability mass in place of cardinality: S is *mass-balanced* if some line has $w(S_\ell^-), w(S_\ell^+) \geq \delta w(S)$, and *mass-flat* otherwise. In a mass-flat state, freezing the mass-majority answer gives $w(C_\ell \cap S) < \delta w(S)$.

Throughout the pseudocode, tables contain only realized hypotheses or patterns, ties use input order, and a returned tree is contracted whenever a query has a constant answer. Cached answers, the origin marker, and infeasible branches therefore create no additional supplied query. The numerical parameters and their certificates are stated with the analysis of each routine.

Separate state quantities from calibration choices. The state S, its mass $w(S)$, pattern classes, and optimal costs describe the input or the progress of the algorithm. Greek parameters such as δ, β, γ specify construction choices. Their admissible conditions are stated where needed; their numerical values are chosen only after the structural proof. A subscripted coefficient such as Φ_β is a derived accounting function, not another independent tuning parameter.

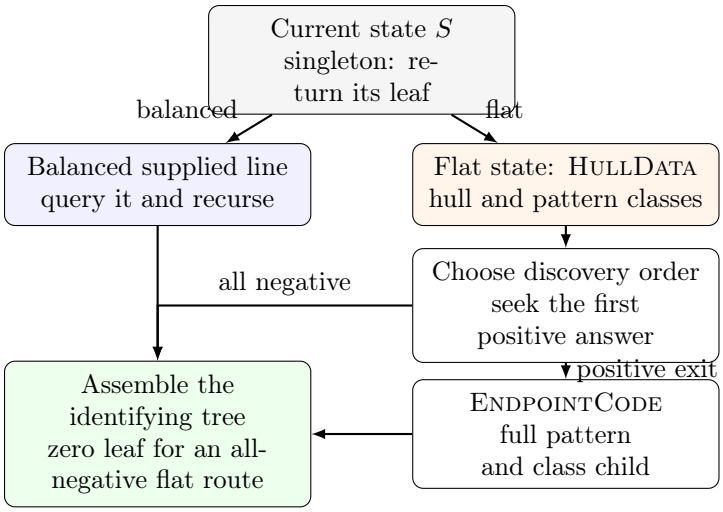


Figure 2: The common recursive structure. Arrows describe the output of one stage passed to the next. The maximum-depth routines construct class children before choosing the schedule, so their stored heights can set deadlines; expected-depth routines use original prior masses. The flat stage has an all-negative leaf only when the zero pattern is present. Scheduling and decoding use supplied hull lines.

Proof strategy.

The accounting facts below have three roles.

1. Prefix codes turn query transcripts into information lower bounds.
2. Restriction deletes queries that a known output class no longer needs.
3. Potential differences cancel across recursive states, so local charges are paid only once.

The statements are collected first. Their proofs appear together in the next technical subsection, before the geometric terminology.

2.2 Binary codes and ordered comparisons

Definition 2.4 (Binary codes and prefix codes)**.** A *binary word* is a finite sequence of bits from $\{0,1\}$; $\{0,1\}^*$ includes the empty word ε, of length zero. A *binary code* on a finite nonempty alphabet $\mathcal{A}$ is an injective map $c : \mathcal{A} \to \{0,1\}^*$. The word $c(a)$ is the *codeword* for a, and $\ell_a = |c(a)|$ is its length. A word u is a *prefix* of v if v starts with all the bits of u. A code is *prefix-free*, or a *prefix code*, if no codeword is a prefix of another distinct codeword.

For a distribution (q_a) on the alphabet, the code's maximum length is $\max_a \ell_a$ and its expected length is $\sum_a q_a \ell_a$. The prefix tree of a code has one node for each prefix of a codeword, with the empty prefix at the root and edges appending 0 or 1. For a prefix code its codewords are exactly the leaves, at depths ℓ_a. Conversely, the root-to-leaf answer words of any binary decision tree form a prefix code. Unary nodes, which have only one child, can be contracted without increasing any depth. The empty word is permissible only when the alphabet is a singleton. For example, $c(a) = 0$, $c(b) = 10$, and $c(c) = 11$ form a prefix code with lengths $1, 2, 2$.

Definition 2.5 (Alphabetic codes)**.** For an ordered alphabet $y_1 < \cdots < y_r$, a prefix code is *alphabetic* if its codewords occur in that same lexicographic order, with $0 < 1$. An *ordered threshold comparison* asks whether the unknown outcome is at most y_j for some j.

In an alphabetic prefix tree, the outcomes below each node form a consecutive range. After unary nodes are contracted, its two children separate that range at one threshold. Thus an alphabetic code can be implemented by ordered threshold comparisons whenever those comparisons are available. The code bits label the two ordered ranges; a line's raw sign answer is routed to the matching range, even if its sign encoding uses the opposite bit. This relabeling adds no query. An arbitrary prefix code need not be executable by the supplied line family. When we call an identifying decision tree a *code*, its internal queries must still be supplied lines. Section 3.4 verifies that the comparisons used for its endpoint codes are available.

A word $b_1 \cdots b_\ell$ specifies the half-open *dyadic cell*

$$D(b_1 \cdots b_\ell) = \left[\sum_{j=1}^{\ell} b_j 2^{-j}, \sum_{j=1}^{\ell} b_j 2^{-j} + 2^{-\ell} \right) \subseteq [0, 1).$$

The empty word specifies $[0, 1)$. Such cells are nested or disjoint; one word is a prefix of another exactly when its cell contains the other's cell. Half-open endpoints avoid ambiguity at dyadic rationals.

Theorem 2.6 (Kraft's inequality for binary prefix codes)**.** *Every finite binary prefix code has lengths satisfying*

$$\sum_{a \in \mathcal{A}} 2^{-\ell_a} \leq 1.$$

Conversely, nonnegative integer lengths $(\ell_a)_{a \in \mathcal{A}}$ are realizable by a binary prefix code if this inequality holds. The converse does not prescribe alphabetic order or guarantee implementation by a restricted query family.

The code $0, 10, 11$ has Kraft sum $2^{-1} + 2^{-2} + 2^{-2} = 1$. More generally, equality holds for a proper binary tree because its two child cells partition each internal node's cell. The coding facts here and the information inequalities below are standard; see Cover and Thomas [2].

2.3 Entropy and information inequalities

Definition 2.7 (Entropy and conditional entropy)**.** A *probability vector* (q_i) has nonnegative entries summing to one; its *support* consists of the indices with $q_i > 0$. For a finite random variable Y with probabilities $q_y = \Pr[Y = y]$, its Shannon entropy in natural-log units is

$$H(Y) = -\sum_{y} q_y \ln q_y.$$

We also write $H(q)$ for the entropy of a probability vector. The joint entropy $H(Y, Z)$ is the entropy of the pair (Y, Z). For Z with finite support, conditional entropy is

$$H(Y \mid Z) = \sum_{z : \Pr[Z=z] > 0} \Pr[Z = z]\, H(Y \mid Z = z),$$

where the inner entropy uses $\Pr[Y = y \mid Z = z] = \Pr[Y = y, Z = z] / \Pr[Z = z]$. Conditioning on several variables means conditioning on their tuple. Entropy in bits is $H(Y)/\ln 2$. For a function g of a finite random variable, $\mathbb{E}[g(Y)] = \sum_y q_y g(y)$. The *binary entropy* is $h(t) = -t \ln t - (1-t) \ln(1-t)$ for $0 \leq t \leq 1$. We write $H_2(t) = h(t)/\ln 2$ when binary entropy is measured in bits.

For nonempty S, the conditional hypothesis probabilities are $q_h = \pi_h / w(S)$. Define the *weighted state entropy* by

$$\mathcal{E}(S) = w(S)H(q) = \sum_{h \in S} \pi_h \ln \frac{w(S)}{\pi_h}, \qquad \mathcal{E}(\varnothing) = 0.$$

It is zero on singletons. Like c_{E}, this quantity includes the original state mass rather than renormalizing it away.

Lemma 2.8 (Log-sum inequality)**.** *For nonnegative a_i and positive b_i, with $A = \sum_i a_i > 0$ and $B = \sum_i b_i$, one has*

$$\sum_i a_i \ln \frac{a_i}{b_i} \geq A \ln \frac{A}{B},$$

with zero summands interpreted by continuity.

Lemma 2.9 (Entropy chain rule)**.** *For finite random variables,*

$$H(Y, Z) = H(Z) + H(Y \mid Z), \qquad H(Y, Z \mid W) = H(Y \mid W) + H(Z \mid Y, W).$$

If Z is determined by Y, then $H(Y) = H(Z) + H(Y \mid Z)$. Also, $0 \leq H(Y) \leq \ln |\operatorname{supp}(Y)|$ and $H(Y \mid Z) \geq 0$; in particular $0 \leq h(t) \leq \ln 2$.

Lemma 2.10 (Alphabetic threshold codes)**.** *Let Y have finite ordered support, and let $H(Y)$ be its entropy with natural logarithms. There is an explicit alphabetic threshold code with depth on outcome y at most $\log_2(1/\Pr[Y = y]) + 2$. In particular its expected depth is at most*

$$\frac{H(Y)}{\ln 2} + 2.$$

This is a dyadic prefix-code construction; see Cover and Thomas [2] for the general coding background. The short proof here also verifies the extra alphabetic (order-preserving) property needed for threshold queries. An optimal expected-depth threshold tree can also be computed by a polynomial interval dynamic program. The explicit code is used when the pointwise depth bound is needed.

Proof of Lemma 2.10. Claims 2.1 and 2.2 construct the stated executable code; averaging its pointwise bound proves the expected-depth inequality. Claim 2.3 gives the optimal threshold-tree algorithm. □

Algorithm 2: Alphabetic code on ordered outcomes

Input: Ordered outcomes $y_1 < \cdots < y_r$ of positive rational masses a_i; mode *pointwise* or *expected*.
Output: A threshold-comparison tree.

if $r = 1$ **then**
  return its zero-depth leaf
if *mode is* pointwise **then**
  Normalize $q_i = a_i / \sum_j a_j$ and put $F_i = \sum_{j<i} q_j$
  foreach i **do**
    Find $d_i = \lceil \log_2(1/q_i) \rceil + 1$ by rational powers-of-two comparisons
    Take the depth-d_i dyadic cell containing $F_i + q_i/2$ as codeword i
  Build the alphabetic prefix tree and contract unary nodes
else
  Set $D_{i,i} = 0$
  for $h = 2, \ldots, r$ **do**
    foreach $[i, j]$ *of length* h **do**
      $D_{i,j} \leftarrow \sum_{a=i}^{j} a_a + min_{i \le t < j}(D_{i,t} + D_{t+1,j})$
      Store a minimizing split after y_t
  Reconstruct the stored interval splits
return the ordered tree, implementing each split by its available threshold

The pointwise mode preserves the per-outcome bound; the expected mode has $O(r^3)$ arithmetic complexity and avoids long expansions for small priors. Their distinct roles are proved in Lemma 2.10.

2.4 Restriction and recursive accounting

Two lower bounds will account for different kinds of work. Restriction measures which queries are unnecessary once a class is known. Entropy measures the information learned from a class label.

Definition 2.11 (Partitions and refinement)**.** A *partition* Π of S is a family of pairwise disjoint nonempty subsets whose union is S. We say Π *refines* $\mathcal{R}$, or $\mathcal{R}$ is *coarser* than Π, if every class of Π is contained in a class of $\mathcal{R}$. The output classes of a partial decision tree form a partition.

Lemma 2.12. *For pairwise disjoint subsets $S_1, \ldots, S_k$ of a state S,*

$$\sum_i \mathrm{OPT}_{\mathrm{E}}(S_i) \le \mathrm{OPT}_{\mathrm{E}}(S), \qquad \mathrm{OPT}_{\mathrm{W}}(S_i) \le \mathrm{OPT}_{\mathrm{W}}(S).$$

Moreover,

$$\mathrm{OPT}_{\mathrm{E}}(S) \ge \frac{\mathcal{E}(S)}{\ln 2}, \qquad \mathrm{OPT}_{\mathrm{W}}(S) \ge \frac{\ln |S|}{\ln 2} \quad \textit{for nonempty } S.$$

Proof of Lemma 2.12. Claim 2.4 gives both restriction inequalities. Claims 2.5 and 2.6 supply the two information lower bounds. □

We call a state quantity used to account for progress a *potential*. For expected depth, the relevant potentials are $\mathrm{OPT}_{\mathrm{E}}$ and $\mathcal{E}$; their decrements are the parent value minus the sum of the child values. For a nonempty state S and a partition Π of it, write

$$\Delta_{\mathrm{E}}(S, \Pi) = \mathrm{OPT}_{\mathrm{E}}(S) - \sum_{A \in \Pi} \mathrm{OPT}_{\mathrm{E}}(A),$$

$$I(S, \Pi) = \mathcal{E}(S) - \sum_{A \in \Pi} \mathcal{E}(A) = \sum_{A \in \Pi} w(A) \ln \frac{w(S)}{w(A)}.$$

Let X be a random hypothesis with $\Pr[X = h] = \pi_h/w(S)$, and let P be its class label under Π. Then

$$I(S, \Pi) = w(S)H(P), \qquad H(P) = -\sum_{A \in \Pi} \frac{w(A)}{w(S)} \ln \frac{w(A)}{w(S)}.$$

Thus $I(S, \Pi)$ is the state mass times the Shannon entropy of the class label, measured in natural-log units. This is the partition form of the chain rule in Lemma 2.9. Both quantities are nonnegative. The first is the decrease in optimal expected depth when the class is revealed; the second is the weighted entropy of that class. The approximation proofs bound each stage by these local decreases. The following cancellation explains why this does not repeatedly charge the same optimal cost.

Proposition 2.13 (Local charges add to a global bound)**.** *Suppose a recursive partition ends in singletons, and its stage at state S costs at most $a\Delta_{\mathrm{E}}(S, \Pi_S) + cI(S, \Pi_S)$. The complete tree then has expected cost at most $a\,\mathrm{OPT}_{\mathrm{E}}(\mathcal{H}) + c\mathcal{E}(\mathcal{H})$.*

Proof. Sum the stage invoices. Every child's potential appears once with a minus sign at its parent and once with a plus sign at its own stage. Only the root values remain, because both potentials vanish on singleton leaves. □

2.5 Proofs of the coding and accounting facts

Proof of Theorem 2.6. For necessity, prefix-free codewords specify disjoint dyadic cells inside $[0, 1)$, whose widths are $2^{-\ell_a}$.

For sufficiency, reorder the lengths as $\ell_1 \le \cdots \le \ell_r$ and put $s_i = \sum_{j<i} 2^{-\ell_j}$. Each s_i is an integer multiple of $2^{-\ell_i}$ because every earlier length is at most ℓ_i. The intervals $[s_i, s_i + 2^{-\ell_i})$ are therefore dyadic cells of the prescribed depths. They are disjoint and lie in $[0, 1)$ by the assumed bound. Their binary descriptions give the required prefix code; assign them back to the original symbols. □

Proof of Lemma 2.8. Put $q_i = a_i/A$ and $r_i = b_i/B$. The inequality $-\ln x \ge 1 - x$ for $x > 0$ follows because $x - 1 - \ln x$ has its minimum zero at $x = 1$. Hence

$$\sum_{i:q_i>0} q_i \ln \frac{q_i}{r_i} \ge \sum_{i:q_i>0} (q_i - r_i) = 1 - \sum_{i:q_i>0} r_i \ge 0.$$

Multiplying by A and adding $A \ln(A/B)$ proves the assertion. □

Proof of Lemma 2.9. Substitute $\Pr[Y = y, Z = z] = \Pr[Z = z] \Pr[Y = y \mid Z = z]$ into the joint entropy sum. Taking its logarithm separates the two terms of the first identity. Apply the same identity under each value of W and average to obtain the conditional form. If Z is determined by Y, the pair (Y, Z) has the same entropy as Y. Entropy and conditional entropy are nonnegative because their summands are $-q \ln q \ge 0$ for $q \in [0, 1]$. Applying Lemma 2.8 to Y's distribution and the uniform distribution on its support gives the upper bound. □

Proof strategy.
For Lemma 2.10, place a small dyadic cell inside each probability interval. Disjoint cells preserve outcome order; their depths give the coding bound. A separate interval recurrence optimizes the executable threshold tree.

Claim 2.1 (Ordered disjoint cells give an alphabetic code)**.** *The midpoint cells are disjoint, occur in outcome order, and form an alphabetic prefix code.*

Proof. For an ordered outcome y of probability a, let $F_y = \sum_{y'<y} \Pr[Y = y']$. Its *cumulative-probability interval* is $[F_y, F_y + a)$, with midpoint $m_y = F_y + a/2$. The *midpoint-dyadic code* chooses the dyadic cell containing m_y at depth $\ell_y = \lceil \log_2(2/a) \rceil$, in the sense of the cells defined before Theorem 2.6. The cell's width is at most $a/2$, so the entire cell lies in the outcome's interval. These cells are disjoint and appear in outcome order. Their binary descriptions form a prefix-free alphabetic code. □

Claim 2.2 (Threshold implementation preserves the depth bound)**.** *Ordered threshold queries implement the code with depth at most* $\log_2(1/\Pr[Y = y]) + 2$ *on outcome* y*.*

Proof. Since $\lceil x \rceil \le x + 1$, their lengths satisfy

$$\left\lceil \log_2 \frac{2}{a} \right\rceil \le \log_2 \frac{1}{a} + 2.$$

After unary nodes are contracted, each internal node separates two consecutive outcome ranges and is a threshold comparison. Taking expectation of this pointwise bound gives

$$\sum_y \Pr[Y = y] \left(\log_2 \frac{1}{\Pr[Y = y]} + 2 \right) = \frac{H(Y)}{\ln 2} + 2.$$

□

Claim 2.3 (An interval recurrence minimizes expected depth)**.** *Optimal expected-depth threshold comparison uses a polynomial interval dynamic program.*

Proof. To minimize expected depth, let $D_{i,j}$ be the minimum weighted depth contributed by outcomes $y_i, \ldots, y_j$, using their original probabilities $p_a = \Pr[Y = y_a]$. Set $D_{i,i} = 0$. For $i < j$, a root threshold after y_t contributes $\sum_{a=i}^{j} p_a$, so

$$D_{i,j} = \sum_{a=i}^{j} p_a + \min_{i \le t < j} \{D_{i,t} + D_{t+1,j}\}.$$

This interval dynamic program uses $O(r^3)$ rational arithmetic operations. The resulting optimal alphabetic tree satisfies the expected-depth bound; no long binary expansion of a very small probability is required. It need not preserve the pointwise bound, which uses the explicit midpoint-dyadic construction. □

Proof strategy.
For Lemma 2.12, contract known-answer queries to obtain feasible subproblem trees. Then use the prefix transcripts in two ways: Kraft and log-sum bound average depth; the number of binary leaves bounds maximum depth.

Claim 2.4 (Restriction cannot increase depth)**.** *Restriction to pairwise disjoint sets cannot increase their total expected optimum or any maximum-depth optimum.*

Proof. Restrict an optimal tree on S to each S_i and contract its constant queries. No hypothesis's depth increases. Summing over disjoint subsets proves the expected-depth inequality; restricting a maximum-depth-optimal tree proves the second inequality.

□

Claim 2.5 (Entropy lower-bounds the expected invoice)**.** *Every identifying tree has expected cost at least* $\mathcal{E}(S)/\ln 2$.

Proof. For entropy, assume $S \neq \varnothing$; the empty-state case is immediate. Let $q_h = \pi_h / w(S)$. The leaf transcripts are a binary prefix code as in Definition 2.4: a transcript cannot continue beyond a leaf. Theorem 2.6 therefore gives $\sum_h 2^{-\ell_h} \leq 1$ for their lengths. Apply Lemma 2.8, dividing by $\ln 2$ to use base two, with $\ell_h = d_T(h)$ and $q_h = \pi_h / w(S)$:

$$\begin{aligned}\sum_h q_h \ell_h - \sum_h q_h \log_2 \frac{1}{q_h} &= \sum_h q_h \log_2 \frac{q_h}{2^{-\ell_h}} \\ &\geq \log_2 \frac{\sum_h q_h}{\sum_h 2^{-\ell_h}} \geq 0.\end{aligned}$$

The first inequality is the log-sum inequality, and the second uses $\sum_h q_h = 1$ and Kraft's bound. Hence every tree identifying S satisfies

$$c_{\mathrm{E}}(T) = w(S) \sum_h q_h d_T(h) \geq w(S) \sum_h q_h \log_2 \frac{1}{q_h} = \frac{w(S)}{\ln 2} \sum_h q_h \ln \frac{1}{q_h} = \frac{\mathcal{E}(S)}{\ln 2}.$$

Minimizing gives the expected-depth lower bound.

□

Claim 2.6 (Binary capacity lower-bounds height)**.** *A binary tree of height* d *has at most* 2^d *leaves, so identifying* S *requires height at least* $\log_2 |S|$.

Proof. For the maximum-depth bound, prove by induction on d that a binary tree of height d has at most 2^d leaves. The claim is immediate for $d = 0$; for $d > 0$, the root has at most two child subtrees, each of height at most $d - 1$, so together they have at most $2 \cdot 2^{d-1} = 2^d$ leaves. Thus $|S| \leq 2^d$, or $d \geq \log_2 |S| = \ln |S| / \ln 2$. □

2.6 Convex geometry and finite set systems

Definition 2.14 (Planar convexity)**.** A set is *convex* if it contains the segment joining any two of its points. A *convex combination* is a sum $\sum_j \lambda_j u_j$ with $\lambda_j \geq 0$ and $\sum_j \lambda_j = 1$. The *convex hull* $\mathrm{conv}(U)$ of a finite nonempty point set U is the set of its convex combinations. A *vertex*, or extreme point, cannot be expressed as a convex combination of other points of the hull. The hull is the convex hull of its vertices; set $\mathrm{conv}(\varnothing) = \varnothing$ by convention.

A nonempty planar hull has affine dimension zero, one, or two according as it is a point, a nontrivial segment, or has nonempty planar interior. In dimension two it is a convex polygon, whose vertices have a cyclic boundary order; its *edges* join consecutive vertices. A *supporting line* meets a convex set and leaves it entirely on one closed side. It *exposes* a point if that point is its unique intersection with the set. A finite point set is in *strictly convex position* if every point is a vertex of its hull.

An open half-plane has the form $\{x : a \cdot x > b\}$ for $a \neq 0$; replacing $>$ by $\geq$ gives its closed counterpart. A point is *strictly* on a chosen side when it is in that open half-plane, not on its boundary. A common strict *witness* for several chosen sides is a point in all their open half-planes; it need not itself be a hypothesis.

Theorem 2.15 (Planar Helly theorem)**.** *If every subfamily of at most three sets in a finite family of planar convex sets has nonempty intersection, then the whole family has nonempty intersection. The sets may be open.*

This is the standard planar form; see Matoušek [3].

Definition 2.16 (Covers and hitting sets)**.** A finite *set system* consists of a universe U, a candidate set V, and supplied subsets $F_v \subseteq U$ indexed by $v \in V$. A *cover* of U is a choice of these sets whose union contains every element of U. In the equivalent demand formulation, there is a candidate set V and a subset $D_h \subseteq V$ for each demand h. A *hitting set* is a subset $P \subseteq V$ with $P \cap D_h \neq \varnothing$ for every demand. Choosing v covers exactly $F_v = \{h : v \in D_h\}$. Thus covering demands by candidates and hitting their subsets are the same incidence problem. A demand with $D_h = \varnothing$ is uncoverable.

With positive demand weights, the weight covered by a choice is the sum of weights of demands hit at least once; repeated hits are not counted again. An *incidence table* records the bits $\mathbf{1}[h \in F_v]$, equivalently $\mathbf{1}[v \in D_h]$. A cardinality budget is a nonnegative integer k permitting at most k selected candidates. We may cap it at $|V|$, since there are no more distinct candidates to choose.

Definition 2.17 (Ordered and circular intervals)**.** On a finite linear order $v_1, \ldots, v_M$, an *ordinary interval* is a consecutive subset $\{v_i, \ldots, v_j\}$; we also allow the empty interval. Its nonempty intervals have first and last elements, called their endpoints. On a cyclic order, a *circular interval* is a consecutive block with wrap-around allowed, also permitting the empty and whole-cycle sets. Two vertices are *adjacent* when consecutive in this order, including v_M, v_1 on a cycle. An *oriented adjacent pair* specifies which of those two vertices is first. A *run* of P in a linear candidate order is a maximal nonempty consecutive block contained in P; $c(P)$ counts such runs, with $c(\varnothing) = 0$.

Cutting a cyclic order at a removed vertex turns any circular interval avoiding that vertex into an ordinary interval. When interval demands occur below, candidates are the ordered vertices, and a demand is hit by selecting a vertex in its interval, not by selecting the interval itself.

2.7 Query orderings and first-cover costs

Let a universe U have positive element weights π_h and supplied sets F_v covering U. These weights need not sum to one. A *cover ordering* $Q = (F_{v_1}, \ldots, F_{v_\ell})$ is a sequence of distinct supplied sets whose union covers U, and $|Q| = \ell$ is its length. The *first-cover time*, or *latency*, of h is $t_Q(h) = \min\{j : h \in F_{v_j}\}$. The *weighted min-sum set cover* objective is

$$\mathrm{MSSC}(Q) = \sum_{h \in U} \pi_h t_Q(h).$$

Equivalently, $\mathrm{MSSC}(Q)$ is the sum of the uncovered mass before each step:

$$\mathrm{MSSC}(Q) = \sum_{j=0}^{|Q|-1} w\left(U \setminus \bigcup_{r=1}^{j} F_{v_r}\right), \qquad w(E) = \sum_{h \in E} \pi_h.$$

The union for $j = 0$ is empty. *Maximum-remaining-weight greedy* repeatedly chooses the supplied set containing the largest total weight of elements not yet covered, counting each element only once.

2.8 Normalized queries and hull patterns

Definition 2.18 (Witness-relative query notation)**.** Enumerate the supplied lines as $\ell_1, \ldots, \ell_m$. For a state S and a point p on no supplied line, put

$$x_h = h - p, \qquad u_i = -\frac{a_{\ell_i}}{f_{\ell_i}(p)}, \qquad C_i = \{y : u_i \cdot (y - p) > 1\}.$$

Thus x_h is a translated hypothesis vector, u_i a normalized query vector, and C_i the cap opposite p in the original plane. The corresponding dual demand is

$$R_h = \{u : u \cdot x_h > 1\}.$$

Write $K = \mathrm{conv}(\{0\} \cup \{u_i : i \in [m]\})$ and let V be its nonzero vertices, in cyclic hull order. Every $v \in V$ equals a supplied query vector; choose one representative line if several coincide, and put $C_v = \{y : v \cdot (y - p) > 1\}$. A *support block* $J_i \subseteq V$ represents u_i as $u_i = \sum_{v \in J_i} \alpha_{iv} v$, with positive coefficients and $\sum_v \alpha_{iv} \leq 1$. We choose blocks of at most two vertices, adjacent when there are two; their existence is proved in Lemma 3.2.

The origin is a known-negative marker, not a query, since $0 \cdot x_h = 0$. If it lies on the hull boundary, it can be inserted at its boundary position as a free cut marker. The hull K, its vertices, and their cyclic order can be computed in polynomial time by standard planar convex-hull algorithms [4].

Indeed,

$$u_i \cdot x_h - 1 = -\frac{f_{\ell_i}(h)}{f_{\ell_i}(p)}.$$

Consequently, $h \in C_i$ exactly when $u_i \in R_h$, and equality $u_i \cdot x_h = 1$ means h lies on ℓ_i. We encode the cap-side answer as 1 and the witness-side answer as 0. A *positive hull answer* is this cap-side bit, regardless of the original affine sign. This relabeling introduces no new query. If $x_h = 0$, the demand R_h is empty. Intersections such as $C_i \cap S$ always contain the original hypotheses h, not their translated vectors.

Definition 2.19 (Hull patterns and pattern classes)**.** The *hull pattern* of $h \in S$ is the bit vector

$$\sigma(h) = \big(\mathbf{1}[v \cdot x_h > 1]\big)_{v \in V}.$$

A *pattern class* consists of all hypotheses with the same vector, and Π is the partition into these classes. The *positive vertices* are the coordinates whose bits are one. Write A_0 for the class with every bit zero, taking $A_0 = \varnothing$ when no hypothesis has that pattern. When a prior is specified, the nonzero-pattern mass is $b(S) = w(S) - w(A_0)$, the mass on which a positive hull answer is possible.

A *discovery schedule* lists individual hull tests in their query order along the all-negative route. Its *first-hit time* g_A for a nonzero pattern class is the index of the first listed vertex positive on that class.

3 Common-witness geometry and interval tools

Using the notation of Definition 2.18, we prove the structural facts shared by both algorithms. Algorithm 3 records their common preprocessing. Flat states admit a common majority witness, each normalized query has at most two adjacent hull supports, and each hypothesis has one circular positive interval. The interval tools below cover these patterns and recover their endpoints without adding queries.

Proof strategy.

1. Flatness makes every three majority half-planes intersect; Helly's theorem gives a common strict witness.
2. Radial projection onto the normalized query hull supplies adjacent supports; a half-plane reads one circular positive interval.
3. Interval covers discover a positive vertex, after which a query-free cut turns supplied hull tests into endpoint comparisons.

Symbol	Meaning
$p,\ x_h,\ u_i$	Strict common witness, translated hypothesis vector $h - p$, and normalized vector for supplied line ℓ_i.
$C_i,\ R_h$	Original-plane cap opposite p for query i, and the dual set of normalized queries positive on h.
$K,\ V$	Convex hull of the normalized query vectors together with 0, and its nonzero vertices in cyclic order.
$J_i,\ \alpha_{iv}$	At most two adjacent support vertices for u_i and their nonnegative coefficients, whose sum is at most one.
$\sigma(h),\ \Pi$	Hull-test bit pattern of h and the partition into equal-pattern classes.
$A_0,\ b(S)$	All-zero pattern class (empty if absent) and, with priors, the mass outside that class.
g_A	First positive test position for a nonzero pattern class A in a discovery schedule.

3.1 A common majority witness

Lemma 3.1 (Common witness)**.** *In either kind of flat state, a point p strictly inside every majority half-plane can be computed in polynomial time.*

Proof. Flatness forces every three majority sides to intersect. Helly gives a common strict point, and a positive-margin linear program finds a rational one. The two claims below justify these steps. □

3.2 Adjacent supports and circular patterns

Lemma 3.2 (Hull support and pattern structure)**.** *The support blocks of Definition 2.18 can be chosen with at most two real hull vertices, adjacent when there are two. For every query,*

$$C_i \cap S \ \subseteq \ \bigcup_{v \in J_i} (C_v \cap S). \tag{1}$$

Negative support bits force the query's witness-side answer. The positive vertices of every hypothesis form a circular interval, $|A_0| \le 1$, and every nonzero pattern class lies in a positive hull cap.

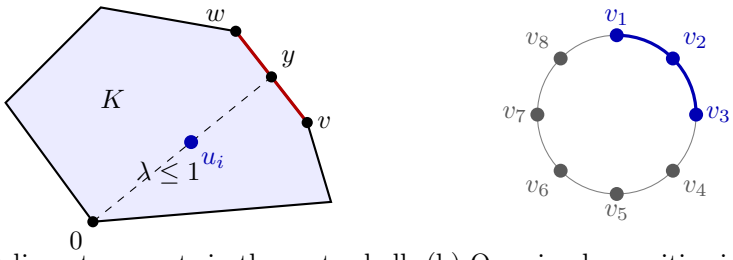
Proof. Project each query vector radially to a hull edge. Its adjacent endpoints dominate that query's cap. A half-plane intersects the hull boundary in one circular interval; separation then makes the all-zero class a singleton at most. These three facts are proved in the technical subsection. □

Algorithm 3: HULLDATA: a flat state's witness, supports, and patterns

Input: Flat state S and its count or mass measure; supplied lines.
Output: Strict witness p, full hull K, real vertices V, supports J_i, partition Π.

Orient every majority inequality as $a_i \cdot p - b_i > 0$
Maximize t subject to $a_i \cdot p - b_i \ge t$ and $0 \le t \le 1$; retain a solution with $t > 0$
Compute $u_i = -a_{\ell_i} / f_{\ell_i}(p)$ and $K = \mathrm{conv}(\{0\} \cup \{u_i\})$
Keep one supplied representative of each real hull vertex in cyclic order
foreach u_i **do**
    Extend its ray from 0 to the last point y in K
    Express y on its boundary edge and scale the coefficients by $\|u_i\|/\|y\|$; omit the origin and zero coefficients to form J_i
Compute all bits $\mathbf{1}[v \cdot (h - p) > 1]$ and group equal rows into Π
return $(p, K, V, (J_i), \Pi)$, retaining the origin's boundary position

The ray scaling is obtained from rational coordinates, without a square-root computation. In dimension at most one use segment endpoints as supports. Lemmas 3.1 and 3.2 prove feasibility and the support invariant.



(a) Adjacent supports in the vector hull (b) One circular positive interval

Figure 3: The first panel depicts normalized query vectors, not hypotheses: y lies on the edge vw and $u_i = \lambda y$ for $0 < \lambda \le 1$. Writing $y = \theta v + (1-\theta)w$ gives $u_i = \alpha v + \beta w$ with $\alpha + \beta = \lambda \le 1$. Consequently, if both support values on a hypothesis are at most one, then $u_i \cdot x_h \le 1$; equivalently, $C_i \subseteq C_v \cup C_w$. The second panel shows the cyclic order only; the positive hull tests of a fixed hypothesis form one interval in that order.

Figure 3 separates the two geometric roles: panel (a) shows how adjacent hull vertices support an original query vector, while panel (b) records only the cyclic order of the hull tests seen positively by one hypothesis.

3.3 Interval hitting and discovery length

Lemma 3.3 (Minimum interval cover)**.** *A minimum-cardinality hitting set for ordinary or circular intervals on a finite candidate order is computable in polynomial time. An empty demand is infeasible; an empty family needs no candidates.*

Proof. On a line, an exchange argument permits choosing the right endpoint of the earliest-ending demand. On a circle, enumerate one forced hit and cut there; the remaining problem is linear. The detailed exchange argument is given below. □

Lemma 3.4 (Greedy cover length with adjacent-run credit)**.** *Let a nonempty family of positively weighted interval demands be covered by a candidate set P of size k. Let $c(P)$ be the number of runs of consecutive candidates in P, measured in the entire candidate order. Maximum-remaining-weight greedy finishes within $k + c(P) - 1$ choices, for every tie rule. In particular, a minimum cover of size k gives the usual bound $2k - 1$. For circular intervals the corresponding bound is $2k$.*

The extra term is the number of runs beyond the first. A scattered cover may force greedy to spend choices repairing several gaps, whereas a cover arranged in a few contiguous blocks has fewer such gaps to repair.

Proof strategy.

Charge greedy to the cover's size plus its number of runs. An extreme choice removes a cover point; an interior choice separates the unhit intervals into two independent sides. Both operations reduce that charge enough to pay for the current query.

Proof. Induct on the comparison cover size. An extreme greedy choice removes an extreme cover point. An interior choice splits the remaining interval demands into independent sides whose combined size-plus-run charge does not increase. The two claims below make this induction precise. A first circular choice reduces the circular case to the linear case. □

3.4 Recovering the interval endpoints

Lemma 3.5 (Query-free endpoint representation)**.** *After one positive hull vertex v is known, a cut computable without queries represents every compatible positive pattern as an ordinary interval containing v. Supplied hull tests compare its endpoint coordinates. A cached negative neighbor fixes one endpoint; two negative neighbors fix the entire pattern.*

Proof strategy.
Use a cached negative neighbor as the cut when available. Otherwise the origin and an opposite hull edge give a cut that no compatible positive interval can cross. All later comparisons are actual hull tests, so no new line or anchor query is introduced.

Proof. A cached negative neighbor supplies a cut and pins one endpoint. Otherwise an opposite hull edge gives a cut without asking a query. In the resulting linear order, supplied hull tests are endpoint comparisons. The three claims below establish legality, including degenerate hulls. □

A positive discovery exit is *pinned* if a neighbor of its discovered vertex has a cached negative answer. It is *singly pinned* when exactly one neighbor is known negative, and *doubly pinned* when both are. Pinning fixes one or both endpoints, as in Lemma 3.5; it introduces no additional query.

Algorithm 4: EndpointCode: legal full-pattern recovery

Input: A positive hull vertex v, compatible patterns, cached answers; code mode; optionally fixed continuation heights h_A.
Output: A supplied-query decoder whose leaves are full pattern classes.

if *the hull has dimension at most one* **then**
  return the already determined nonzero pattern
if *v has a cached negative neighbor* **then**
  Cut at that neighbor and orient so v is the first positive vertex
else
  Cut at a boundary origin, or at the opposite edge met by the ray $v0$
List the realized left and right endpoint pairs in the cut order
if *no continuation heights are supplied* **then**
  Search each unfixed endpoint using Algorithm 2; condition the second search on the first answer
else
  foreach *nonempty realized endpoint rectangle U, in increasing size* **do**
    if *U has a single realized class A* **then**
      $E(U) \leftarrow h_A$
    else
      $E(U) \leftarrow \min_\ell(1 + \max\{E(U \cap \ell^-), E(U \cap \ell^+)\})$, over proper supplied endpoint thresholds
  Reconstruct the minimizing rectangle splits, retaining full-pattern leaves and their recorded continuation heights
return the decoder; skip fixed endpoints and cached thresholds

Lemma 3.5 proves the cut is legal without an anchor query. With continuation heights this is the complete-exit optimization of Section 8; its $O(M^4)$ rectangles and $O(M)$ thresholds give polynomial running time. The decoder is returned with pattern leaves; the recursive algorithm attaches each stored child once at its corresponding leaf. The values $E(U)$ include those continuations for height comparison only.

3.5 Proofs of the geometric and interval facts

The conceptual bottleneck is the conversion from planar caps to executable endpoint comparisons. Radial projection gives a *cover implication*, not equivalence of a cap with its supports. The proofs keep that distinction explicit. The endpoint cut is deduced from geometry or cached answers, so it costs no additional query.

Proof of Lemma 3.1.

Claim 3.1 (Three majority half-planes have a common strict point)**.** *Every subfamily of at most three majority half-planes intersects.*

Proof. Use the measure defining flatness, either cardinality or probability. The union of any three minority caps has measure less than 3δ times the measure of S, and therefore omits a hypothesis. That hypothesis belongs strictly to all three majority half-planes, because no hypothesis lies on a query line. The same argument covers subfamilies of fewer than three lines. Thus every subfamily of at most three majority half-planes has nonempty intersection. Theorem 2.15 implies that the whole family has nonempty intersection. □

Claim 3.2 (A positive margin gives a rational strict witness)**.** *The common point can be found with polynomial encoding length.*

Proof. To obtain a strict rational witness, orient each majority inequality as $a_i \cdot p - b_i > 0$ and maximize t subject to $a_i \cdot p - b_i \geq t$ for every i and $0 \leq t \leq 1$. The open intersection supplies a feasible positive t. This is a *linear program*: its objective and constraints are linear in the unknown coordinates of p and the margin t. Rational linear programming therefore returns a strict witness with polynomial encoding length [5]. □

□

Proof of Lemma 3.2.

Claim 3.3 (Radial boundary projection needs only adjacent supports)**.** *Each normalized vector has a nonnegative support expression whose coefficients sum to at most one.*

Proof. Extend the ray from 0 through u_i to its last point y in K. If y lies on an edge with endpoints v, w, write $y = \theta v + (1-\theta)w$ and $u_i = \lambda y$, with $0 < \lambda \leq 1$. The resulting coefficients are nonnegative and sum to $\lambda \leq 1$. Delete origin endpoints and zero coefficients to obtain J_i. A boundary vertex needs only itself. In dimension at most one, the last ray point is a segment endpoint, so the same conclusion holds directly. □

Claim 3.4 (Negative support answers force the original answer)**.** *The hull-cap inclusion* (1) *holds.*

Proof. If every support has $v \cdot x_h \leq 1$, then $u_i \cdot x_h = \sum_{v \in J_i} \alpha_{iv}(v \cdot x_h) \leq \sum_v \alpha_{iv} \leq 1$. This proves (1). Equality cannot occur at a hypothesis because it would put it on the original supplied line. The origin has dot product zero, so its bit is always negative and it is never queried. □

Claim 3.5 (The hull patterns are intervals with at most one zero point)**.** *Every nonzero class belongs to a supplied positive hull cap.*

Proof. The positive vertices are $V \cap R_h$. A half-plane selects a consecutive block of a convex polygon's vertices. Removing the known-negative origin, if it is a vertex, leaves a circular interval on V. Degenerate hulls have at most two real vertices and satisfy the same assertion. An all-zero pattern forces every original query to have the witness's answer; two such hypotheses would contradict separability. Finally, a nonzero class has some positive vertex v, and all its hypotheses belong to $C_v \cap S$. □

□

Proof of Lemma 3.3. For ordinary intervals, choose the right endpoint q of an earliest-ending interval I. Any cover has a point $u \in I$. Replacing u by q loses no demand containing u: its right endpoint is at least q and its left endpoint is at most $u \le q$. Thus an optimal cover can choose q. Delete the intervals hit there and repeat, proving the endpoint greedy rule is exact.

For circular intervals, choose a nonempty demand and enumerate a forced candidate v in it. Delete demands hit by v and cut the cycle there. All remaining demands are ordinary intervals, so use the preceding rule and add v. Some forced choice belongs to an optimal circular cover; take the smallest resulting solution. □

Proof of Lemma 3.4. Induct on k. When $k = 1$, its sole candidate hits every demand, and any greedy maximum does so as well. Let v be the first greedy choice.

Claim 3.6 (An extreme greedy choice removes an extreme cover point)**.** *The remaining demands admit the cover $P \setminus \{u\}$ with no increase in run count.*

Proof. If it lies at or outside an extreme of P, let u be the nearest extreme cover point. For example, if $v < \min P = u$, every demand containing v must also meet P, and hence must contain u. Thus the demands hit by v are a subset of those hit by u. Greedy maximality gives the reverse inequality on their total weights, so the two candidates hit the same positive-weight demands. Thus $P \setminus \{u\}$ covers the remainder, and removing an extreme does not increase its run count. Induction plus the first query gives at most $k + c(P) - 1$ choices. □

Claim 3.7 (An interior choice creates independent left and right problems)**.** *The sum of their size-plus-run charges is at most $k + c(P)$.*

Proof. Otherwise v lies strictly between the extremes of P. The unhit intervals lie entirely on one side of v. The parts P_-, P_+ strictly to its left and right cover these two families, and

$$|P_-| + c(P_-) + |P_+| + c(P_+) \le k + c(P).$$

If $v \notin P$, cutting in its gap leaves the total run count unchanged. If $v \in P$, removing it reduces size by one and can increase total run count by at most one, which proves the displayed inequality in either case. Greedy may interleave the two sides, but a choice on one side covers no interval on the other; hence the choices restricted to either side are still a greedy sequence for that side. If both families are nonempty, induction plus the first query gives

$$1 + \big(|P_-| + c(P_-) - 1\big) + \big(|P_+| + c(P_+) - 1\big) \le k + c(P) - 1.$$

If only one is nonempty, the other cover part still has size plus run count at least two, so the same bound holds. If neither remains, one query suffices. □

Finally $c(P) \leq k$, giving $2k - 1$ without adjacency information.

For circular intervals, make the first greedy choice and cut the cycle there. Every unhit interval becomes an ordinary interval. The original k cover candidates still cover these demands, so at most $2k - 1$ further choices are needed. □

Proof of Lemma 3.5.

Claim 3.8 (A cached negative neighbor pins an endpoint)**.** *Only the opposite endpoint remains to be learned.*

Proof. After discovery, one positive hull vertex v is known. First inspect its cached neighbor answers. If a neighbor is known to be negative, cut at that vertex and orient the order so that v is first. The positive interval starts at v, so only its other endpoint needs a search. If both neighbors are negative, the pattern consists of v alone and no endpoint query is needed. □

Claim 3.9 (The opposite-edge cut costs no query)**.** *Without a cached cut, the normalized hull supplies a cut that no compatible positive interval can wrap around.*

Proof. Otherwise use a geometric cut. If the origin lies on the hull boundary, cut at its free negative marker. If it is interior, the ray from v through 0 meets an opposite hull edge with consecutive endpoints a, b. The origin is a convex combination of v, a, b, with positive coefficient on v. Thus a, b cannot both be positive: if all three dot products with x_h exceeded one, their convex combination would also exceed one, contradicting $0 \cdot x_h = 0$. Cut the cyclic order between a and b. A circular positive interval could wrap around this cut only if both endpoints were positive, which was just excluded. If the ray meets a vertex, that vertex is necessarily negative and is itself a free cut point. □

Claim 3.10 (Endpoint comparisons are supplied hull queries)**.** *Both endpoint searches, including their degenerate cases, are legal.*

Proof. No anchor query is made. The positive vertices now form an ordinary interval containing v. To its left, querying a vertex asks whether the left endpoint is at or before that vertex. To its right, querying a vertex asks whether the right endpoint is at or after it. Thus supplied hull queries give exactly the ordered comparisons needed to locate the two endpoints.

When neither endpoint is already fixed, learn the left endpoint, then the right endpoint conditioned on the left endpoint and the earlier transcript. For consecutive possible left endpoints, querying the earlier endpoint separates them; for consecutive possible right endpoints, querying the later endpoint separates them. These are real hull vertices, so all comparisons used by the codes are supplied queries. The conditional endpoint distributions come from the hypothesis table.

If K has dimension at most one, it has at most two real vertices. With two real endpoints, one positive endpoint forces the other negative because 0 lies between them. With one real vertex, its positive bit is the complete nonzero pattern. Thus no further query is needed in either degenerate case. □

□

4 Maximum-depth scheduling ingredients

The maximum-depth algorithm of Section 8 constructs its pattern children first and then schedules their discovery. This section isolates its local tools. The priority scheduler is an internal fallback: it guarantees a passing height trial even when the exact scheduler's logarithmic cap is too small. We do not use it as a separate approximation algorithm. No prior probabilities are used by this maximum-depth construction.

Let $\delta \in (0, 1/3)$ be the count-balance threshold and let $\gamma > 2$ be the coefficient of the count potential $\Phi(s) = \lfloor \gamma \log_2 s \rfloor$. Assume

$$\gamma \log_2 \frac{1}{1-\delta} > 1, \qquad \gamma \log_2 \frac{1}{\delta} > 5. \tag{2}$$

The first inequality pays for a balanced query; the second pays for endpoint recovery at a flat state. Their calibrated rational values are selected only after the main proof, in Section 8.3.

4.1 Local credits and the priority fallback

Proof strategy.

1. Restrict an optimal tree to a pattern class: its common witness prefix disappears and gives that class a discovery deadline.
2. Recover the pattern with supplied endpoint comparisons; the count-potential drop pays for this decoder.
3. Use priorities to meet all deadline prefixes in one ordering. Every accepted ordering is checked against actual stored child heights.

In a count-flat state S of size s, use the common witness and hull patterns of Lemmas 3.1 and 3.2. Every nonzero class A has $a = |A| < \delta s$; the zero class A_0 is empty or a singleton. Write

$$z_A = \lfloor \log_2(\delta s / a) \rfloor.$$

All child heights below are the heights of already constructed codes, not optima supplied to an algorithm.

Proposition 4.1 (Integer count credit)**.** *A balanced child of size $a \le (1-\delta)s$ satisfies $\Phi(s) - \Phi(a) \ge 1$. A nonzero flat-state class satisfies*

$$\Phi(s) - \Phi(a) \ge \lfloor \gamma \log_2(s/a) \rfloor \ge z_A + 5. \tag{3}$$

Proposition 4.2 (Supporting-prefix credit)**.** *Follow a common witness in a proper optimal tree of height $D = \mathrm{OPT}_W(S)$ along a path of length r. Let t_A be the first path query whose real hull supports meet the positive pattern of A; put $t_A = r + 1$ if none does. Then*

$$\mathrm{OPT}_W(A) \le D - (t_A - 1).$$

The supporting singleton or adjacent-pair blocks cover every nonzero class by its index t_A, after appending one positive support for an uncovered terminal nonzero singleton when necessary.

Proposition 4.3 (Legal endpoint allowances)**.** *After a first positive hull answer, the pointwise alphabetic endpoint codes recover class A with at most*

$$e_A^{(4)} = z_A + 4$$

additional supplied queries. If a neighboring hull vertex is already known negative, one endpoint is fixed and the allowance is $e_A^{(2)} = z_A + 2$. A hull of dimension at most one needs no further query after discovery.

Proposition 4.4 (Local route-height certificate)**.** *Let h_A be a stored class code's actual height. Discover a positive vertex in a fixed ordering truncated when all nonzero patterns are covered. If its first positive position for A is g_A, the whole route through A has height at most $g_A + h_A + e_A^{(4)}$, or $g_A + h_A + e_A^{(2)}$ on a pinned exit. If every such bound is at most H, the whole tree has height at most H, including its zero route when that class exists.*

Proposition 4.5 (One ordering for all block deadlines)**.** *Force an oriented adjacent real pair (v, w) first. Consider only classes negative on both, with integer block deadlines $d_A \geq 2$. Suppose a comparison sequence begins with $\{v, w\}$ and hits A within d_A singleton or adjacent-pair blocks. Sort the distinct budgets as $d^{(1)} < \cdots < d^{(r)}$, give a deadline-$d^{(j)}$ class weight $(s+1)^{r-j}$, and complete by maximum-remaining-weight greedy. This polynomial-time ordering satisfies $g_A \leq 3d_A - 2$ for every surviving class simultaneously.*

Proposition 4.6 (Priority fallback height)**.** *Suppose stored pattern children satisfy the fallback bound*

$$h_A \leq 3\,\mathrm{OPT}_W(A) + \Phi(|A|).$$

At a flat state, Algorithm 5 has a passing trial at

$$H^* = 3\,\mathrm{OPT}_W(S) + \Phi(s).$$

A balanced query also fits this height if its two stored children obey the same bound. Consequently the hybrid recursion can always retain $C_W(S) \leq 3\,\mathrm{OPT}_W(S) + \Phi(s)$ as its fallback invariant.

For a trial H and a forced pair, first-positive exits must pass $1 + h_A + e_A^{(4)} \leq H$, and second-query exits must pass $2 + h_A + e_A^{(2)} \leq H$. Classes negative on both receive

$$u_A = h_A + e_A^{(4)}, \qquad B_A = H - u_A, \qquad d_A = \lfloor (B_A + 2)/3 \rfloor.$$

Reject any $B_A < 4$ and accept only after checking the resulting actual first-hit positions against $g_A + u_A \leq H$. These tests describe an executable height certificate; the optimal comparison blocks are used only to prove that some trial passes.

Algorithm 5: PRIORITYTRIAL: internal fallback for one height trial

Input: Flat state's hull and classes; child heights h_A; allowances $e_A^{(4)}, e_A^{(2)}$; target H.
Output: A passing discovery order, or infeasible.

foreach *oriented adjacent real pair* (v, w) **do**
  Form $\mathcal{F}, \mathcal{S}, \mathcal{N}$ by their first hit on (v, w)
  if *some* $A \in \mathcal{F}$ *has* $1 + h_A + e_A^{(4)} > H$, *or some* $A \in \mathcal{S}$ *has* $2 + h_A + e_A^{(2)} > H$ **then**
    continue
  foreach $A \in \mathcal{N}$ **do**
    $u_A \leftarrow h_A + e_A^{(4)}$; $B_A \leftarrow H - u_A$; $d_A \leftarrow \lfloor (B_A + 2)/3 \rfloor$
  if *some* $B_A < 4$ **then**
    continue
  Sort distinct budgets as $d^{(1)} < \cdots < d^{(r)}$
  Give each deadline-$d^{(j)}$ class weight $(|S| + 1)^{r-j}$
  Force v, w and complete by maximum-remaining-weight greedy; truncate when all nonzero classes are hit
  if *every* $A \in \mathcal{N}$ *has actual* $g_A + u_A \leq H$ **then**
    return this order
return infeasible

4.2 Technical proofs of the local ingredients

Proof of Proposition 4.1. Use $\lfloor x \rfloor - \lfloor y \rfloor \geq \lfloor x - y \rfloor$. The balanced statement follows from the first inequality in (2). In a flat class put $x = \log_2(s/a)$ and $\ell = \log_2(1/\delta)$. Since $x > \ell$, $\gamma \geq 1$ and $\gamma\ell > 5$ imply $\gamma x > 5 + (x - \ell)$. Taking floors gives the second statement. □

Proof of Proposition 4.2.

Claim 4.1 (The common prefix disappears under restriction)**.** *Every member of A follows the witness's first $t_A - 1$ answers.*

Proof. Before that index all supporting hull bits vanish throughout A. The convex support identity forces the original query's witness-side answer. Restricting the optimal tree to A contracts these constant queries, giving $\mathrm{OPT}_W(A) \leq D - (t_A - 1)$. □

Claim 4.2 (The supporting blocks form a comparison cover)**.** *Only an uncovered terminal nonzero singleton needs an appended block.*

Proof. Each class with $t_A \leq r$ is hit by that path query's supports. An unhit class follows the entire original path and is therefore the single leaf hypothesis reached by the witness. If its pattern is nonzero, append a positive hull vertex at index $r+1$; if it is zero, no positive hit is needed. Repeated supports do not invalidate any first-hit deadline. □

□

Proof of Proposition 4.3. The query-free cut of Section 3.4 makes the positive pattern an ordinary interval. Let R be the hypotheses compatible with the first positive transcript, so $|R| < \delta s$. Use uniform counts, conditioned on that transcript, as code-design probabilities. They do not randomize the algorithm.

Claim 4.3 (Two endpoint searches telescope their probabilities)**.** *If a_L compatible hypotheses have the target's left endpoint, its full pattern is learned in fewer than $\log_2(\delta s/a) + 4$ queries.*

Proof. Lemma 2.10 gives the pointwise bound

$$\log_2 \frac{|R|}{a_L} + 2 + \log_2 \frac{a_L}{a} + 2 = \log_2 \frac{|R|}{a} + 4 < \log_2 \frac{\delta s}{a} + 4.$$

The second distribution is conditioned on the learned first endpoint; the endpoint pair uniquely specifies the pattern. Query counts are integers, so the claimed $z_A + 4$ allowance follows. □

Claim 4.4 (A cached negative neighbor removes one search)**.** *A pinned positive vertex fixes one interval endpoint.*

Proof. Cut at the negative neighbor and orient the order to place the positive vertex first. One alphabetic search learns the other endpoint in fewer than $\log_2(\delta s/a) + 2$ queries. The integer allowance is $z_A + 2$. The negative neighbor is a real cached answer, so no anchor query is added. In dimension at most one the positive pattern is already determined; inference and cached answers can only lower cost. □

□

Proof of Proposition 4.4. Add discovery, endpoint recovery, and the stored child height. The all-negative route has length at most $\max_{A \neq A_0} g_A \leq H$ and ends at the zero singleton. If that class is absent its leaf is omitted. A nonsingleton state has a nonzero pattern because $|A_0| \leq 1$. □

Proof of Proposition 4.5.

Claim 4.5 (Later deadlines cannot change a prefix greedy choice)**.** *The global ordering is weighted greedy for every surviving deadline prefix until that prefix is covered.*

Proof. For prefix j, scores from classes with deadlines at most $d^{(j)}$ are integer multiples of $(s+1)^{r-j}$. All later classes together weigh less than this unit: there are at most s classes, each weighing at most $(s+1)^{r-j-1}$. They cannot reverse a strict prefix-score comparison. Ties remain legitimate prefix greedy ties, and a positive prefix score beats a zero score. The same argument holds for every prefix in the one global ordering. □

Claim 4.6 (The first pair leaves at most $3d-4$ later greedy choices)**.** *A nonempty deadline-d prefix finishes by actual position $3d-2$.*

Proof. Cut at v and delete its adjacent vertex w. Surviving demands are ordinary intervals. Remove these two vertices from the union P of the first d comparison blocks. Each of the other $d-1$ blocks contributes at most three to $|P| + c(P)$, where $c(P)$ counts runs: a pair contributes two vertices and one run, and a singleton at most two. Size and run count are subadditive under union, so $|P| + c(P) \leq 3(d-1)$. This covers the prefix. Lemma 3.4 needs at most $|P| + c(P) - 1 \leq 3d - 4$ later choices; the forced pair adds two. An empty prefix needs no further choice. □

The auxiliary weights have $O(s \log s)$ bits, and their incidence scores are computed from the explicit class-to-vertex table. Thus the ordering and actual first-hit times have polynomial bit complexity. □

Proof of Proposition 4.6. A balanced child has optimum at most $D = \mathrm{OPT}_W(S)$, and its count-potential drop pays for the root query. At a flat state, enumerate an initial pair containing the first optimal-witness query's supports; pad a singleton support by an adjacent real neighbor.

Claim 4.7 (Both initial positive exits fit H^*)**.** *The first-query exit has at least one query of discovery credit; the pinned second-query exit has at least three.*

Proof. For either class, restriction gives $\mathrm{OPT}_W(A) \le D$. The stored-child bound and (3) leave $z_A + 5$ queries beyond its child. Subtracting $e_A^{(4)} = z_A + 4$ leaves one, and subtracting $e_A^{(2)} = z_A + 2$ leaves three. This includes classes hit only by the padding vertex. □

Claim 4.8 (Every later class meets its computed deadline)**.** *For a class negative on the initial pair, $d_A \ge t_A$.*

Proof. Here $t_A \ge 2$. Supporting-prefix restriction and count credit give

$$B_A = H^* - h_A - e_A^{(4)} \ge 3(t_A - 1) + (z_A + 5) - (z_A + 4) = 3t_A - 2 \ge 4.$$

Thus $d_A = \lfloor (B_A + 2)/3 \rfloor \ge t_A$. The padded first block and the remaining comparison blocks, including the terminal singleton block if needed, satisfy Proposition 4.5. Its ordering has

$$g_A \le 3d_A - 2 = 3\lfloor (B_A + 2)/3 \rfloor - 2 \le B_A.$$

The last step follows from the remainder of $B_A + 2$ modulo three. □

Every actual route therefore fits H^* by Proposition 4.4, including the truncated zero route. In dimension at most one, two extreme tests determine the pattern and the same count credit pays for them. This proves the local fallback statement; induction gives the stated invariant for the hybrid tree without constructing a separate fallback tree. □

5 Scheduling the first positive answer

The maximum-depth algorithm schedules discovery around the heights of its child codes. For expected depth, a cap containing a likely target should instead be encountered early. This is a weighted min-sum set cover (MSSC) problem. The sets remain the supplied caps; singleton sets are not added unless an actual supplied test realizes them.

In the decision-tree application, a positive hull answer exits the all-negative route and reveals that the target lies in a small cap. The all-zero pattern has no positive answer, so on the full-cover route it is identified after every positive pattern has been covered (and an earlier singleton inference only shortens that route). Its contribution is therefore at most its weight times the schedule length. This is why the ordering analysis keeps both the first-cover sum and a separate length term.

Symbol	Meaning
$U,\ \pi_h,\ p_0$	Coverable positive-pattern demands, their positive weights, and the weight of a target found only after the schedule finishes.
$t_Q(h),\ \mathrm{MSSC}(Q)$	First-cover position of demand h in ordering Q and the sum $\sum_h \pi_h t_Q(h)$.
k_0	Minimum number of candidate tests needed to cover all demands in U.
$\beta,\ \Phi_\beta,\ Z_\beta$	Geometric budget factor and the first-cover and total-length accounting functions.
$r_Q(x),\ G_u$	Comparison-order mass with first-cover time greater than x, and the constructed schedule for shift u.

For background, weighted MSSC has a classical greedy four-approximation [1]. We do not use that guarantee in either proof. Instead, exact coverage with a cardinality budget gives the stronger circular-interval ordering proved below, including a bound on its total length.

Lemma 5.1 (Exact budgeted interval coverage)**.** *For positively weighted interval or circular-interval demands on a finite candidate set, a set of at most k candidates hitting maximum demand weight is computable in polynomial time.*

Proof. Algorithm 6 gives both interval oracles. For ordinary intervals, number the candidates $1, \ldots, M$. If the previous selected candidate is p and the next is $q > p$, the newly hit demands are exactly the intervals containing q whose left endpoint is greater than p. An interval containing q and an earlier selected point must also contain p. Conversely, any interval containing q with left endpoint greater than p contains no earlier selected candidate. Thus the added weight $g(p, q)$ depends only on these two positions, not on the earlier choices.

For exactly t selected candidates ending at q, use the recurrence

$$D(t, q) = \max_{p<q} \{D(t-1, p) + g(p, q)\}, \qquad D(0, 0) = 0.$$

The position 0 is a *sentinel*, an artificial initial marker before every interval, not a selectable candidate. All other zero-candidate or nonexistent states are infeasible and have value $-\infty$ in this maximization. Take the maximum over $t \leq k$ and final positions, including the empty selection. This is a polynomial dynamic program.

For circular intervals and $k \geq 1$, enumerate a selected candidate v. Credit the demands it hits and delete them. Cutting at v turns the remaining demands into ordinary intervals, on which the dynamic program selects at most $k - 1$ more candidates. One enumeration contains a candidate of an optimal nonempty selection, so taking the best is exact. The $k = 0$ case is empty. □

Algorithm 6: IntervalCover: exact minimum cover or budgeted coverage

Input: Ordered interval demands with weights; either *minimum* or budget k; ordinary or circular order.
Output: A minimum hitting set, or a maximum-weight set of size at most k.

if *order is circular* **then**
  Enumerate a forced selected candidate v; delete its hit demands and cut there
  Solve each residual ordinary instance, with budget $k - 1$ in budget mode
  return the best residual solution plus v, also considering the empty budget solution
if *mode is* minimum **then**
  if *an empty demand exists* **then**
    return infeasible
  $P \leftarrow \varnothing$
  while *a demand remains* **do**
    Choose the right endpoint q of an earliest-ending remaining demand
    $P \leftarrow P \cup \{q\}$; delete all demands containing q
  return P
else
  Set $D(0, 0) = 0$ and all other unreachable states to $-\infty$
  Precompute $g(p, q)$: weight of intervals containing q with left endpoint $> p$
  for $t = 1, \ldots, \min\{k, M\}$ **do**
    for $q = 1, \ldots, M$ **do**
      $D(t, q) \leftarrow \max_{p<q}(D(t-1, p) + g(p, q))$; store a maximizing p
  return the reconstruction of the best state with $t \leq k$, including $(0, 0)$

For minimum circular cover the forced candidate need only range over one nonempty demand. For budget coverage, empty demands contribute zero covered weight. The empty family returns the empty set in both modes. Lemmas 3.3 and 5.1 establish exactness.

The next lemma uses exact budgeted covers at geometrically increasing budgets. Small budgets can discover likely targets early; the minimum cover at the final budget guarantees that every positive target is eventually found. Shifting the budget sequence changes where it crosses integer thresholds. Averaging over shifts bounds both positive first-cover times and the zero-pattern waiting time; a finite search over the distinct resulting schedules removes the random shift.

5.1 The batch-ordering guarantee

Fix a nonempty demand family U, its minimum cover size k_0, and the waiting weight p_0. The only independent scheduling parameter is the rational growth factor $\beta > 1$. The following functions record the shift-averaged first-cover and length invoices; they are derived quantities rather than further choices.

Put

$$r_\beta = \lfloor \beta \rfloor, \qquad \lambda_\beta = \frac{\beta + 1}{2 \ln \beta}, \qquad \nu_\beta = 1 + \frac{1}{\ln \beta}.$$

For integer $t, k \geq 1$, define the accounting functions

$$\begin{aligned}
\Phi_\beta(1) &= \frac{r_\beta + 1}{2} - \frac{\ln(r_\beta!)}{2 \ln \beta}, \\
\Phi_\beta(t) &= \Phi_\beta(1) + \frac{1}{2 \ln \beta} \int_1^t \frac{\lfloor x \rfloor + \lfloor \beta x \rfloor}{x} \, dx, \\
Z_\beta(k) &= k + \frac{k \ln k - \ln(k!)}{\ln \beta}.
\end{aligned}$$

Proposition 5.2 (A shifted schedule has a discrete invoice)**.** *For a comparison ordering Q, put $r_Q(x) = \sum_{h : t_Q(h) > x} \pi_h$. The shifted construction G_u using budgets $k^{(j)} = \lfloor \beta^j u \rfloor$ satisfies*

$$\mathrm{MSSC}(G_u) \leq \frac{\lfloor u \rfloor + 1}{2} w(U) + \frac{1}{2} \sum_{j \geq 0} (k^{(j)} + k^{(j+1)}) r_Q(\beta^j u).$$

Each such schedule has length at most $(1 + \beta/(\beta - 1))k_0$.

Proposition 5.3 (One shift distribution pays both invoices)**.** *For the log-uniform shift on $[1, \beta)$,*

$$\mathbb{E}[\mathrm{MSSC}(G_u)] \leq \sum_h \pi_h \Phi_\beta(t_Q(h)), \qquad \mathbb{E}[|G_u|] \leq Z_\beta(k_0).$$

Moreover $\Phi_\beta(t) \leq \lambda_\beta(t - 1) + \Phi_\beta(1)$ and $Z_\beta(k) \leq \nu_\beta k - 1/\ln \beta$.

Proposition 5.4 (A finite shift search suffices)**.** *For fixed rational β, polynomially many rational shifts represent every distinct G_u. Minimizing the joint invoice $\mathrm{MSSC}(G_u) + p_0|G_u|$ over these representatives attains at most its log-uniform average.*

Proof strategy.

1. Exact budget coverage bounds the mass left after each phase.
2. Greedy order within a phase gives a half-batch first-cover invoice.
3. Average over the same shift for discovery and waiting, then choose the best of finitely many distinct schedules.

The averaging is a proof device; the returned schedule is deterministic.

Lemma 5.5 (Geometric-batch ordering)**.** *Let U be a nonempty family of coverable weighted circular intervals. Let k_0 be its minimum cover size and let $p_0 \geq 0$ be the weight of a target covered only when the entire schedule finishes. A deterministic polynomial-time ordering G satisfies, for every comparison cover ordering Q,*

$$\mathrm{MSSC}(G) + p_0|G| \leq \sum_{h \in U} \pi_h \Phi_\beta(t_Q(h)) + p_0 Z_\beta(k_0).$$

The simpler bounds $\Phi_\beta(t) \leq \lambda_\beta(t-1) + \Phi_\beta(1)$ and $Z_\beta(k) \leq \nu_\beta k - 1/\ln\beta$ also hold. Every candidate schedule considered by the construction also has length at most $\big(1 + \beta/(\beta-1)\big)k_0$.

Proof. Proposition 5.2 supplies the shifted schedules. By Proposition 5.3, their joint expected invoice has the stated bound. Proposition 5.4 returns a schedule no more expensive than this average. The length bound holds for every candidate and therefore for the selected one. □

Algorithm 7: BATCHORDERS: deterministic family of geometric-batch schedules

Input: Weighted circular demands and rational $\beta > 1$.
Output: All schedules needed for deterministic shift minimization.

```
Compute a minimum cover Q_* of size k_0 using IntervalCover
for k = 1, ..., k_0 − 1 do
    Compute a maximum-weight budget-k set Q_k
Find the first J with β^J ≥ k_0 by rational multiplication
Form the breakpoints 1, β and k/β^j ∈ [1, β), 0 ≤ j ≤ J, 1 ≤ k ≤ k_0
foreach breakpoint below β and each midpoint of consecutive breakpoints do
    u ← that rational representative; Q ← empty order
    for j = 0, 1, ... do
        k ← ⌊β^j u⌋; P ← Q_k if k < k_0, and P ← Q_* otherwise
        Greedily order P by newly covered weight, append its productive unused candidates to Q
        if k ≥ k_0 then
            stop this shift's phases
    Retain Q
return the retained family, removing duplicate schedules
```

An empty demand family needs no schedule. Lemma 5.5 bounds the family's minimum joint first-cover and waiting cost. The number of representatives is $O(k_0 \log k_0)$ for fixed β.

5.2 Proofs of the batch-ordering propositions

Proof of Proposition 5.2. Compute a minimum cover Q_* using Lemma 3.3 and, for each integer $k < k_0$, an exact maximum-weight coverage set Q_k of at most k candidates using Lemma 5.1. Fix the selected sets deterministically.

Fix a comparison cover ordering Q, and write

$$r_Q(x) = \sum_{h:t_Q(h)>x} \pi_h$$

for the weight whose first-cover index in Q exceeds x. For a shift $u \in [1, \beta)$, use budgets $k^{(j)} = \lfloor \beta^j u \rfloor$, $j = 0, 1, \ldots$. In each phase, process $Q_{k^{(j)}}$ when $k^{(j)} < k_0$ and Q_* otherwise. Order that set greedily by the weight it covers among the demands still unhit by earlier queries. Omit repeated queries and those hitting no remaining demand. Stop after the first phase whose budget reaches k_0.

Claim 5.1 (Phase coverage and schedule length)**.** *After a phase with budget $k < k_0$, the weight still unhit is at most $r_Q(k)$. The complete shifted schedule has length at most $\big(1 + \beta/(\beta - 1)\big)k_0$.*

Proof. The union of all queries processed through this phase covers every demand covered by Q_k: a query omitted for hitting no remaining demand has its covered demands already hit by an earlier phase. The set Q_k covers at least as much total weight as the first k queries of Q, since those queries are one feasible at-most-k candidate set. Hence the schedule has hit at least the weight hit by that comparison prefix, leaving at most $r_Q(k)$. The final Q_* phase covers all demands.

Let J be the first phase whose budget reaches k_0. The pre-final phases use at most

$$\sum_{j<J} \lfloor \beta^j u \rfloor < \sum_{j<J} \beta^j u < \frac{\beta}{\beta - 1} k_0$$

queries: the last pre-final unrounded budget is below k_0, and the preceding terms decrease geometrically by a factor $1/\beta$. The final minimum cover uses k_0 queries, proving the length bound. □

Claim 5.2 (First-cover cost within one batch)**.** *Suppose a phase processes m queries, newly hitting weights $g_1, \ldots, g_m$ in order, and starts and ends with unhit weights r and r'. Its contribution to the first-cover sum is at most*

$$mr' + \sum_{i=1}^{m} i g_i \le mr' + \frac{m+1}{2}(r - r') = \frac{m+1}{2} r + \frac{m-1}{2} r'.$$

Proof. After each greedy choice, the remaining coverage of every unused candidate can only decrease. Therefore the newly hit weights are nonincreasing: $g_1 \ge \cdots \ge g_m$. Pairing ranks i and $m + 1 - i$ shows

$$\sum_{i=1}^{m} i g_i \le \frac{m+1}{2} \sum_{i=1}^{m} g_i.$$

Indeed, in each pair the earlier rank has at least as much weight as the later one, so replacing both ranks by their mean cannot decrease the weighted sum. A demand first hit at rank i remains unhit for exactly i queries in this batch, while each demand still unhit at the end remains unhit for all m queries. Since $\sum_i g_i = r - r'$, their combined contribution gives the claim. □

Now let r_j be the unhit weight after phase j, and put $r_{-1} = w(U)$. If phase j processes m_j queries, the preceding claim bounds its first-cover contribution by

$$\frac{m_j + 1}{2} r_{j-1} + \frac{m_j - 1}{2} r_j.$$

Since $m_j \le k^{(j)}$, replacing m_j by its integer budget and combining adjacent phase coefficients gives

$$\mathrm{MSSC}(G_u) \le \frac{\lfloor u \rfloor + 1}{2} w(U) + \frac{1}{2} \sum_{j \ge 0} \big(k^{(j)} + k^{(j+1)}\big) r_Q(\beta^j u).$$

For every pre-final phase, the first claim gives $r_j \le r_Q(k^{(j)}) = r_Q(\beta^j u)$; the equality uses the integer-valued comparison first-cover indices. The final phase has residual zero. The displayed infinite sum only adds nonnegative upper bounds for the final and later phases.

□

Proof of Proposition 5.3.

Claim 5.3 (Shift averaging yields the discrete first-cover function)**.** *The same log-uniform shift satisfies* $\mathbb{E}[\mathrm{MSSC}(G_u)] \le \sum_h \pi_h \Phi_\beta(t_Q(h))$.

Proof. Average this expression over a *log-uniform shift*, with probability density $1/(u \ln \beta)$ on $[1, \beta)$. This density integrates to one because $\int_1^\beta du/u = \ln \beta$. Splitting the first term's integral at its integers gives

$$\begin{aligned} \mathbb{E}[\lfloor u \rfloor] &= \frac{\displaystyle\sum_{k=1}^{r_\beta - 1} k \ln \frac{k+1}{k} + r_\beta \ln \frac{\beta}{r_\beta}}{\ln \beta} \\ &= r_\beta - \frac{\ln(r_\beta!)}{\ln \beta}. \end{aligned}$$

The intervals $[\beta^j, \beta^{j+1})$ partition $[1, \infty)$, so averaging the remaining terms gives

$$\begin{aligned} \mathbb{E}[\mathrm{MSSC}(G_u)] &\le (\Phi_\beta(1)) w(U) \\ &\quad + \frac{1}{2 \ln \beta} \int_1^\infty \frac{\lfloor x \rfloor + \lfloor \beta x \rfloor}{x} r_Q(x)\, dx \\ &= \sum_{h \in U} \pi_h \Phi_\beta(t_Q(h)). \end{aligned}$$

For each real x, write $r_Q(x) = \sum_{h : t_Q(h) > x} \pi_h$. Interchanging this finite sum with the integral makes each hypothesis contribute the integral from 1 to its integer first-cover time. Adding the initial $(\Phi_\beta(1)) w(U)$ term gives exactly $\sum_{h \in U} \pi_h \Phi_\beta(t_Q(h))$. Bounding the integrand by $\beta + 1$ recovers $\Phi_\beta(t) \le \lambda_\beta(t-1) + \Phi_\beta(1)$, but would discard the savings from all later integer budgets.

□

Claim 5.4 (Shift-averaged schedule length)**.** *Under the same shift distribution,*

$$\mathbb{E}[|G_u|] \le Z_\beta(k_0) \le \nu_\beta k_0 - \frac{1}{\ln \beta}.$$

Proof. The final cover contributes k_0 queries. The earlier phases contribute at most $\sum_{j:\, \beta^j u < k_0} \lfloor \beta^j u \rfloor$. After averaging and substituting $x = \beta^j u$, their expected length is

$$\frac{1}{\ln \beta} \int_1^{k_0} \frac{\lfloor x \rfloor}{x}\, dx.$$

On each interval $[j, j+1)$, $\lfloor x \rfloor = j$, so summing the integrals gives

$$\int_1^{k_0} \frac{\lfloor x \rfloor}{x}\, dx = k_0 \ln k_0 - \ln(k_0!).$$

Consequently $\mathbb{E}[|G_u|] \le Z_\beta(k_0)$. Since $\ln x$ is increasing,

$$\ln(k_0!) = \sum_{j=2}^{k_0} \ln j \ge \int_1^{k_0} \ln x\, dx = k_0 \ln k_0 - k_0 + 1.$$

Substituting this in $Z_\beta(k_0)$ gives $Z_\beta(k_0) \le \nu_\beta k_0 - 1/\ln\beta$. The formula also gives $Z_\beta(1) = 1$. □

The first-cover and length bounds hold under the same shift distribution. Therefore

$$\mathbb{E}[\mathrm{MSSC}(G_u) + p_0|G_u|] \le \sum_{h \in U} \pi_h \Phi_\beta(t_Q(h)) + p_0 Z_\beta(k_0).$$

□

Proof of Proposition 5.4. No random shift is used by the algorithm. Let J be the first integer with $\beta^J \ge k_0$. All budget changes occur at the rational breakpoints $u = k/\beta^j$ for $0 \le j \le J$, $1 \le k \le k_0$, inside $[1, \beta)$. Enumerate these breakpoints and one rational midpoint between each consecutive pair, construct the resulting orderings, and take the one minimizing the rational quantity $\mathrm{MSSC}(G_u) + p_0|G_u|$. There are $O(k_0 \log k_0)$ intervals for fixed β. The budgets, selected sets, and their greedy internal orders are constant on each interval. Their minimum joint value is at most the average just bounded. The waiting-time weight p_0 is used only to choose the schedule; all queries remain supplied candidates. □

6 A polynomial approximation for expected depth

We give one algorithm for arbitrary positive rational priors. At a state admitting a balanced supplied line, it queries that line. At a flat state, it discovers a positive hull answer, learns the complete hull pattern, and then recurses within that pattern class. Its polynomial-time factor is below 5.182976.

The key is to pay for a whole pattern stage rather than for each endpoint search separately. Queries removed from an optimal tree pay for discovery; information already learned during discovery pays for endpoint recovery. These two credits telescope across recursive states. The geometric and interval-coverage primitives were proved in Sections 3 and 5.

6.1 The algorithm and its supporting propositions

Use fixed rational calibration parameters β, δ, p_*, whose values are deferred to Section 6.3. Here $3 < \beta < 4$, $0 < \delta < 1/3$, and $0 < p_* < 1/2$. A line is mass-balanced if each side has mass at least $\delta w(S)$. Otherwise the common-witness construction supplies the normalized hull. Keep its full cyclic order, including the constant-negative origin marker when it is a hull vertex. Candidate blocks are its edges: each block uses only its one or two real supplied endpoint tests.

Write Π for the full-pattern partition and A_0 for its all-negative class, which is empty or a singleton. Throughout this section the prior weights retain their original normalization:

$$w = w(S), \qquad p_0 = w(A_0), \qquad b = w - p_0, \qquad I = I(S, \Pi), \qquad \Delta = \Delta_{\mathrm{E}}(S, \Pi).$$

The quantity I is the information needed to learn the pattern; Δ is the optimal cost made unnecessary after its class is known. Neither is evaluated by the algorithm.

Choose the coverage schedule. Compute a maximum-weight block cover Q_k for each budget $k < k_{\min}$ and a minimum full cover Q_* of size $k_{\min}$. These are circular-interval DPs. For a geometric shift u, set $k_j = \lfloor \beta^j u \rfloor$. Try exactly the following initial budget sequences, followed by the common geometric tail:

$$\begin{aligned} &q, r, k_2, k_3, \ldots, && 1 \le q \le 6, \quad r \in \{k_1 - 1, k_1, k_1 + 1\}, \quad q < r < k_2, \\ &q, k_2, k_3, \ldots, && 1 \le q \le 6. \end{aligned}$$

At each budget $k < k_{\min}$, greedily append productive unused blocks of Q_k, ordering them by newly covered weight. At the first budget reaching $k_{\min}$, use Q_* and finish. There are at most twenty-four variants per shift.

Refine the usual geometric shift partition by the constant set

$$\frac{k}{\beta},\ \frac{k}{\beta^2} \in [1, \beta), \qquad 1 \le k \le \lceil \beta^3 \rceil. \tag{4}$$

Represent each cell by its rational midpoint, and also include its left breakpoint. Thus initial budgets remain represented even when $k_{\min}$ truncates the ordinary geometric partition. In particular, $r = k_1 - 1$ can first complete the cover at $k_1 = k_{\min} + 1$.

Choose the two-endpoint mode. On an all-negative schedule prefix, let $x \le y$ be the newly hit masses exclusive to the two endpoints of the next block, and let z be their shared mass. Put $h_* = 1 - 2p_*$ and define the rational entropy credit and saving

$$\begin{aligned} E_*(a, z) &= \min\{(h_*/p_*)a,\ h_*(a + z),\ (h_*/p_*)z\}, \\ D(x, y, z) &= \max\{y,\ x + y - z + E_*(y, z)\}. \end{aligned} \tag{5}$$

The first choice stops at the first positive answer, asking the smaller exclusive mass first. The second choice asks both endpoints, asking the larger exclusive mass first. The latter order gives at least as much credited information because $E_*(a, z)$ is nondecreasing in a. Select the larger saving using rational arithmetic. The exact binary-entropy bound behind E_* is checked only in the final calibration.

For a candidate schedule G, let k_A be its first block hitting positive class A, and set

$$F_*(G) = 2 \sum_{A \ne A_0} w(A) k_A + 2p_0|G| - \sum_{\text{blocks}} D. \tag{6}$$

This one score prices positive discovery, the entire zero-pattern waiting route, and the mode savings. Choose its minimum. At each positive exit, build the legal expected-cost endpoint decoder and attach the recursive tree for its full pattern class.

Algorithm 8: Rational choice between stopping and querying both endpoints

Input: New exclusive masses x, y, shared mass z; calibrated p_* and $h_* = 1 - 2p_*$.
Output: Stored endpoint order, mode, and rational saving D.

Label the endpoints so $x \le y$
$E \leftarrow \min\{(h_*/p_*)y,\ h_*(y + z),\ (h_*/p_*)z\}$
if $y \ge x + y - z + E$ **then**
  Record stop mode with the x endpoint first; $D \leftarrow y$
else
  Record query-both mode with the y endpoint first; $D \leftarrow x + y - z + E$
return the recorded order, mode, and D

These new masses are computed on the all-negative prefix before the block. In the stored discovery stage, stop mode ends at the first positive answer; query-both mode learns both bits before endpoint recovery. Their union and first-cover rank are the same.

Algorithm 9: ExpectedTree: the polynomial expected-depth algorithm

Input: State S, original positive rational priors; calibrated β, δ, p_*.
Output: A supplied-query identification tree.

if $|S| \le 1$ **then**
  return its leaf
if *a mass-balanced supplied line ℓ exists* **then**
  return ℓ with children ExpectedTree(S_ℓ^-) and ExpectedTree(S_ℓ^+)
Compute HullData(S), its pattern classes, and zero weight $p_0 = w(A_0)$
if *there are at most two real extreme hull tests* **then**
  Build their direct full-pattern stage
else
  Use edges of the full hull as blocks, retaining its free origin marker
  Select the schedule and stored modes by Algorithm 10
  Build discovery along its all-negative prefix, with those modes on each first hit
  At each positive exit build expected EndpointCode to recover the full pattern
Attach ExpectedTree(A) to each nonzero pattern class A
Use a zero-query leaf for the zero singleton; omit it when absent
return the tree after contracting constant and cached questions

Every child inherits the same calibration and again has access to all supplied queries. Scores, modes, endpoint codes, and child trees are constructed from the known incidence table and rational weights. The unknown hypothesis subsequently follows answers in that stored tree.

Algorithm 10: Two-budget schedule selection for ExpectedTree

Input: Positive-pattern block demands, original weights, zero weight p_0; calibrated β, p_*.
Output: A schedule and stored modes minimizing the rational score F_*.

Compute Q_* of size $k_{\min}$ and every Q_k for $k < k_{\min}$
Refine the geometric shift cells by (4)
foreach *rational breakpoint or cell midpoint* $u \in [1, \beta)$ **do**
  $k_1 \leftarrow \lfloor \beta u \rfloor$; $k_2 \leftarrow \lfloor \beta^2 u \rfloor$
  for $q = 1, \dots, 6$ **do**
    $\mathcal{V} \leftarrow \{(q,0)\} \cup \{(q,r) : r \in \{k_1 - 1, k_1, k_1 + 1\},\ q < r < k_2\}$
    foreach $(q,r) \in \mathcal{V}$ **do**
      $G \leftarrow$ empty order
      Process q, then r if $r > 0$, then $k_2, k_3, \dots$
      At each budget k, greedily append productive unused blocks from Q_k if $k < k_{\min}$, and from Q_* otherwise
      Stop these phases at the first budget reaching $k_{\min}$
      Compute the new masses and stored mode of each block by Algorithm 8
      Retain (G, modes) with its rational score $F_*(G)$
return a retained pair of minimum score

The proof uses four propositions. Their statements isolate the geometric comparison, the physical stage, the schedule bound, and the recursive conversion; their proofs follow the theorem. Recall the binary entropy h from Definition 2.7, and define

$$\sigma(\delta) = -k_\delta \delta \ln \delta - r_\delta \ln r_\delta, \quad k_\delta = \lfloor 1/\delta \rfloor, \quad r_\delta = 1 - k_\delta \delta,$$

with $0 \ln 0 = 0$. The latter is the minimum entropy of a fully capped mass distribution, including its possible remainder.

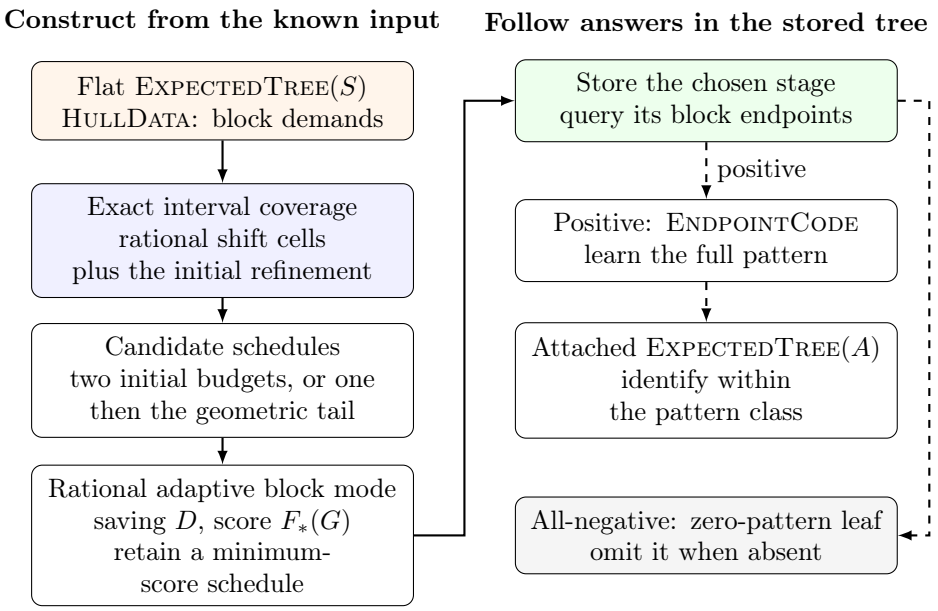


Figure 4: The polynomial expected-depth algorithm. The two-budget schedule family is Algorithm 10. The shift cells include the refinement (4). Solid arrows are construction-time data flow, computed from the supplied incidence table and rational priors. Dashed arrows show how the unknown hypothesis traverses the resulting tree; endpoint decoders and recursive children are already constructed. Adaptive modes choose whether the first positive block stops early or queries both endpoints. The certificate's mixture is used in the proof of the bound, not to randomize execution.

Proposition 6.1 (Witness-path restriction credit)**.** *Follow the common witness through an optimal tree, along a path of length ℓ. For each pattern class A, let t_A be the first path query whose hull supports meet its positive pattern; put $t_A = \ell + 1$ if none do. Then*

$$\Delta \geq \sum_{A \neq A_0} w(A)(t_A - 1) + p_0 \ell. \tag{7}$$

The corresponding positive witness masses $m_t = \sum_{A \neq A_0 : t_A = t} w(A)$ sum to b and satisfy $m_t \leq 2\delta w$. An edge comparison ordering hits every positive class by rank t_A. If A_0 exists, it covers all positive classes with at most ℓ blocks, so $k_{\min} \leq \ell$.

Proposition 6.2 (Pattern-stage cost and discovery information)**.** *For every candidate schedule and its selected modes, the full-pattern stage uses only supplied queries and satisfies*

$$W_{\mathrm{pat}}(S) \leq F_*(G) + 3b + \frac{I - J_*}{\ln 2}, \qquad 0 \leq J_* \leq I, \tag{8}$$

where J_* is the information of a coarsened discovery transcript. Every positive coarsened exit is contained in one hull cap, and hence

$$J_* \geq b\sigma(\delta). \tag{9}$$

The zero transcript is the zero-pattern singleton when it exists.

Proposition 6.3 (Polynomial schedule and the local bound)**.** *With the final calibration, the two-budget schedule algorithm is polynomial in the input and rational bit length. At every flat state,*

including a degenerate hull, its full-pattern stage satisfies

$$W_{\mathrm{pat}}(S) \le A\Delta + Bb + \frac{I - J_*}{\ln 2}, \qquad A = \frac{\beta + 1}{\ln \beta}, \tag{10}$$

where B is the positive-mass coefficient certified in Section 6.3.

Proposition 6.4 (Stage-to-tree conversion)**.** *Suppose flat stages satisfy $W \le A\Delta + Bb + (I - J)/\ln 2$ and $0 \le J \le I$, $J \ge b\sigma(\delta)$, with nonnegative A, B. A recursion using those stages and mass-balanced queries has*

$$c_{\mathrm{E}}(T(S)) \le A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S), \qquad c = \max\{1/h(\delta), B/\sigma(\delta)\}.$$

Consequently its ratio is at most $A + c\ln 2$.

Proof strategy.

Expected-depth proof roadmap.

1. Replace an optimal witness query by its adjacent hull supports. Restriction pays for the resulting first-hit ranks.
2. Choose whether the first positive block stops early or learns both endpoint bits. Its recorded information reduces the endpoint-decoding cost.
3. Exact coverage transfers witness prefix mass to actual batches. A fixed certificate pays for changed initial ranks and for the complete zero waiting route.
4. Bound every stage by its restriction and entropy decreases, then telescope. Numerical tuning is deferred until after these proofs.

Theorem 6.5 (Polynomial expected-depth approximation)**.** *For a separating supplied family of binary unit-cost planar line queries and arbitrary strictly positive rational priors,* ExpectedTree *runs in deterministic polynomial time and returns*

$$c_{\mathrm{E}}(T(S)) \le A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S), \qquad \boxed{A + c\ln 2 < 5.182976.}$$

Here $c = \max\{1/h(\delta), B/\sigma(\delta)\}$. Its running-time exponent is independent of the smallest prior.

Proof. Proposition 6.3 gives the local stage bound; Proposition 6.2 gives its discovery-information lower bound. Proposition 6.4 sums them over the recursion. The calibration in Section 6.3 gives the stated factor. Each nonsingleton stage partitions its state into proper disjoint classes, so the polynomial stage constructions form at most $n - 1$ recursive stages. □

6.2 Proofs of the supporting propositions

Proof of Proposition 6.1.

Claim 6.1 (An unhit support forces an optimal-prefix query to be constant)**.** *Before the first supporting hit t_A, every member of A follows the common witness through its first $t_A - 1$ queries.*

Proof. All supporting hull bits of each earlier query are negative on A. The hull cap-cover implication therefore forces that supplied query to have the witness's answer throughout A. Induct along the path. Restricting the optimal tree to A contracts these constant queries, saving at least $w(A)(t_A - 1)$. Summing over the disjoint classes and using optimality of each child tree gives the restriction inequality. A class unhit on the entire path reaches its leaf z, so it is $\{z\}$. When A_0 exists, this is exactly A_0 and its credit is $p_0\ell$. □

Claim 6.2 (Adjacent comparison blocks cover witness prefixes)**.** *Replace each witness query by a full-hull edge containing its one or two real supports. A singleton support may use either incident edge. This comparison hits every class by its index t_A.*

Proof. Enlarging the support set to an edge cannot delay a hit. Removing repeated or unproductive edges improves ranks further. If the terminal singleton has a nonzero pattern and remains unhit, append an edge incident to one of its positive vertices; its rank is at most $\ell + 1 = t_A$. If A_0 exists, all positive classes were already hit within the ℓ witness blocks. Each ordinary first-hit mass lies in at most two flat-state caps, giving $m_t \le 2\delta w$; an appended singleton lies in one such cap. This proves all remaining statements of the proposition. The witness order is an analysis object, not an order computed by the algorithm. □

Proof of Proposition 6.2.

Claim 6.3 (Two endpoint modes have explicit physical costs)**.** *A positive class first hit by block k has nonentropy allowance $2k + 3$. Stop mode saves y weighted queries; query-both mode saves $x + y - z$ before accounting for its additional information.*

Proof. At most $2(k - 1)$ tests precede the hitting block. In stop mode the first-positive route uses one discovery query and two endpoint codes, with overhead four. The second-positive route uses two discovery queries, but its adjacent negative answer pins one boundary, leaving only overhead two. Relative to $2k + 3$, exactly the second endpoint's exclusive mass y saves one query. In query-both mode each exclusive route uses two discovery queries and one endpoint code, for allowance $2k + 2$; the shared route uses two queries and two codes, for allowance $2k + 4$. Their weighted saving is $x + y - z$. A free origin endpoint or a cached answer can only lower these costs. Finish the recorded mode before singleton inference in the analysis, then contract constant questions in the constructed tree. □

Claim 6.4 (Coarsening accounts for the additional information once)**.** *In a query-both block, forget the second answer on routes whose first answer was positive. The resulting positive events lie in individual caps, and the forgotten information pays for $E_*(a, z)$, where a is the first endpoint's exclusive mass.*

Proof. The coarsening merges masses a, z inside the first endpoint's cap; the other exclusive event lies in the second endpoint's cap. It loses exactly $(a + z)h(a/(a + z))$ information, interpreted as zero when $a + z = 0$. Binary-entropy concavity and symmetry imply

$$H_2(v) \ge \min\{(h_*/p_*)v,\ h_*,\ (h_*/p_*)(1 - v)\},$$

provided the calibration satisfies $H_2(p_*) \ge h_*$. Its homogeneous form is $E_*(a, z)$.

Let J_{act} be the actual discovery information. The entropy chain rule bounds the subsequent decoding cost by its code overhead plus $(I - J_{\mathrm{act}})/\ln 2$. Replacing J_{act} by J_* credits exactly the forgotten information in query units. This adds $E_*(a, z)$ to the physical query-both saving, without charging the remaining entropy twice. Because each of the three linear forms in $E_*(a, z)$ is nondecreasing in a, asking the larger exclusive mass first maximizes this credit. Stop mode instead asks the smaller one first. Summing the chosen savings and the first-hit allowances gives (8). □

Claim 6.5 (Capped discovery exits have entropy proportional to positive mass)**.** *A probability vector with one unrestricted entry p_0 and all remaining entries at most δ has entropy at least $(1 - p_0)\sigma(\delta)$.*

Proof. Put $f(a) = -a \ln a$ with $f(0) = 0$, and let $b = 1 - p_0$. Moving mass from a smaller capped entry to a larger one cannot increase entropy, by concavity of f. Repeating gives the minimum shape: $j = \lfloor b/\delta \rfloor$ entries of mass δ and one possible remainder. Its total entropy is

$$T(b) = f(1-b) + jf(\delta) + f(b - j\delta).$$

On each interval between consecutive multiples of δ, T is concave. It suffices to check $T(b) \geq b\sigma(\delta)$ at those multiples and at $0, 1$. At $b = j\delta > 0$,

$$T(b)/b = L + g(1-b), \quad L = \ln(1/\delta), \quad g(a) = \frac{-a \ln a}{1-a}.$$

Since $g'(a) = (-\ln a - 1 + a)/(1-a)^2 \geq 0$, these ratios decrease as b increases. If the final remainder $r_\delta = 1 - \lfloor 1/\delta \rfloor \delta$ is zero, the last ratio is $L = \sigma(\delta)$. Otherwise its excess over $T(1) = \sigma(\delta)$ is $r_\delta L + r_\delta f(r_\delta)/(1-r_\delta) \geq 0$. The endpoints $0, 1$ have equality, so concavity proves the claim. Also $\sigma(\delta) = \ln(1/\delta) + r_\delta \ln(\delta/r_\delta) \geq \ln(1/\delta)$, with the second term zero at $r_\delta = 0$. □

Every nonzero coarsened discovery cell lies in a cap of mass below δw. Normalize by w and apply the claim, then rescale, to obtain (9). The transcript is a function of the full pattern, so $0 \leq J_* \leq I$. Endpoint recovery identifies the complete pattern, while a negative full cover identifies A_0. This finishes Proposition 6.2.

Proof of Proposition 6.3. The next claims separate actual coverage and mode savings from their optimal-witness comparison. Actual first-hit ranks price the constructed schedule; witness ranks provide restriction credit. Exact coverage is the connection between them.

Claim 6.6 (Block mode savings have two affine lower bounds)**.** *For newly hit mass $m = x + y + z$, with endpoint caps bounded by δw, the selected saving satisfies*

$$D \geq 0, \qquad D \geq m - \delta w, \qquad D \geq c_* m - d_* \delta w, \quad c_* = \frac{2 - p_*}{1 - p_*}, \quad d_* = \frac{2}{1 - p_*}.$$

Proof. Stop mode gives $D \geq y \geq 0$ and $m - y = x + z \leq \delta w$. For the last inequality normalize $\delta w = 1$. It is immediate if $m \leq 1$. For $1 \leq m \leq 2$, the feasible (x, y) triangle has vertices $(m-1, m-1)$, $(m-1, 1)$, and $(m/2, m/2)$. The concave expression $x + y - z + E_*(x, z)$ is a lower bound on the selected query-both saving, since $E_*(y, z) \geq E_*(x, z)$. At the two vertices with $z = 0$ it equals m; at the other it is

$$f(m) = 3m - 4 + \min\{(h_*/p_*)(m-1),\ h_*,\ (h_*/p_*)(2-m)\}.$$

For $m \leq 1 + p_*$, $c_* m - d_* \leq m - 1$, already covered by stop mode. On $[1 + p_*, 2]$, concavity and the endpoint values $f(1 + p_*) = p_*$, $f(2) = 2$ give the chord $c_* m - d_*$. Also $m \geq c_* m - d_*$ for $m \leq 2$. All vertices therefore dominate the required line; concavity and rescaling finish the proof. □

Claim 6.7 (Exact first-batch coverage transfers witness prefix mass)**.** *Let $M_q = \sum_{t \leq q} m_t$. A first budget-$q$ batch gives total schedule savings at least each of*

$$\mathsf{A}_q = M_q - \delta w q, \qquad \mathsf{C}_q = c_* M_q - d_* \delta w q, \qquad 0.$$

Proof. The witness comparison covers M_q using at most q blocks. Thus exact budgeted coverage selects a set covering mass at least M_q. Sum either affine mode bound over the at most q productive first-batch blocks. All subsequent mode savings are nonnegative. If $q \geq k_{\min}$, the minimum cover instead hits all positive mass with at most q blocks, so the same bound holds. A convex mixture of the three minorants compares the same total saving once. □

Claim 6.8 (Changing the initial budgets has signed first-hit and zero-route costs)**.** *Relative to the geometric sequence $k_0, k_1, k_2, \ldots$, the exact doubled first-hit improvement is*

$$\begin{aligned} d_{q,r}(u,t) &= k_0 - q + (k_0+k_1)\mathbf{1}_{\{t>k_0\}} + (k_1+k_2)\mathbf{1}_{\{t>k_1\}} \\ &\quad - (q+r)\mathbf{1}_{\{t>q\}} - (r+k_2)\mathbf{1}_{\{t>r\}}, \quad r>0, \\ d_{q,0}(u,t) &= k_0 - q + (k_0+k_1)\mathbf{1}_{\{t>k_0\}} + (k_1+k_2)\mathbf{1}_{\{t>k_1\}} \\ &\quad - (q+k_2)\mathbf{1}_{\{t>q\}}. \end{aligned} \tag{11}$$

The full all-negative length invoice changes by

$$z_{q,r}(u,k) = q\mathbf{1}_{\{q<k\}} + r\mathbf{1}_{\{r<k\}} - k_0\mathbf{1}_{\{k_0<k\}} - k_1\mathbf{1}_{\{k_1<k\}}, \qquad k = k_{\min}. \tag{12}$$

Proof. The within-batch greedy argument of Lemma 5.5 bounds first-hit cost for any increasing budgets $a_0, a_1, \ldots$ by

$$\frac{a_0+1}{2} + \frac{1}{2}\sum_{j\ge 0}(a_j + a_{j+1})\mathbf{1}_{\{t>a_j\}}.$$

Subtract the expressions for the old and modified sequences: only the two initial transitions differ. The expression is nondecreasing in t, so comparison edges that hit earlier than t_A only help. Early completion removes nonnegative continuation terms. For the negative route, count every budget below $k_{\min}$ and add the final minimum cover; cancel their identical geometric tails. This gives the second identity, with the r term zero when $r = 0$. Neither change is assumed nonnegative. □

Claim 6.9 (One capped-mass calculation includes the zero prior)**.** *Suppose $\sum_t m_t = b$, $m_t \le 2\delta w$, and coefficients a_t are at least τ except on a finite set E. Define*

$$\chi = \tau - 2\delta\sum_{t\in E}(\tau - a_t)_+ - \delta\mu.$$

If $\chi + \zeta \ge \tau$, then

$$\sum_t m_t a_t - \delta w\mu + \zeta p_0 \ge \chi b. \tag{13}$$

Proof. Only coefficients below τ need their individual mass caps. Thus the left side is at least

$$\tau b - (\tau - \chi)w + \zeta p_0 = \chi b + p_0(\chi + \zeta - \tau) \ge \chi b.$$

This argument permits arbitrary zero-pattern mass and uses no identification of witness ranks with actual discovery ranks. □

Assembling the local bound. Retain $\Phi_\beta, Z_\beta, \lambda_\beta, \nu_\beta$ from the geometric-batch lemma. Define the nondecreasing discrete discount

$$e_\beta(t) = 2[\lambda_\beta(t-1) + \Phi_\beta(1) - \Phi_\beta(t)] = \frac{1}{\ln\beta}\int_1^t \left[\beta + 1 - \frac{\lfloor x\rfloor + \lfloor \beta x\rfloor}{x}\right] dx.$$

The integrand is nonnegative. Proposition 6.1 provides the comparison ranks. Log-uniform averaging of the old first-hit bound gives $\Phi_\beta(t)$; the signed correction subtracts $d_{q,r}(u,t)/2$. The zero length bound is at most $Z_\beta(k_{\min}) + \mathbb{E}[z_{q,r}(u,k_{\min})]$.

The fixed distribution in Section A compares only valid two-budget variants and first-batch minorants. Its coefficient definition is

$$a_t = e_\beta(t) + \mathbb{E}[d_{q,r}(u,t) + s\mathbf{1}_{\{t\le q\}}], \qquad \mu = \mathbb{E}[\rho], \tag{14}$$

where the selected minorant is $sM_q - \rho\delta w$. Consequently

$$\sum_t m_t e_\beta(t) + \mathbb{E}\left[\sum_t m_t d_{q,r}(u,t) + \mathrm{Save}(G)\right] \geq \sum_t m_t a_t - \delta w \mu.$$

The final calibration proves three simple conditions: all a_t except a_1 exceed τ; every mean zero-length change is nonpositive; and $\chi + 2/\ln\beta \geq \tau$, where

$$\chi = \tau - 2\delta(\tau - a_1) - \delta\mu, \qquad B = 3 + 2\Phi_\beta(1) - \chi. \tag{15}$$

It also gives $\lambda_\beta \geq \nu_\beta$. Therefore the restriction credit, $k_{\min} \leq \ell$ when $p_0 > 0$, and $Z_\beta(k) \leq \nu_\beta k - 1/\ln\beta$ give

$$\mathbb{E}[F_*(G)] \leq A\Delta + 2\Phi_\beta(1)b - \sum_t m_t a_t + \delta w\mu - \frac{2p_0}{\ln\beta}.$$

The capped-mass claim with $E = \{1\}$ and $\zeta = 2/\ln\beta$ turns the last terms into $-\chi b$. The chosen minimum rational score is no larger than this average. Adding the stage's $3b$ and remaining-information term proves (10).

Claim 6.10 (Degenerate hulls and finite shift enumeration preserve the bound)**.** *The direct stage and the rational shift family are polynomial, including all terminal cases.*

Proof. With at most two real extreme hull tests, query them directly to learn the full pattern, at cost at most $2w$. In a nonsingleton state with $p_0 > 0$, the witness path has $\ell \geq 1$, so $\Delta \geq p_0$; otherwise $b = w$. The calibrated $A, B > 2$ imply $2w \leq A\Delta + Bb$, and here $J_* = I$.

For a nondegenerate hull, all coverage calls are polynomial interval DPs. The geometric partition has polynomially many rational cells; (4) adds only a constant number. Within a refined cell every executed budget, candidate order, mode, and score is fixed. Thus every schedule in the proof average has an actual enumerated representative. There are at most twenty-four variants per cell. Prior weights enter only polynomial-bit rational sums and comparisons; no logarithm or proof distribution is computed. Empty positive demands occur only at a terminal singleton. Early inferences, cached questions, free origin markers, and empty branches can only reduce the constructed cost. □

This completes Proposition 6.3.

Proof of Proposition 6.4.

Claim 6.11 (Each stage is paid for by two potential decreases)**.** *Every balanced query or flat stage costs at most $A\Delta + cI$.*

Proof. A balanced query has information at least $w(S)h(\delta)$, so its cost $w(S)$ is at most cI. Also $c \geq 1/\ln 2$. For a flat stage, $Bb \leq cJ$ and therefore $Bb + (I - J)/\ln 2 \leq cI$. Add its restriction term. □

Claim 6.12 (Restriction and information telescope over pattern classes)**.** *The sum of stage costs is at most $A\,\mathrm{OPT}_{\mathrm{E}}(S) + c\mathcal{E}(S)$.*

Proof. By their definitions,

$$\Delta = \mathrm{OPT}_{\mathrm{E}}(S) - \sum_{A\in\Pi} \mathrm{OPT}_{\mathrm{E}}(A), \qquad I = \mathcal{E}(S) - \sum_{A\in\Pi} \mathcal{E}(A).$$

Sum these differences over the disjoint recursive classes; both potentials vanish at singleton leaves. Finally the binary-tree entropy lower bound gives $\mathcal{E}(S) \leq \ln 2\ \mathrm{OPT}_{\mathrm{E}}(S)$, completing the proposition and its multiplicative guarantee. □

6.3 Calibration of the expected-depth bound

The preceding proof separates the algorithm from its numerical tuning. Only three fixed rational values enter the construction:

$$\beta = \frac{3512863}{10^6}, \qquad \delta = \frac{393931}{2500000}, \qquad p_* = \frac{21317}{125000}. \tag{16}$$

All other coefficients are derived from these and from the fixed comparison distribution in Section A. That distribution is used to prove the score bound; the algorithm minimizes its rational score over the entire candidate family.

What must be checked. The proof requires the entropy tent $H_2(p_*) \geq 1 - 2p_*$, $\lambda_\beta \geq \nu_\beta$, and three certificate properties: a single exceptional rank coefficient; no increase in mean zero waiting length; and enough retained zero credit. Set $\tau = 2190079/10^6$. The exact coefficient construction (14) gives

$$1.276476788627 < a_1 < 1.276476788629 < \tau, \qquad \min_{2 \leq t \leq 14} a_t > 2.1900797 > \tau.$$

All first-batch cutoffs are at most six, $k_0 \leq 3$, $k_1 \leq 12$, and $r \leq 13$. Thus the signed corrections are constant for $t \geq 14$; $a_t = e_\beta(t) + \kappa$ on that entire tail. Its monotonicity reduces all later coefficients to the single a_{14} check.

Likewise, writing $D_0(k) = \mathbb{E}[z_{q,r}(u, k)]$,

$$D_0(1) = 0, \qquad \max_{2 \leq k \leq 14} D_0(k) < -0.06992, \qquad D_0(k) = D_0(14) \quad (k \geq 14).$$

This proves the required mean waiting inequality for every possible minimum-cover size, including small ones. The derived coefficients satisfy $A, B > 2$ and

$$\chi > 1.199202587061, \qquad B < 4.374727111314, \qquad \chi + 2/\ln\beta > \tau.$$

Here $3 + 2\Phi_\beta(1) = 7 - \ln 6/\ln\beta$. The strict inequality $\beta - 1 > 2\ln\beta$ gives $\lambda_\beta > \nu_\beta$.

How the checks are certified. For rational $x \geq 1$, write $z = (x-1)/(x+1)$ and $L_N(x) = 2\sum_{j=0}^{N-1} z^{2j+1}/(2j+1)$. The positive logarithm series has the rational enclosure

$$L_N(x) \leq \ln x \leq L_N(x) + \frac{2z^{2N+1}}{(2N+1)(1-z^2)}. \tag{17}$$

Range reduction to $[1, 2]$ makes $N = 100$ terms sufficient for the comparisons above. For integer $t \geq 2$, put $n_t = \lfloor \beta t \rfloor$. Integrating the discrete discount gives

$$e_\beta(t) = \frac{(\beta+1)(t-1) - (t+n_t)\ln t + \ln(t!)}{\ln\beta} - (n_t - 3) + \frac{\ln(n_t!/6)}{\ln\beta}.$$

Together with the complete fixed rational table, these formulas reproduce every finite coefficient and waiting-length check. They also verify $h(p_*) > (1 - 2p_*)\ln 2$, which supplies the entropy tent. No optimizer or floating-point comparison is assumed.

Finally $6 < 1/\delta < 7$, so $\sigma(\delta) = -6\delta \ln \delta - (1 - 6\delta)\ln(1 - 6\delta)$. The same rational enclosures certify

$$\boxed{A + \max\{\ln 2/h(\delta), B \ln 2/\sigma(\delta)\} < 5.182975508794 < 5.182976.}$$

The file `expected-two-phase-check.py` checks these enclosures with exact fractions. Its supplementary tests verify 9,900 signed-phase identities and 1,650 modified-batch comparisons, including early minimum-cover completion and full negative waiting invoices. They support the implementation audit; the stated claims and the exact fixed-table certificate establish the theorem.

7 Quasipolynomial approximations approaching two

The same construction works for both objectives. It schedules adjacent hull blocks with almost no loss in their ranks, then optimizes a short initial part of the decision tree exactly. The second step matters: the first gives a leading coefficient close to two and an additive penalty, whereas lookahead removes that penalty from the ratio. We present only the completed algorithm; its graded recursion is an internal continuation procedure.

7.1 Algorithm and the three supporting propositions

Given $0 < \varepsilon \le 1$, use grade spacing $\lambda = 1 + \varepsilon/2$ and write $a = 2\lambda$. The only user-selected parameter is ε. The balance threshold and continuation coefficients will be fixed at the end of the proof; their particular values do not optimize the final factor. GradeDP stores a small number of cumulative rank budgets. Rollout compares each legal root query with stopping lookahead and attaching the graded continuation. All its comparisons use actual constructed costs, rather than unknown optimal trees.

Proposition 7.1 (Graded interval scheduling)**.** *For ordinary interval demands with r nested rank budgets at most B,* GradeDP *decides simultaneous feasibility and returns a schedule in $(M + 1)^{O(r)}(B + 1)^{O(r)}$ time, where M is the number of candidates. Circular demands add enumeration of their first candidate.*

For positive demands of weights $w(I)$ and zero-pattern mass p_0, the same DP, with delay levels 0 and the distinct $\lfloor \lambda^j \rfloor$, returns an ordering whose joint delay cost

$$F(Q) = 2 \sum_I w(I)(\mathrm{first}_Q(I) - 1) + 2p_0|Q|$$

is at most λ times the minimum. It uses $r = O(\varepsilon^{-1} \log n)$ grades and polynomial-bit rational arithmetic. The subtraction of one prices delay before the hitting block; the zero term prices the complete negative route.

Proposition 7.2 (Graded continuation bounds)**.** *There are fixed balance parameters for which the two graded continuations, zero on singletons, satisfy*

$$B_W(S) \le a\,\mathrm{OPT}_W(S) + 3\log_2|S|, \tag{18}$$

$$B_E(S) \le a\,\mathrm{OPT}_E(S) + 4\mathcal{E}(S). \tag{19}$$

They use only supplied lines and run in $(mn)^{O(\varepsilon^{-1}\log n)}$ time, times a polynomial in the input bit length. The expected-depth bound has no dependence on the smallest prior.

Proposition 7.3 (Lookahead removes a bounded continuation penalty)**.** *Suppose a legal continuation on every restricted state S' obeys either $B_W(S') \le a\,\mathrm{OPT}_W(S') + L_W$, or $B_E(S') \le a\,\mathrm{OPT}_E(S') + L_E w(S')$, with $a > 1$. Exact k-level lookahead is an a-approximation if, respectively, $k \ge L_W/(a-1)$ or $k \ge L_E/(a-1)$. The continuation costs at singletons are zero.*

Algorithm 11: ROLLOUT: exact initial-tree optimization with a baseline

Input: State S, objective o, lookahead j, and legal baseline routine B.
Output: The cheapest initial tree of at most j levels, completed by B.

if $|S| \le 1$ **then**
  return its leaf
if (S, o, j) *is memoized* **then**
  return the stored tree and cost
Compute the actual baseline tree $B(S)$ and retain it as the initial best choice
if $j > 0$ **then**
  foreach *supplied line ℓ splitting S* **do**
    Recurse on S_ℓ^-, S_ℓ^+ with lookahead $j-1$
    Compare $1 + \max\{v_-, v_+\}$ if $o = \mathrm{W}$, or $w(S) + v_- + v_+$ if $o = \mathrm{E}$, with the best cost
    Store a lower-cost choice and its two child trees
Memoize and return the best choice

For Theorem 7.4, use Algorithm 13 with $\lambda = 1 + \varepsilon/2$ and $k = \min\{n-1, \lceil 3\lceil \log_2 n \rceil/(1+\varepsilon)\rceil\}$. This one exactly computable cap covers both objectives.

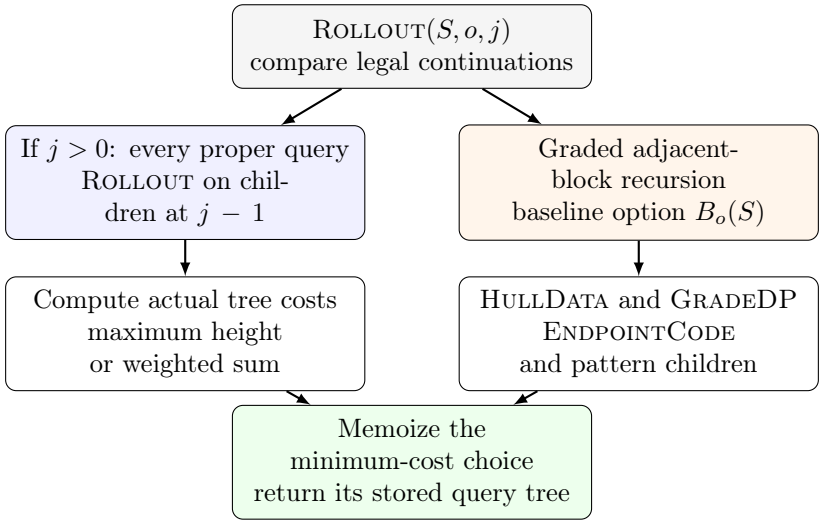


Figure 5: Exact lookahead wrapped around the quasipolynomial baseline. The arrows denote calls and the returned trees or costs. The baseline may end lookahead early at any reached state. The logarithmic cap on j bounds the number of reachable transcripts; it does not truncate the completed identification tree. The proof compares this finite search with an optimal tree, but the search evaluates only constructed trees.

Proof strategy.

1. Retain several cumulative budgets together. Exact feasibility requires their prefixes to be nested, not independently coverable.
2. Map an optimal witness path to adjacent hull blocks. A query already removed by restriction pays for its block delay; a count or entropy potential pays for endpoint recovery.
3. Compare lookahead with the first levels of an optimal tree. Each paid prefix query replaces a query previously charged at factor a, which absorbs the continuation penalty.

Theorem 7.4. *For every fixed rational $\varepsilon > 0$, maximum depth and arbitrary-prior expected depth have supplied-query quasipolynomial $(2+\varepsilon)$ approximations. For $0 < \varepsilon \le 1$, their time is*

$$(mn)^{O(\varepsilon^{-1}\log n)}$$

times a polynomial in the input bit length.

Proof. Use the continuations of Proposition 7.2 with $a = 2+\varepsilon$. Their penalties on any restricted state are at most $3\log_2 n$ and $4w(S')\ln n$, respectively. The lookahead proposition removes them after $O(\log n)$ levels. Concretely, the final calibration permits

$$q_n = \lceil \log_2 n \rceil, \qquad k = \min\{n-1, \lceil 3q_n/(1+\varepsilon)\rceil\}.$$

When the cap is $n-1$, every proper identifying tree is available, so the result is exact. Otherwise the lookahead guarantee applies. Only $(2m)^{O(k)}$ answer states are reached; multiplying by the graded-continuation cost preserves the stated time. For $\varepsilon > 1$, use the construction with $\varepsilon = 1$. □

7.2 Proofs of the supporting propositions

7.2.1 Graded schedules retain compatible prefixes

Claim 7.1 (Counts and last positions decide every interval deadline)**.** *For each grade prefix j, store its selected count c_j and last selected position p_j. When a demand $[l,q]$ ends, it has been hit within that grade exactly when $p_j \ge l$.*

Proof. Scan the candidates in spatial order. Selecting position q at grade i increments c_j and replaces p_j by q for every $j \ge i$; skipping changes neither. These are exactly the cumulative counts and last positions. No selected position exceeds the scanned endpoint q, so $p_j \ge l$ is equivalent to a hit in $[l,q]$. Reject any count exceeding its budget. Sorting the selected candidates by grade therefore meets every deadline.

Conversely, assign each candidate of a feasible ordering to its earliest available deadline bucket. Its qualifying prefixes meet the DP checks. Thus the recurrence is exact, with at most $(M+1)^r(B+1)^r$ states and $r+1$ transitions per position. For circular demands, force the first candidate, delete its hit demands and cut there. Its absolute rank consumes one unit of *every* cumulative capacity, including the total-count cap. □

Independent prefix coverage would be insufficient. For example, $[1,2]$ and $[2,3]$ can both be hit by rank one, while $\{1\}$ and $\{3\}$ can both be hit by rank two. The first requirement forces candidate 2 first, leaving too few ranks for the second. The DP stores exactly this compatibility information.

Claim 7.2 (Rounded delays bound the joint objective)**.** *Charging a demand at its earliest selected grade gives a λ approximation for F, including zero-pattern waiting.*

Proof. Let $d_0 = 0$ and let the positive d_j be the distinct $\lfloor \lambda^j \rfloor$ up to a level covering n. Grade j has capacity $d_j + 1$. At the right endpoint of demand I, charge $2w(I)d_j$ for the least j whose last selected position lies in I. Its actual first-cover delay is at most d_j. Keep the least accumulated charge for each state, and add $2p_0 c_{\text{last}}$ at completion.

For a comparison order, rank one uses grade zero. At rank $t \geq 2$, the first positive level at least $t-1$ is smaller than $\lambda(t-1)$: the preceding unrounded power is less than the integer $t-1$. Duplicate-floor plateaus do not change this argument. These assigned grades respect every cumulative capacity. Thus the minimum surrogate is at most λ times the positive delay plus the unchanged waiting cost, hence at most λF. An optimum uses at most the number of demands, after unproductive candidates are removed. For a circular order the forced first candidate has zero delay, all capacities decrease by one, and the final waiting charge is $2p_0(1 + c_{\text{last}})$. □

Algorithm 12: GRADEDP: cumulative-grade interval feasibility or weighted delay

Input: Ordinary intervals; capacities $b_0 < \cdots < b_r$; feasibility deadlines, or weights, delay levels $d_j = b_j - 1$, and zero-pattern mass p_0.
Output: A feasible graded selection, or a minimum surrogate-delay selection.

```
Initialize the state (c_0, ..., c_r; s_0, ..., s_r) to all zeros, with cost zero; all other states are unreachable
for q = 1, ..., M do
    From each reachable state, try skipping q or selecting it at grade i
    A grade-i selection increments c_j and sets s_j = q for all j ≥ i
    Reject any transition with c_j > b_j
    foreach interval I = [l, q] ending here do
        if mode is feasibility then
            Reject if s_j < l for its deadline grade j
        else
            Find the least j with s_j ≥ l; reject if none
            Add 2w(I)d_j to this transition's cost
    Keep one feasible witness, or the minimum cost, for each resulting state
In weighted mode add 2p_0 c_r to each final state's cost
Reconstruct a winning selection and order it by grade
return the selection and schedule, or infeasible
```

For circular demands enumerate the forced first candidate, remove its demands, cut there, and reduce *every* capacity by one. Residual expected costs still use the original delay levels and add $2p_0(1 + c_r)$ at completion. In maximum-depth mode append a final grade with the total-count cap if it is not already a capacity. Section 7 proves the state invariant and the $(M+1)^{O(r)}(B+1)^{O(r)}$ bound.

7.2.2 Restriction pays for a graded pattern stage

Claim 7.3 (An adjacent block has a uniform endpoint allowance)**.** *If a class A first meets block k, its discovery and endpoint queries use at most*

$$2(k-1) + z_A + 5, \qquad z_A = \lfloor \log_2(\delta|S|/|A|) \rfloor \tag{20}$$

on each route. For expected depth, the analogous overhead is at most five times the nonzero-pattern mass, plus the pattern information remaining after discovery.

Proof. At most $2(k-1)$ endpoint queries precede the hitting block. A first endpoint positive costs one query and has endpoint allowance $z_A + 4$. A second endpoint positive costs two queries but has a cached negative neighbor, reducing its allowance to $z_A + 2$. A block containing the free

origin has one real endpoint and obeys the first bound. The expected-cost alphabetic codes give the same overheads and the remaining pattern entropy. In particular, this calculation retains the negative-neighbor saving. □

For maximum depth, build all pattern-child codes first. For a trial height H, set

$$B_A = \left\lfloor \frac{H - B_W(A) - z_A - 5}{2} \right\rfloor.$$

Reject negative budgets. Round each budget down to a delay level, use block deadline $\widehat{B}_A + 1$, and cap the total selected blocks at $\lfloor H/2 \rfloor$. Enumerate the first block and apply GradeDP. The trial range ends at $\lceil (a+3)(|S|-1) \rceil$. Low-dimensional hulls use their direct extreme tests.

Claim 7.4 (Downward rounding retains every required integer rank)**.** *At $H^* = \lfloor aD + \gamma \log_2 s \rfloor$, where $D = \mathrm{OPT}_W(S)$ and $s = |S|$, every comparison class is hit by its computed deadline, provided*

$$\gamma \log_2(1/(1-\delta)) \geq 1, \qquad \gamma \log_2(1/\delta) \geq 5, \qquad \gamma \geq 1. \tag{21}$$

Proof. Write $t = t_A$, $x = \log_2(s/|A|)$ and $u_\delta = \log_2(1/\delta)$. The witness-prefix proposition gives $\mathrm{OPT}_W(A) \leq D - (t-1)$. Induction and $\lfloor u \rfloor - \lfloor v \rfloor \geq \lfloor u - v \rfloor$ give

$$H^* - B_W(A) - z_A - 5 \geq \lfloor a(t-1) + (\gamma - 1)x + u_\delta \rfloor - 5 \geq \lfloor a(t-1) \rfloor.$$

The second step uses $x > u_\delta$ and $\gamma u_\delta \geq 5$. Therefore $B_A \geq \lfloor \lambda(t-1) \rfloor$. For $t = 1$, level zero suffices. Otherwise choose the last exponent j with $\lfloor \lambda^j \rfloor \leq B_A$, including duplicate floors. Maximality implies $\lambda^{j+1} \geq B_A + 1 > \lambda(t-1)$, so $\widehat{B}_A \geq t - 1$. Rounding a real expression before using this integer credit would not give that conclusion.

Map the optimal witness path to incident hull edges. Each class is hit by rank at most t_A; append an incident edge for an uncovered terminal nonzero singleton. The comparison uses at most D blocks when the zero pattern exists, and at most $D + 1$ otherwise. The final calibration has $\gamma = 3$, giving $\lfloor H^*/2 \rfloor \geq D + 1$ for $s \geq 2$. It therefore meets the total cap. Exact graded feasibility supplies a passing trial, and the endpoint allowance bounds every positive route by H^*. The cap also bounds the full negative route. At balanced states the first inequality in (21) pays for the root query. Low-dimensional hulls use at most two real extreme tests; their nonzero classes have the same count-potential drop. This proves the maximum-depth continuation bound. □

Claim 7.5 (Weighted delay and discovery information bound the expected stage)**.** *The graded expected-depth stage satisfies*

$$W \leq 2\lambda\Delta + 5b + (I - J)/\ln 2, \qquad J \geq b\sigma(\delta).$$

Proof. The witness-edge order hits class A by rank t_A. When the zero class exists, its length is at most the witness-path length. Thus the restriction-credit proposition bounds its positive delay and zero waiting jointly by 2Δ. Without the zero class, an appended terminal singleton has its index $t_A = \ell + 1$ and no waiting term. The weighted graded DP multiplies this joint quantity by at most λ.

Add the endpoint allowance. The actual discovery transcript's positive cells each lie in one vertex cap, even though a comparison block can meet two caps. The capped-discovery entropy claim gives $J \geq b\sigma(\delta)$. Apply the stage-to-tree proposition with $A = 2\lambda$ and $B = 5$: a coefficient c satisfying $ch(\delta) \geq 1$ and $c\sigma(\delta) \geq 5$ gives $B_E(S) \leq a\,\mathrm{OPT}_E(S) + c\mathcal{E}(S)$. The final calibration permits $c = 4$. For a hull of dimension at most one, direct extreme testing costs at most $2w$. If $p_0 > 0$, its nonempty zero witness path gives $\Delta \geq p_0$, so $2w \leq 2\lambda\Delta + 5b$. If $p_0 = 0$, $2w = 2b \leq 5b$. Thus the same stage bound covers the degenerate branch without invoking the graded DP. □

Algorithm 13: Internal graded continuation for the quasipolynomial algorithm

Input: State S; objective o; rational $\lambda > 1$; fixed balance threshold $1/4$.
Output: The baseline tree of Section 7.

Stop at a singleton; at a balanced state query a balanced line and recurse
At a flat state compute HullData(S) and use full-hull edges as blocks
Handle a hull of dimension at most one by its direct extreme tests
if $o = \mathrm{W}$ **then**
    Build the pattern children once and record their heights $C_W(A)$
    for *trial heights H in increasing order* **do**
        Compute $B_A = \lfloor (H - C_W(A) - z_A - 5)/2 \rfloor$; reject negative budgets
        Round each down to the last level in $\{0\} \cup \{\lfloor \lambda^j \rfloor\}$ not exceeding it
        Use deadlines $\widehat{B}_A + 1$ and total cap $\lfloor H/2 \rfloor$ in circular GradeDP; stop at the first feasible trial
else
    Use distinct levels $d_0 = 0$, $d_j = \lfloor \lambda^j \rfloor$ through a level covering n
    Minimize the circular weighted-delay surrogate with capacities $d_j + 1$ by GradeDP and zero-pattern mass $w(A_0)$
Execute the selected blocks, use pointwise endpoint codes for W or expected endpoint codes for E, and attach recursive pattern children
Identify the zero singleton after the negative cover if present; omit it otherwise
return the contracted tree

The maximum-depth trial range is $1 \le H \le \lceil (2\lambda+3)(|S|-1) \rceil$. No optimum value or comparison witness is an algorithm input.

There are $O(\varepsilon^{-1} \log n)$ grades, with polynomial counts and spatial positions. Rational powers of λ have polynomial encoding length for fixed ε; all objectives are rational sums of actual costs. Proper disjoint-child recursion has at most $n - 1$ stages. This proves the time and legality parts of Proposition 7.2.

7.2.3 Paid prefix queries absorb continuation penalties

Claim 7.6 (The maximum-depth penalty is paid on every frontier path)**.** *An optimal height-D tree, truncated at depth k, has continuation height at most $D - k$ at each surviving frontier state.*

Proof. If $D < k$, the whole optimal tree is a lookahead candidate. Otherwise each completed frontier path has height at most $k + a(D-k) + L_W = aD - (a-1)k + L_W \le aD$. Earlier leaves are already within height D. Exact lookahead can only improve this candidate. □

Claim 7.7 (Expected prefix cost pays for every surviving frontier mass)**.** *Let P be an optimal tree's expected cost through depth k, and M_k the mass still requiring continuation there. Then $P \ge kM_k$.*

Proof. Every surviving hypothesis has paid for all k prefix queries. If R is the sum of the optimal continuation costs, then $P + R = \mathrm{OPT}_E(S)$. Completing the frontier with the baseline costs at most

$$P + aR + L_E M_k = a\,\mathrm{OPT}_E(S) - (a-1)P + L_E M_k \le a\,\mathrm{OPT}_E(S) + [L_E - (a-1)k]M_k.$$

This is at most $a\,\mathrm{OPT}_E(S)$ under the stated condition. It assumes neither a bound on the optimal tree's height nor a lower bound on its prior masses. These two claims prove the lookahead proposition. □

7.3 A simple calibration; no optimization is needed

Parameter calibration.
Fix the balance threshold at $\delta = 1/4$, the count coefficient at $\gamma = 3$, and the entropy coefficient at $c = 4$. These values affect only the continuation penalty and lookahead length, which disappear from the final ratio. Thus no table of optimized constants is needed for this scheme.

Indeed, $(4/3)^3 > 2$ and $3\log_2 4 = 6 > 5$ verify (21). Here $\sigma(1/4) = \ln 4$ and $h(1/4) \geq \frac{1}{2}\ln 2$. The elementary bounds $2/3 \leq \ln 2 < 3/4$ give $4h(1/4) > 1$ and $4\sigma(1/4) > 5$. The lower bound follows from the first term of the logarithm series in (17); the upper bound is the strict trapezoidal bound for $\int_1^2 dx/x$. Finally $4\ln n < 3\log_2 n$, so the single rational lookahead cap in the theorem covers both objectives. For singleton inputs, all procedures return the zero-query leaf.

8 Polynomial maximum depth with rank-subset scheduling

The algorithm combines exact scheduling inside a logarithmic rank cap with the priority fallback of Section 4. It constructs one code for each pattern child and compares scheduling alternatives against those codes' actual heights. The optimal tree is used only as an analytical witness that some trial is feasible.

8.1 The algorithm and its guarantee

Use the count threshold δ and potential coefficient γ of (2). For the original input size n, fix once

$$q = \lceil \log_2 n \rceil, \qquad R = \left\lfloor \frac{\gamma q}{\gamma - 1} \right\rfloor, \qquad L = R + 1. \tag{22}$$

Every child inherits this same cap. Singleton states stop, and a count-balanced supplied query is followed by the two recursive codes.

At a flat state S, construct every full hull-pattern child once and record its actual height $h_A = C_W(A)$. Try increasing integer heights H through $\lfloor (3+\gamma)(|S|-1) \rfloor$. Each trial uses the same stored children and considers the priority fallback and exact adjacent-block scheduling. For the latter, candidates are edges of the full normalized hull, with the origin kept as a free negative marker. A positive vertex arc meets a circular interval of these edge blocks. Write

$$z_A = \lfloor \log_2(\delta|S|/|A|) \rfloor, \qquad B_A = \left\lfloor \frac{H - h_A - z_A - 5}{2} \right\rfloor.$$

Reject negative budgets; otherwise assign block deadline $B_A + 1$ and use the total cap

$$\min\{L, \lfloor H/2 \rfloor\}. \tag{23}$$

There is no deadline rounding. An optional exact whole-exit decoder, proved safe below, may be included in the same trial. Return a passing alternative at the least height, attach its endpoint decoders and the stored child codes, and contract constant queries. A low-dimensional hull uses its at most two real extreme tests directly.

Algorithm 14: HybridTree: general polynomial maximum depth

Input: State S, original size n, fixed global cap $L = R + 1$, where $q = \lceil \log_2 n \rceil$ and $R = \lfloor \gamma q/(\gamma - 1) \rfloor$, and the count parameters calibrated in Section 8.3.
Output: A tree satisfying Theorem 8.1.

Stop at a singleton; at a count-balanced state query a balanced line and recurse
Compute HullData(S) and build each child's hybrid code once
Handle a low-dimensional hull directly; otherwise record child heights h_A
for $H = 1, \ldots, \lfloor (3 + \gamma)(|S| - 1) \rfloor$ **do**
    Try PriorityTrial with these stored children
    Set $z_A = \lfloor \log_2(\delta |S|/|A|) \rfloor$ and $B_A = \lfloor (H - h_A - z_A - 5)/2 \rfloor$
    if *all* $B_A \geq 0$ **then**
        Try circular SlotDP on full-hull-edge demands with deadlines $B_A + 1$ and slot cap $\min\{L, \lfloor H/2 \rfloor\}$
    Optionally try whole-exit SlotDP using actual endpoint-continuation heights and the same total cap, without the fixed-allowance budget precheck
    if *any alternative passes* **then**
        Attach its endpoint decoders and the stored child trees
        Terminate the negative cover at A_0 when present; omit that leaf otherwise
        return a passing tree, contracting constant queries

The global cap is inherited unchanged by every child. The priority fallback proves that a trial always passes; the algorithm never tests the unknown condition $\mathrm{OPT}_W(S) \leq R$.

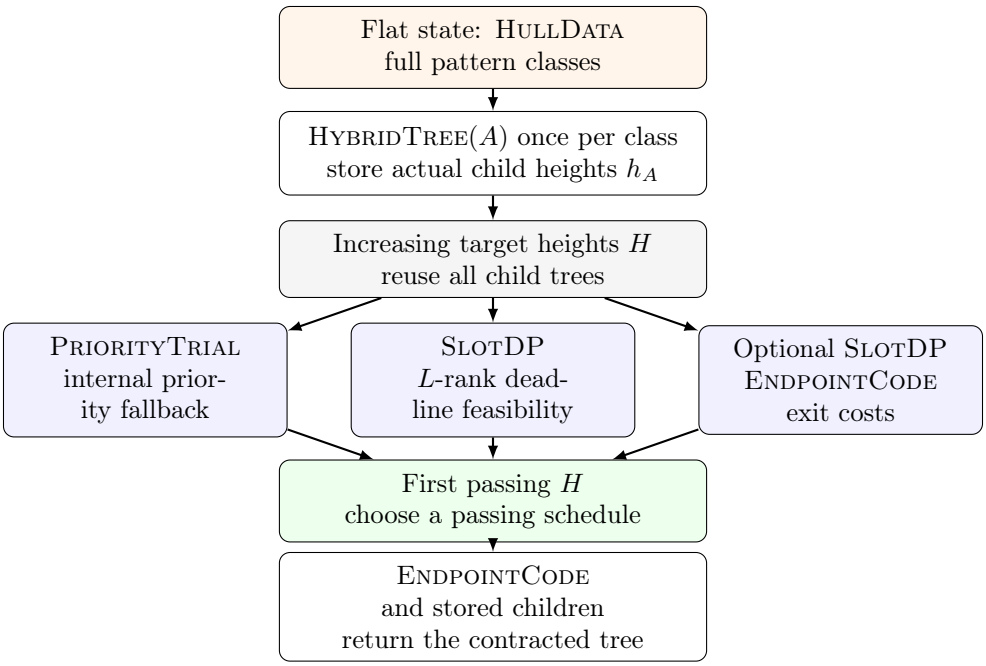


Figure 6: Construction of the polynomial maximum-depth tree at a flat state. Arrows pass explicitly computed geometry, heights, or schedules. The logarithmic cap L is fixed at the original root. The priority alternative guarantees that some height trial passes; the exact and whole-exit alternatives can accept an earlier trial. The unknown optimum is used only in the proof, never in this procedure.

Theorem 8.1 (General polynomial maximum-depth approximation)**.** *For separating supplied binary unit-cost planar line queries, Algorithm 14 runs in deterministic polynomial time and returns*

a tree satisfying

$$\boxed{C_W(\mathcal{H}) \le (2+\gamma)\,\mathrm{OPT}_W(\mathcal{H}) < 4.464966\ \mathrm{OPT}_W(\mathcal{H})}$$

with the calibrated parameters of Section 8.3. A singleton has cost zero, and every query is a supplied line.

Proof strategy.

1. Solve interval deadlines exactly by partitioning a small set of absolute ranks between spatial left and right regions.
2. Prove two recursive bounds simultaneously: the priority fallback works for every optimum; exact scheduling improves it when the optimum fits the global cap.
3. At the root, a small optimum uses the stronger bound. A large optimum absorbs the fallback's count potential. Calibrate the two count-credit inequalities only after these structural arguments.

Proposition 8.2 (Exact rank-subset scheduler)**.** *For N interval demands on M ordered candidates, with integer first-hit deadlines and at most L distinct selected candidates, feasibility and a schedule can be found in $O(M^3 3^L + NM^2)$ time with $O(M^2 2^L)$ states. Circular intervals add a factor M by enumerating the first candidate.*

Proposition 8.3 (Two simultaneous recursive guarantees)**.** *At every recursive state S, with $s = |S|$ and $D = \mathrm{OPT}_W(S)$,*

$$C_W(S) \le 3D + \Phi(s), \tag{24}$$

$$D \le R \implies C_W(S) \le 2D + \Phi(s). \tag{25}$$

The threshold R is global, so restriction preserves the second invariant's inductive hypothesis in every child.

Proposition 8.4 (Executable trials and polynomial reconstruction)**.** *Every accepted trial has complete height at most H, including its zero route when present. A passing trial is reached within the stated range; geometry, scheduling, endpoint decoding, and reconstruction have polynomial bit complexity. The optional whole-exit alternative preserves both guarantees in Proposition 8.3.*

Proof of Theorem 8.1. At the root, if $D \le R$, the stronger invariant and $D \ge \log_2 n$ give $C_W \le (2+\gamma)D$. If $D > R$, integrality implies $D > \gamma q/(\gamma - 1)$, and the fallback invariant gives

$$\frac{C_W}{D} \le 3 + \frac{\gamma \log_2 n}{D} < 3 + (\gamma - 1) = 2 + \gamma.$$

Proposition 8.4 supplies the legal polynomial construction. The calibration below gives the numerical factor. These cases cover every input without asking the algorithm to know D. □

8.2 Technical proofs of the propositions

Proof of Proposition 8.2. Candidates are $1, \ldots, M$. A demand $[a, b]$ of deadline d must contain a selected candidate assigned a rank at most d. For region $[l, r]$, let $\mathcal{I}(l, r)$ contain the original demands wholly inside it. Define $F(l, r, T)$ for an available subset $T \subseteq [L]$ of absolute ranks; unused ranks are allowed.

Claim 8.1 (Contained intervals are exactly the unhit demands)**.** *The spatial region and the absolute rank subset are sufficient state.*

Proof. An interval crossing a previously queried separator was hit there. An interval wholly in a remaining region contains no ancestor and is still unhit. Absolute ranks retain the deadlines when queries in left and right regions are interleaved; their counts alone would not. □

Claim 8.2 (The earliest available rank gives an exact recurrence)**.** *For nonempty $\mathcal{I}(l, r)$ and T, put $t = \min T$. Reject if a contained deadline is less than t; otherwise accept precisely when some $p \in [l, r]$ and partition $T \setminus \{t\} = T_L \dot\cup T_R$ satisfy*

$$F(l, p-1, T_L) \quad \textit{and} \quad F(p+1, r, T_R).$$

Proof. Query p at rank t. It hits all contained intervals containing p; every remaining demand is wholly on one side. Conversely, move the first query of a feasible state to $\min T$, which only improves hits. Its spatial Cartesian tree partitions the later assigned ranks between its left and right descendants, giving this transition. An empty demand family accepts with no selection; a nonempty family with $T = \varnothing$ rejects. □

Claim 8.3 (Reconstruction and circular cuts preserve feasibility)**.** *The recurrence has the stated time and state bounds.*

Proof. Sort selected points by their assigned ranks and remove unused slots; all hits accelerate. Precompute existence and minimum deadline of contained demands in $O(NM^2)$ time. There are $O(M^2 2^L)$ states, and the rank partitions over all masks total at most $\sum_T 2^{|T|} = 3^L$. Trying M root candidates gives the time bound. For a circle enumerate its first selected point, delete demands hit there, and cut at it. Subtract one from surviving deadlines and use $L-1$ slots; a surviving deadline at most one rejects. This adds the factor M and considers every feasible first point. □

□

Algorithm 15: SlotDP: exact interval deadlines with a rank subset

Input: Ordinary interval demands and deadlines; region $[l, r]$; available absolute ranks $T \subseteq [L]$.
Output: A feasible rank assignment, or infeasible.

```
if no original demand is wholly contained in [l, r] then
    return the empty assignment
if T = ∅ then
    return infeasible
if this state is memoized then
    return its stored result
t ← min T
if a wholly contained demand has deadline < t then
    return infeasible
foreach p ∈ [l, r] and T_L ⊆ T \ {t} do
    T_R ← T \ (T_L ∪ {t})
    Recurse on ([l, p − 1], T_L) and ([p + 1, r], T_R)
    if both assignments are feasible then
        Memoize their union with (p, t) and return it
Memoize and return infeasible
```

Sort selected candidates by their ranks and suppress unused ranks. The circular wrapper forces the first candidate, cuts there, and subtracts one from surviving deadlines and the slot cap. For the optional complete-exit variant, precompute the two oriented decoder costs by Algorithm 4; replace the deadline precheck by the two local height tests of Section 8. The same rank partitions apply.

Proof of Proposition 8.3. Induct on s, with zero-cost singleton states, proving both bounds simultaneously. A balanced child has optimum at most D and count potential at least one smaller, by Proposition 4.1. Thus a balanced query preserves either bound. The priority alternative preserves the universal bound by Proposition 4.6; smaller actual children only improve its budgets.

For the stronger flat-state bound, assume $D \le R$ and put $H^* = 2D + \Phi(s)$. All children meet the same global threshold, so their stronger inductive bound is available.

Claim 8.4 (Actual child heights give feasible comparison deadlines)**.** *Every nonzero class satisfies $B_A + 1 \ge t_A$ at H^*.*

Proof. Supporting-prefix restriction gives $\mathrm{OPT}_W(A) \le D - (t_A - 1)$. Together with the strong child bound and the integer count credit,

$$H^* - h_A - z_A - 5 \ge 2(t_A - 1). \tag{26}$$

Taking the floor after division by two gives $B_A \ge t_A - 1$. This links an unknown optimal witness to deadlines computed from actual stored codes. □

Claim 8.5 (The comparison fits both total caps)**.** *All classes can be covered in at most $D + 1$ edge blocks without missing any deadline.*

Proof. Map each witness-path query to a full-hull edge containing its supports. Enlarging a singleton support only accelerates hits; delete repeated and unproductive blocks. If the path ends at an uncovered nonzero singleton, append an incident edge at its supporting index $r + 1$, with $r \le D$. If the zero class exists it is the terminal witness leaf and no append is needed. The comparison has at most D blocks with a zero class and at most $D + 1$ otherwise, and it hits A by rank t_A. Since $D \le R$, this fits $L = R + 1$. Also $s \ge 2$ and $\gamma > 2$ give $\Phi(s) \ge 2$, so $\lfloor H^*/2 \rfloor \ge D + 1$. Both caps therefore hold. □

Claim 8.6 (A feasible block trial bounds every physical route)**.** *Class A first hit at block k has complete route height at most*

$$2(k - 1) + z_A + 5 + h_A. \tag{27}$$

Proof. A first-endpoint positive answer costs at most $2k - 1$ discovery queries and $z_A + 4$ endpoint queries. A second-endpoint first-positive answer costs at most $2k$ and is pinned, so needs only $z_A + 2$ more. A block with one real endpoint obeys the first bound. Cached answers and the origin marker cost nothing. The deadline $k \le B_A + 1$ puts either route within H^*. The all-negative route has at most $2\lfloor H^*/2 \rfloor \le H^*$ queries and identifies A_0 if present; an absent zero leaf is omitted. □

The exact scheduler therefore finds a passing trial at H^*. If the hull has dimension at most one, its at most two extreme tests learn the pattern directly, paid for by the same count credit. Both invariants follow by induction. □

Proof of Proposition 8.4. The local route certificate and (27) prove the accepted-height tests. The universal invariant supplies a passing integer trial: $\log_2 s \le D \le s - 1$ implies $3D + \Phi(s) \le \lfloor (3 + \gamma)(s - 1) \rfloor$.

Claim 8.7 (The global cap makes exact scheduling polynomial)**.** *All arithmetic and the complete reconstructed tree have polynomial size.*

Proof. For fixed γ, put $\alpha = \gamma/(\gamma-1)$. The cap is $L = \lfloor \alpha q \rfloor + 1 = O(\log n)$, so 3^L and 2^L are polynomial in n. There are polynomially many trial heights and first blocks, and at most $n-1$ nonterminal recursive stages with disjoint child sets. The witness LP, hull incidence, and endpoint codes have polynomial bit complexity. Every recursive child is smaller: balanced queries split properly, nonzero patterns have size below δs, and A_0 is a singleton. Compute q, z_A by rational powers-of-two comparisons and R by integer division of a fixed rational multiple of q. Priority weights have $O(n \log n)$ bits. Contracting constant queries gives a proper tree with at most n leaves and $n-1$ internal nodes. □

Claim 8.8 (Exact whole-exit decoding cannot weaken a trial)**.** *The optional alternative is polynomial under the same cap and accepts every fixed-allowance schedule.*

Proof. After the legal cut, each class has one realized endpoint pair; let M be the number of hull candidate positions. For a coordinate rectangle U, give a one-class leaf value h_A and compute

$$E(U) = \min_{\text{proper supplied coordinate thresholds } \ell} \big(1 + \max\{E(U \cap \ell^-), E(U \cap \ell^+)\}\big),$$

ignoring empty outcomes. There are $O(M^4)$ rectangles and $O(M)$ thresholds each; this is the exact minimum whole height among these endpoint trees, including its stored class continuations. A pinned endpoint is the one-coordinate case.

At state $([l, r], T)$, unhit demands are precisely the original intervals contained in $[l, r]$. For candidate block p in either orientation, compute the whole decoder heights $E_1(l, r, p)$ and $E_2(l, r, p)$ for its first-positive and second-first-positive cells. These cells depend only on the region and block: an outside query cannot hit a contained interval, and a crossing interval was already hit by an ancestor. At $k = \min T$, replace the fixed-deadline check by

$$2(k-1) + 1 + E_1(l, r, p) \le H, \qquad 2k + E_2(l, r, p) \le H,$$

omitting empty cells. Keep the rank partitions, circular first-block enumeration, and negative-route cap. These prefixes conservatively include cached endpoints and the origin. The explicit pointwise endpoint codes are feasible decoder trees, so minimizing whole height only lowers a passing route bound. Precomputation and the rank-subset transitions are polynomial times 3^L. □

□

8.3 Parameter calibration

Parameter calibration.

The balanced-query and flat-decoder credits require

$$\gamma(\delta) = \max\left\{ \frac{1}{\log_2(1/(1-\delta))}, \frac{5}{\log_2(1/\delta)} \right\}.$$

The decreasing and increasing terms meet at $\delta = (1-\delta)^5$. Choose the nearby rational values

$$\delta = \frac{245122334}{10^9}, \qquad \gamma = \frac{246496526}{10^8}.$$

The rational logarithm bounds (17) certify both strict credit inequalities, and therefore

$$\boxed{2 + \gamma = 4.46496526 < 4.464966.}$$

The same calibration gives $\alpha = \gamma/(\gamma - 1)$. Since $q < \log_2 n + 1$, the rank-mask contributions obey

$$3^L < 3^{\alpha+1} n^{\alpha \log_2 3}, \qquad 2^L < 2^{\alpha+1} n^{\alpha},$$

Thus each is polynomial in n, as required for the scheduling component. These are not exponents for the entire algorithm. The rational ceiling q is computed from integer powers of two.

9 Stronger algorithms for small caps

Consider instances with a distinguished hypothesis p and orient each supplied query so that its *cap* is the answer side missing p. Let P be the other N hypotheses. These assumptions concern the supplied queries themselves; a common external witness at a recursive flat state is not automatically a distinguished hypothesis. The restricted algorithms below clarify which small-cap constructions can support approximation hardness.

9.1 Exact maximum depth with bounded caps

Theorem 9.1. *If every cap has size at most a fixed integer b, maximum depth is exactly solvable in polynomial time. A running-time bound is $m^{b-1} 2^{O(b^2)} \operatorname{poly}(n, m, L)$, where L is the input bit length.*

Proof strategy.

1. A minimum cap cover gives a lower bound K from the witness spine and a tree of height at most $K + b - 1$.
2. Only the last $b - 1$ witness queries need to be guessed. Their caps contain at most $b(b - 1)$ points, so guess their residual state and find the earlier cover exactly.
3. Exact local subset tables verify every suffix exit. A feasible optimal tree supplies one of these guesses, proving exactness.

9.1.1 Supporting propositions and the algorithm

Let K be the minimum number of supplied caps covering P. For a target height $D \in [K, K + b - 1]$, let $d_0 = \max\{0, D - b + 1\}$. A suffix guess consists of at most $D - d_0$ ordered witness queries and the set $R \subseteq P$ alive immediately before them.

Proposition 9.2 (The optimum lies in a short height interval)**.** *A minimum supplied-halfplane cover is computable in polynomial time, and the optimal height lies in $[K, K + b - 1]$.*

Proposition 9.3 (Each suffix guess is exactly checkable)**.** *Guess R inside the suffix caps' union. A prefix cover of $P \setminus R$ using caps disjoint from R, followed by the suffix, gives height at most D if its cover size is at most d_0 and every suffix first-hit group A_j satisfies $j + h(A_j) \le D - d_0$. Here h is exact local identification height. These conditions are decidable in polynomial time for fixed b.*

Proposition 9.4 (Enumeration is complete and polynomial)**.** *Every height-D tree provides an accepted suffix guess. There are $O(m^{b-1})$ ordered suffixes and $2^{O(b^2)}$ candidate residual sets per suffix. For $b = 1$, a minimum cover itself is exact.*

Algorithm 16: Exact maximum depth with a distinguished miss hypothesis and bounded caps

Input: Distinguished hypothesis p; other hypotheses P; supplied caps of size at most fixed b.
Output: An exact minimum-height supplied-query tree.

if $P = \varnothing$ **then**
  return the zero-query tree
Compute the minimum supplied-halfplane cover size K of P
for $D = K, \ldots, K + b - 1$ **do**
  $d_0 \leftarrow \max\{0, D - b + 1\}$
  foreach *ordered supplied-query suffix of length* $q \le D - d_0$ **do**
    foreach R *contained in the union of its caps* **do**
      if *the suffix does not cover* R **then**
        continue
      Find a minimum cover Q of $P \setminus R$ using only caps disjoint from R
      if $|Q| > d_0$ *or the cover is infeasible* **then**
        continue
      Compute the suffix's first-hit groups A_j on $R \cup \{p\}$
      if *some* $j + c_W(\textsc{ExactTree}(A_j, \mathrm{W})) > D - d_0$ **then**
        continue
      Execute Q followed by the suffix on the all-miss spine
      Attach exact local supplied-query trees to all positive first-hit groups
      return this tree

The empty suffix permits only $R = \varnothing$. Each suffix has at most $b - 1$ queries and at most $b(b-1)$ points in its union. The minimum-cover oracle is the supplied-halfplane algorithm cited in Theorem 9.1; it is not the hull-interval oracle unless that special representation is available.

Proof of Theorem 9.1. For $P = \varnothing$, return the zero-query tree. Otherwise, Proposition 9.2 limits the target heights. Proposition 9.3 proves that every accepted guess constructs a legal tree, and Proposition 9.4 ensures acceptance at the optimum. Returning the first accepted height is exact; the stated enumeration and polynomial cover and local optimizations give the running time. □

9.1.2 Technical proofs of the height propositions

Proof of Proposition 9.2. Let K be the minimum number of supplied caps covering P. Minimum planar halfplane cover is exactly solvable in polynomial time [7]. Separability ensures that the cover exists. The path identifying p in any tree must cover P, so its height is at least K. Conversely, query a minimum cover along the all-miss spine and identify each positive hit group locally. A hit group has at most b members and therefore admits a proper identification tree of height at most $b - 1$. This gives height at most $K + b - 1$. Only these b target heights need to be tested.

□

Proof of Proposition 9.3. For a target $D \in [K, K + b - 1]$, put $d_0 = \max\{0, D - b + 1\}$. Guess an ordered suffix of $q \le D - d_0 \le b - 1$ supplied queries on the witness spine. Guess also the set $R \subseteq P$ remaining just before that suffix. Require that the suffix caps cover R; thus R is a subset of their union and has size at most $b(b-1)$. For each guess, compute a minimum cap cover of $P \setminus R$ using only supplied caps disjoint from R, and require size at most d_0. An empty suffix permits $R = \varnothing$.

Claim 9.1 (Local suffix tests include the witness leaf)**.** *The exact checks verify every positive exit and the full miss route.*

Proof. Execute the guessed suffix on $R \cup \{p\}$. For each nonempty first-hit group A_j at position j, compute its exact local identification height $h(A_j)$ by subset Bellman DP on at most b hypotheses. Require $j + h(A_j) \le D - d_0$ and $q \le D - d_0$. These tests verify the entire suffix tree, including the witness leaf. The prefix cover takes at most d_0 queries, preserves R on its all-miss route, and its positive exits cost at most $b-1$ additional queries. Thus its exits have height at most $d_0+b-1 = D$ when $d_0 > 0$; if $d_0 = 0$, the prefix is empty. The suffix starts no later than depth d_0, so an accepted guess constructs a tree of height at most D.

□

□

Proof of Proposition 9.4. For completeness, take an identifying tree of height at most D. If its witness path ends before depth d_0, use an empty suffix and $R = \varnothing$; its witness queries cover P within d_0. Otherwise take the remaining witness queries after depth d_0 as the suffix, and the actual nonwitness state there as R. The prefix caps are disjoint from R and cover $P \setminus R$. The suffix has at most $b-1$ queries and covers R; its positive subtrees satisfy the exact local-height tests. This guess is accepted. There are $O(m^{b-1})$ suffix descriptions and $2^{O(b^2)}$ candidate subsets per suffix, and all remaining computations are polynomial for fixed b. For $b = 1$, a minimum cover itself is exact. □

9.2 An EPTAS for weighted singleton and pair caps

Assume every cap has size one or two, all singleton caps for P are supplied, and the graph G of two-point caps is a disjoint union of paths and cycles. The latter condition follows for planar instances with all singleton caps: every nonwitness point is a convex-hull vertex, and a two-point cap joins consecutive vertices. Let the positive raw weights be w_0 for p and w_v for $v \in P$, and put $\Omega = w_0 + \sum_v w_v$.

For a matching M, replace each matched pair by a group of mass $w_u + w_v$ and leave unmatched vertices as singleton groups. Let $J(M)$ be the group multiset, $k = N - |M|$, and $P(M) = \sum_{uv \in M}(w_u + w_v)$. With group masses $a_1 \ge \cdots \ge a_k$, write $C(J) = \sum_j j a_j$. The exact matching invoice is

$$G(M) = w_0 k + C(J(M)) + P(M). \tag{28}$$

Theorem 9.5. *For this weighted singleton/pair path-and-cycle class, expected depth has a deterministic $(1+\varepsilon)$ approximation in time $2^{O(\varepsilon^{-3})}\,\mathrm{poly}(n, L, \log(1/\varepsilon))$ for $0 < \varepsilon \le 1$. The polynomial exponent is independent of ε; this is an EPTAS.*

Proof strategy.

1. Delete one residue class of path and cycle edges. Averaging finds a near-optimal matching on bounded-size components.
2. Enumerate components containing the heaviest vertices. Their complement has little mass, so only light components need rounding.
3. Express the exact matching invoice as a convex quadratic program on component configurations. Its fractional optimum lower-bounds the best integral invoice; the computed feasible solution has only an additive numerical optimization error.
4. Independent configuration rounding adds only a controlled variance term. Conditional expectations make it deterministic.

9.2.1 Supporting propositions and the algorithm

Work with raw weights and put $F^* = \min_M G(M) = \Omega \operatorname{OPT}_E$. The graph, weights, group counts, and count functions below describe the instance. The tuning controls are integers $K \geq 1$, $H \geq 0$ and an additive accuracy $0 < \eta \leq 1$, chosen only in the final calibration. Put $B = 2K$, the resulting component-size bound. Take the H heaviest nonwitness vertices, or all of them if fewer than H exist, and let Ω_{tail} be the mass of the others.

Proposition 9.6 (The matching invoice is exact)**.** *Minimizing* (28) *is exactly the minimum raw weighted depth, including arbitrary adaptive trees. Every matching is executable with supplied singleton and pair queries.*

Proposition 9.7 (Residue deletion and heavy enumeration)**.** *Deleting edge numbers in one residue modulo K leaves components of at most B vertices. Some residue has a surviving matching of cost at most $(1 + 1/K)F^*$. Moreover, $\Omega_{\text{tail}} \leq 2F^*/(H + 1)$, and all matching choices on components meeting a heavy vertex can be enumerated in at most 2^{BH} trials.*

For a fixed heavy trial, enumerate the matching configurations c of each remaining, light component i. Let k_{ic} be its group count, P_{ic} its matched-pair mass, and $z_{ic}(t)$ its number of groups of mass at least t. Let $Z_f(t), k_f, P_f$ be the corresponding totals of the fixed components. Fractional configuration weights satisfy $x_{ic} \geq 0$ and $\sum_c x_{ic} = 1$. Set $\mu(t) = Z_f(t) + \sum_{ic} x_{ic} z_{ic}(t)$ and minimize

$$Q(x) = w_0 \left(k_f + \sum_{ic} x_{ic} k_{ic} \right) + P_f + \sum_{ic} x_{ic} P_{ic} + \frac{1}{2} \int_0^\infty (\mu(t)^2 + \mu(t))\, dt. \tag{29}$$

Proposition 9.8 (An exact convex quadratic model)**.** *The displayed objective equals the matching invoice on integral configurations. Its minimum over the product of configuration simplices lower-bounds the best integral invoice. It is a rational convex quadratic program with $N2^{O(B)}$ variables and $O(N)$ mass breakpoints. A feasible rational solution within raw additive error $\eta\Omega$ is computable with polynomial bit complexity and time polynomial in the model size, L, and* $\log(1/\eta)$.

Proposition 9.9 (Deterministic rounding costs only light mass)**.** *From a feasible rational x, conditional expectations produce a matching of cost at most $Q(x) + (B/2)\Omega_{\text{tail}}$ in polynomial time in the model size and bit length. If every trial uses a feasible rational x within $\eta\Omega$ of its fractional optimum, the cheapest tree over all residues and heavy trials has cost at most*

$$\left(1 + \frac{1}{K} + \eta + \frac{B}{H+1}\right) F^*.$$

Algorithm 17: Expected-depth EPTAS for weighted singleton/pair paths and cycles

Input: Distinguished miss hypothesis, supplied singleton tests and pair graph G; positive raw weights; $0 < \varepsilon \leq 1$.

Output: A deterministic $(1+\varepsilon)$ approximation.

Use the controls K, H, η from Section 9.2.3; set $B \leftarrow 2K$
if $n = 1$ **then**
    return the zero-query tree
Find the H heaviest nonwitness vertices (or all of them if fewer exist)
for $r = 0, \ldots, K-1$ **do**
    Delete edges numbered r modulo K on each path or cycle
    Enumerate each remaining component's matching configurations
    foreach *joint configuration on components meeting the heavy vertices* **do**
        Fix these heavy configurations
        Solve the light-component convex program (29) to raw additive accuracy $\eta\Omega$
        foreach *unfixed light component* **do**
            Fix a configuration minimizing the conditional expected invoice, using rational first and second count moments
        Form the matched-pair and singleton groups; sort them by decreasing mass
        Query the supplied group tests on the miss spine, using one supplied singleton test to identify each pair hit
        Retain the resulting tree and its exact expected cost
return the cheapest retained tree

The component bound, heavy enumeration, convex relaxation, and conditional-expectation guarantee are proved in Theorem 9.5. No fractional tree is output.

Proof of Theorem 9.5. For $n = 1$, return the exact zero-query tree. For $n \geq 2$, choose K, H, η by Section 9.2.3. Proposition 9.7 provides a good residue and an enumerated heavy part of its near-optimal matching. Propositions 9.8 and 9.9 return cost at most $(1+\varepsilon)F^*$. Proposition 9.6 turns the matching into a legal supplied-query tree. Divide by Ω to obtain the stated expected-depth ratio. The calibration gives $BH = O(\varepsilon^{-3})$; the other stages have polynomial exponent independent of ε. □

9.2.2 Technical proofs of the EPTAS propositions

Proof of Proposition 9.6. Indeed an arbitrary tree's first-hit groups on the witness spine form a matching, sorted group order minimizes their weighted ranks, and a pair exit needs one more query. Every matching is executable using its pair tests and the supplied singleton tests. Hence minimizing (28) gives the optimum.

□

Proof of Proposition 9.7.

Claim 9.2 (One residue loses at most a $1/K$ fraction)**.** *Splitting the removed matched edges leaves a legal near-optimal matching.*

Proof. Number edges consecutively on each path or cycle. For each common residue $r \in \{0, \ldots, K-1\}$, delete edges numbered r modulo K. Each remaining component has at most B vertices, allowing the possibly longer wraparound component of a cycle.

Splitting a matched pair of weights $a \geq b$ at its original spine position into two consecutive singleton queries changes the cost of the temporary schedule, retaining the old group order, by

exactly $R_j - a$, where R_j is the later group mass plus w_0. These changes add for simultaneous splits; sorting afterward can only decrease the cost. The sum of R_j over matched positions is at most the original tree's spine cost, hence at most its full invoice. Every edge is deleted in exactly one residue trial. Applying the splits to an optimal matching and then sorting the new groups shows that some residue admits a surviving matching of cost at most $(1 + 1/K)F^*$.

□

Claim 9.3 (Heavy components leave a low-mass rounding problem)**.** *The rank lower bound controls all unfixed vertex mass.*

Proof. Order individual nonwitness weights as $w_{(1)} \geq \cdots \geq w_{(N)}$. Because a first-hit group contains at most two vertices, rearrangement gives

$$F^* \geq \sum_{i=1}^{N} \lceil i/2 \rceil w_{(i)}, \qquad \Omega_{\text{tail}} := \sum_{i>H} w_{(i)} \leq \frac{2F^*}{H+1}.$$

For each residue, enumerate all matching configurations in components containing one of the H heaviest vertices. There are at most H such components and at most 2^{BH} joint choices. This enumeration is independent of n in its exponent. The remaining components' total vertex mass is at most Ω_{tail}. If $H \geq N$, all components are fixed and there is no rounding stage.

□

□

Proof of Proposition 9.8. For any integral group multiset with count function $Z(t)$,

$$C(J) = \frac{1}{2} \int_0^\infty (Z(t)^2 + Z(t))\, dt,$$

since the groups above each threshold occupy ranks 1 through $Z(t)$. Thus (29) is an exact invoice on integral configurations and a lower-bounding convex relaxation. Its quadratic part is a positive sum of squares of affine functions. The only breakpoints are singleton and available-pair masses, at most $N + |E(G)| = O(N)$ rational values. It is therefore a rational convex quadratic program with $N2^{O(B)}$ variables over a product of simplices. After normalizing by Ω, solve it to additive accuracy η using polynomial-time convex quadratic optimization [8]. Equivalently the raw error is $\eta\Omega$. The bounded simplices, rational coefficients, and objective bound $O(n)$ give polynomial bit complexity and a feasible rational approximate solution.

□

Proof of Proposition 9.9.

Claim 9.4 (Only the quadratic count term incurs variance)**.** *The expected rounded cost is at most* $Q(x) + (B/2)\Omega_{\text{tail}}$.

Proof. Independently round each light component according to its fractional configuration distribution. Its random count $X_i(t)$ satisfies $0 \leq X_i(t) \leq B$ and $\operatorname{Var} X_i(t) \leq B\mathbb{E}X_i(t)$. Independence and the rank integral give

$$\begin{aligned} \mathbb{E}G(M) &= Q(x) + \frac{1}{2} \int \sum_i \operatorname{Var} X_i(t)\, dt \\ &\leq Q(x) + \frac{B}{2}\Omega_{\text{tail}} \leq Q(x) + \frac{B}{H+1}F^*. \end{aligned}$$

The integral of a component's count function is its total group mass, which equals its vertex mass for every configuration. The witness and pair-continuation terms are linear, so they incur no variance penalty. □

Claim 9.5 (Exact conditional expectations give a supplied tree)**.** *One component at a time can be fixed without increasing the expected invoice.*

Proof. Derandomize by exposing one component at a time and choosing a configuration with minimum conditional expected invoice. These expectations are exact rational first and second count moments at the same $O(n)$ breakpoints; each choice preserves the upper bound. Sorting the final groups yields a supplied-query tree.

□

For the good residue, enumeration contains the heavy part of an optimum surviving matching. The fractional optimum is no greater than that matching's invoice. Since $\Omega \le F^*$ in a nontrivial instance, additive QP error $\eta\Omega$ is at most ηF^*. Combining residue loss, numerical error, and rounding proves the stated bound. □

9.2.3 Calibration of the EPTAS controls

Parameter calibration.
For $0 < \varepsilon \le 1$, choose

$$K = \lceil 3/\varepsilon \rceil, \qquad B = 2K, \qquad H = \lceil 3B/\varepsilon \rceil, \qquad \eta = \varepsilon/3.$$

Then $1/K + \eta + B/(H+1) \le \varepsilon$. Also $B = O(\varepsilon^{-1})$ and $H = O(\varepsilon^{-2})$, so heavy enumeration costs $2^{O(\varepsilon^{-3})}$. Configuration enumeration, convex optimization, and conditional expectations keep a polynomial exponent independent of ε.

All-singleton availability is essential both for the invoice's converse and for splitting removed pairs. The EPTAS includes the weighted pair-path hardness instances below, and its exponential dependence on $1/\varepsilon$ is compatible with strong NP-completeness. This subclass therefore cannot establish APX-hardness unless P = NP.

10 A PTAS for uniform shallow caps in convex position

The bounded-cap restriction also permits an approximation scheme for uniform expected depth when the hypotheses are in convex position. The construction first clears short gaps in their cyclic order. Every remaining query then acts inside one bounded-size block. A count-vector dynamic program retains all choices of how to interleave queries from these blocks. Uniform weights make blocks with the same local test family interchangeable.

Theorem 10.1. *Suppose $\mathcal{H} = P \cup \{p\}$ is in strictly convex position, p is a distinguished hypothesis, and every supplied side missing p contains at most a fixed number $b \ge 1$ of hypotheses. Under the uniform prior, expected depth admits a polynomial-time approximation scheme. Every query in the returned tree is supplied; singleton caps need not be available.*

For $0 < \varepsilon \le 1$, the returned cost is at most $(1+\varepsilon)\,\mathrm{OPT}_E$. For the block length ℓ chosen in the final calibration, put $d_{b,\ell} = 2^{(b+1)\ell}$. One running-time bound is $f(b,\ell)(n+1)^{d_{b,\ell}}\,\mathrm{poly}(n,m,L)$, where L is the input bit length and f depends only on b, ℓ. The polynomial exponent depends on b and ε; the guarantee is a PTAS.

Proof strategy.

1. Clear short gaps in the convex cyclic order using supplied caps containing marked points. A cap cannot reach two remaining blocks.
2. Describe each small block by its supplied interval masks. Equal point weights make blocks with the same type and live mask interchangeable, so only their multiplicities need to be stored.
3. Optimize all block interleavings by an exact count-vector DP, using exact supplied-query codes on positive branches.
4. Charge the clearing prefix against the witness-spine rank lower bound. Large enough blocks make that charge at most ε times the optimum.

10.1 Supporting propositions and the algorithm

Write $N = |P| = n - 1$, $G(T) = \sum_{h \in \mathcal{H}} d_T(h)$, and $\mathrm{OPT}_G = \min_T G(T) = n\,\mathrm{OPT}_E$. The control ℓ is a positive integer with $\ell \geq \max\{2b, b(b-1)\}$, calibrated only after the proof. When $N < \ell$, exact subset Bellman optimization solves the instance. Assume henceforth that $N \geq \ell$.

Proposition 10.2 (Clearing gaps isolates small residual blocks)**.** *Partition the convex cyclic order into pieces of length ℓ and mark the first b points of each piece, or the whole piece if shorter. The number r of marked points satisfies*

$$r \leq b\lceil N/\ell \rceil \leq \frac{2bN}{\ell}. \tag{30}$$

A legal supplied-query prefix clears all marked points in at most r spine queries and has total raw contribution

$$G_{\mathrm{prefix}} \leq r(N+1) + b(b-1)r \leq 2r(N+1). \tag{31}$$

Every remaining cap acts inside one residual block of size at most ℓ.

Proposition 10.3 (Block types retain all continuation information)**.** *A residual block's type is its size and its family $\mathcal{F}_T$ of nonempty supplied interval masks. There are at most $2^{b\ell}$ types and at most $d_{b,\ell}$ pairs (T, R) of a type and a nonempty live mask. Under uniform weights, blocks with the same (T, R) can be exchanged without changing the optimum continuation cost. Each mask retains a realizing query for every physical block of its type.*

Let $c_{T,R}$ count blocks in local state (T, R), and put $W(c) = 1 + \sum_{T,R} c_{T,R}|R|$, including the distinguished hypothesis. For each type, use Algorithm 1 to compute the exact raw local identification cost $g_T(A)$ for every subset A using its supplied masks; $g_T(A) = 0$ for $|A| \leq 1$. The positive branch has no distinguished witness. Let $e_{T,R}$ be a unit count vector, with $e_{T,\varnothing} = 0$.

Proposition 10.4 (The residual count-vector recurrence is exact)**.** *The optimum residual raw cost is*

$$\begin{aligned} F(0) &= 0, \\ F(c) &= W(c) + \min_{\substack{c_{T,R}>0,\ q\in\mathcal{F}_T \\ A=R\cap q\neq\varnothing}} \left\{ g_T(A) + F\big(c - e_{T,R} + e_{T,R\setminus A}\big) \right\}. \end{aligned} \tag{32}$$

The DP has at most $(N+1)^{d_{b,\ell}}$ states, polynomial bit complexity, and a supplied-query reconstruction realizing its value. The recurrence includes every adaptive interleaving of the blocks.

Proposition 10.5 (The clearing charge is small relative to the optimum)**.** *The exact residual optimum is at most* OPT_G*, and*

$$\mathrm{OPT}_G \geq \sum_{i=1}^{N} \lceil i/b \rceil \geq \frac{N(N+1)}{2b}. \tag{33}$$

The prefix followed by the exact residual tree therefore has $G_{\mathrm{alg}} \leq (1 + 8b^2/\ell)\,\mathrm{OPT}_G$*.*

Algorithm 18: Uniform expected-depth PTAS for bounded convex-position caps

Input: Uniform hypotheses $P \cup \{p\}$ in convex position; supplied caps missing p, each of size at most fixed b; $0 < \varepsilon \leq 1$.
Output: A supplied-query tree of expected cost at most $(1+\varepsilon)\,\mathrm{OPT}_E$.

$N \leftarrow |P|$; use ℓ from Section 10.3
if $N < \ell$ **then**
  return ExactTree$(P \cup \{p\}, \mathrm{E})$ with raw unit weights
Partition the convex cyclic order of P into pieces of length ℓ; mark their first $\min\{b, \text{piece size}\}$ points as D
$U \leftarrow P$; start an empty miss-spine prefix
while $D \cap U \neq \varnothing$ **do**
  Choose $x \in D \cap U$ and a supplied cap q containing x
  $A \leftarrow U \cap q$
  Append q to the prefix; attach a legal local code for A of height at most $|A| - 1$ on its hit side
  $U \leftarrow U \setminus A$; continue on the miss side
Intersect the unmarked pieces with U to obtain the residual blocks
Store each physical block's map from its masks to realizing supplied queries; group identical mask families as types T and compute every local cost $g_T(A)$ by ExactTree
Compute the initial count vector c of the full residual block masks
Evaluate (32), memoizing minimizing transitions
Backtrack from c: select a physical block in the chosen state, query that block's stored realizing line, and translate the local positive code using the same block's query map
Attach the reconstructed continuation to the prefix's miss leaf
return the resulting tree

Proof of Theorem 10.1. Choose ℓ by Section 10.3. If $N < \ell$, exact subset optimization takes time depending only on b, ε, apart from polynomial input processing. Otherwise, Proposition 10.2 constructs the legal prefix, and Propositions 10.3 and 10.4 give an exact supplied-query residual tree with the stated time bound. Proposition 10.5 and the calibration yield $G_{\mathrm{alg}} \leq (1+\varepsilon)\,\mathrm{OPT}_G$. Divide by n to obtain the expected-depth ratio. No step introduces an unsupplied singleton test. □

10.2 Technical proofs of the supporting propositions

Proof of Proposition 10.2.

Claim 10.1 (Marked gaps prevent caps from crossing blocks)**.** *Every residual cap meets at most one unmarked block.*

Proof. Each cap is a cyclic interval in the convex order of P: a line crosses the boundary of its convex polygon at most twice. Partition that order into consecutive pieces of length ℓ, with a possibly shorter last piece. Mark the first b points of each piece, or the whole piece when its size is at most b. Let D be the marked set and $r = |D|$. Since $N \geq \ell$, counting marked points gives (30). The unmarked portions are blocks of at most ℓ points. Between neighboring nonempty blocks,

including across the cyclic wraparound, there are at least b marked points. A cyclic cap of size at most b cannot meet two such blocks, because it would also contain an entire intervening gap and hence at least $b + 2$ points.

□

Claim 10.2 (Supplied queries clear the marked set with controlled cost)**.** *No private or arbitrary singleton query is required by the prefix.*

Proof. While a marked point x remains on the miss spine, query any supplied cap containing x. Such a cap exists because the supplied tests distinguish x from p. Identify its first-hit group by a proper local supplied-query tree, and continue on the miss side. Separation gives a local height at most $|A| - 1 \leq b - 1$ for each hit group A: repeatedly query a line separating two remaining hypotheses. Each spine query clears a new marked point, so at most r are used. Additional unmarked points may leave through these exits. The residual portions of the blocks still have size at most ℓ, and every residual cap lies in one block.

The prefix's spine nodes cost at most $r(N + 1)$ in total. At most br points leave through its hit exits, each with at most $b - 1$ additional local queries. Thus the complete prefix contribution satisfies (31), using $N \geq \ell \geq b(b - 1)$.

□

□

Proof of Proposition 10.3. Relabel the points of each residual block by their positions. Its type T is its size together with the family $\mathcal{F}_T$ of all distinct nonempty supplied-query masks on that block. No cap can wrap around a block's ends, since it would contain an entire marked gap. Deleting additional points preserves interval order. Thus every mask is an ordinary interval of at most b positions. Padding to ℓ positions gives at most $b\ell$ possible masks, hence at most $2^{b\ell}$ families. The union of a family's masks is its full block, since every point is separated from p, so the family also determines the block size. A local state (T, R) records a nonempty live mask R in a block of type T. There are at most $d_{b,\ell}$ local states.

Two blocks of the same type have identical available splits after relabelling positions. Store a realizing supplied query separately for each physical block and mask: a representative line from a different block need not work on the chosen block. Each realizing query misses all other residual blocks. Uniform costs depend only on their live cardinalities, so exchanging blocks of the same (T, R) preserves every continuation cost. The count vector therefore retains all needed information.

□

Proof of Proposition 10.4.

Claim 10.3 (Every adaptive query is one recurrence transition)**.** *A positive branch is local; the miss branch changes only one block state.*

Proof. Every transition removes a live point, so the recurrence is acyclic. At any proper query, its hit group lies in one block and its miss branch retains p and the other blocks. The positive subtree can be optimized locally, giving precisely the recurrence's lower bound for every adaptive tree. Conversely, a transition chooses a physical block of the specified type and live mask, and an original supplied query realizing q. That query misses every other residual block. Its stored local positive code and miss continuation realize the recurrence. Backtracking therefore reconstructs a legal identifying tree, including all choices of block interleaving.

□

Claim 10.4 (The compressed state space has the stated bit complexity)**.** *Only counts, supplied masks, and integral costs are needed.*

Proof. Each count lies between zero and N, giving at most $(N+1)^{d_{b,\ell}}$ states. Local Bellman tables and the transitions per state depend only on ℓ, and constructing the types takes polynomial time in the input. All raw costs on reachable states are integral and at most $n(n-1)$, so their bit complexity is polynomial.

□

□

Proof of Proposition 10.5. Restrict an optimal full tree to the residual hypotheses, suppressing constant-answer nodes. Restriction can only decrease depths, so the exact residual cost from (32) is at most OPT_G. On the other hand, the witness spine of any full tree has first-hit groups of size at most b, giving the rank lower bound (33). Combining (31), (30), and (33) gives

$$\frac{G_{\mathrm{alg}} - \mathrm{OPT}_G}{\mathrm{OPT}_G} \le \frac{4br}{N} \le \frac{8b^2}{\ell}.$$

Dividing by n proves the expected-depth guarantee. □

10.3 Calibration of the block length

Parameter calibration.
For $0 < \varepsilon \le 1$, choose

$$\ell = \max\left\{2b,\ b(b-1),\ \left\lceil \frac{8b^2}{\varepsilon} \right\rceil \right\}.$$

The first two bounds ensure gap separation and the prefix estimate; the last gives $8b^2/\ell \le \varepsilon$. For fixed b, ε, ℓ and $d_{b,\ell}$ are constants, yielding the claimed PTAS time.

The proof uses only the cyclic-interval property of the caps. In particular it also applies when the nonwitness points alone are in convex position. For fixed b, a constant-factor hardness reduction through uniform one-layer shallow-cap instances would contradict this PTAS unless P = NP. A construction based on bounded-size caps must therefore prove interactions beyond this cyclic-interval subclass to establish such a gap.

11 Randomized per-hypothesis depth and optimal mixing

Randomization leads to a distinct objective when the hidden hypothesis is fixed before the random bits are drawn. Every execution must identify it without error, but different hypotheses may be favored by different sampled trees. For a distribution μ over legal supplied-query trees, define

$$R(\mu) = \max_{h \in \mathcal{H}} \mathbb{E}_{T\sim\mu} d_T(h), \qquad R^* = \min_{\mu} R(\mu). \tag{34}$$

Thus the expectation is taken separately for each hypothesis, before taking the maximum. We call this the Las Vegas per-hypothesis depth objective. It is different from the two objectives studied above. For a fixed prior π, linearity gives

$$\min_{\mu} \sum_h \pi_h \mathbb{E}_\mu d_T(h) = \min_T \sum_h \pi_h d_T(h), \qquad \min_{\mu} \mathbb{E}_\mu\big[\max_h d_T(h)\big] = \min_T \max_h d_T(h).$$

These mixtures preserve the optima of ordinary expected depth and expected tree height; the order of expectation and maximum in (34) changes the optimum.

A strict planar separation. Take the three points (t, t^2), $t = 0, 1, 2$, and supply the three lines

$$4y - 8t_0x + 4t_0^2 - 1 = 0, \qquad t_0 = 0, 1, 2.$$

Each line's negative side contains only its own point: its expression there is -1, and its value at another point is at least 3. The three possible proper depth vectors are permutations of $(1, 2, 2)$. Their uniform mixture gives every hypothesis expected depth $5/3$, whereas deterministic maximum depth is 2. Every tree has uniform-prior expected depth at least $5/3$, so $R^* = 5/3$ exactly. All coordinates and line coefficients are integers, and no hypothesis lies on a line.

11.1 One algorithm and its supporting propositions

Let $n = |\mathcal{H}| \geq 2$ and $L = n - 1$. Contracting queries constant on their current state makes every legal identification tree proper, with all depths at most L. Write $\mathcal{T}$ for the finite family of these trees and

$$c_\pi(T) = \sum_{h \in \mathcal{H}} \pi_h d_T(h), \qquad \mathrm{OPT}_E(\pi) = \min_{T \in \mathcal{T}} c_\pi(T).$$

The family $\mathcal{T}$ defines the comparison optimum; the algorithm stores only the trees that it constructs. In this comparison notation, a prior may have zero masses, but every tree still identifies all of $\mathcal{H}$.

Use an expected-depth oracle $\mathcal{A}_E$ with a fixed rational guarantee $\alpha \geq 1$:

$$c_\pi(\mathcal{A}_E(\pi)) \leq \alpha\, \mathrm{OPT}_E(\pi) \quad \text{for every strictly positive rational prior } \pi. \tag{35}$$

The polynomial or quasipolynomial expected-depth construction supplies this oracle. A rational learning rate $0 < \eta \leq 1$ and a round count K will be calibrated at the end for the desired tolerance.

Initialize equal positive weights. In each round normalize them to a prior, construct its oracle tree, and increase a hypothesis's weight according to its depth in that tree. This directs later trees toward hypotheses that earlier trees identify slowly. If $\eta = p/q$ in lowest terms, use the integer update

$$w_h \leftarrow w_h\big(qL + p\, d_{T_t}(h)\big). \tag{36}$$

Its common factor qL cancels on normalization. This is the usual multiplicative-weights update, implemented exactly with positive integers. The underlying game and LP method is standard [9, 10]; the depth-scaled analysis below is self-contained.

After constructing $T_1, \ldots, T_K$, optimize their actual mixture by the *restricted master LP*

$$\begin{aligned} &\text{minimize} && r, \\ &\text{subject to} && \sum_{t=1}^{K} \mu_t d_{T_t}(h) \leq r && (h \in \mathcal{H}), \\ & && \sum_{t=1}^{K} \mu_t = 1, \quad \mu_t \geq 0 \ (1 \leq t \leq K), \quad r \geq 0. \end{aligned} \tag{37}$$

Retain an optimal basic mixture, sample from its exact rational probabilities, and execute the selected tree. The LP improves the mixture used by the algorithm; it is not merely an analysis device.

Algorithm 19: LP-compressed Las Vegas per-hypothesis identification

Input: Identifiable $\mathcal{H}$, supplied lines $\mathcal{L}$; expected-depth oracle $\mathcal{A}_E$ of fixed rational factor α; rational tolerance $0 < \varepsilon \le 1$.

Output: A rational mixture of at most $|\mathcal{H}|$ legal identifying trees.

if $|\mathcal{H}| = 1$ **then**
  return its leaf with probability one
Set $n = |\mathcal{H}|$, $L = n - 1$, and choose η, K by (40); write $\eta = p/q$ in lowest terms
Set $w_h = 1$ for every $h \in \mathcal{H}$
for $t = 1, \dots, K$ **do**
  Set $\pi_{h,t} = w_h / \sum_g w_g$
  Construct $T_t = \mathcal{A}_E(\pi_t)$ and contract constant queries
  Record every depth $d_{T_t}(h)$
  foreach $h \in \mathcal{H}$ **do**
    Set $w_h \leftarrow w_h(qL + p\, d_{T_t}(h))$
Solve (37), retaining an optimal basic solution (μ, r)
Discard trees whose probability is zero
return the retained rational mixture; sample it exactly and execute the selected tree on each identification run

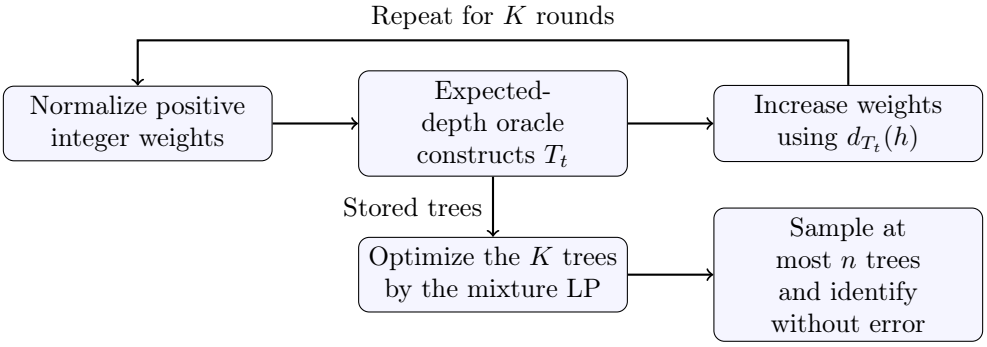


Figure 7: The oracle adapts to hypotheses previously identified slowly. The master LP jointly optimizes the stored trees' sampling probabilities; it never introduces an additional line query.

Proposition 11.1 (The comparison game and its entropy lower bound)**.** *The optimum in* (34) *also covers adaptive zero-error randomized algorithms that terminate almost surely on every hypothesis. It satisfies*

$$R^* = \max_{\pi \in \Delta(\mathcal{H})} \mathrm{OPT}_E(\pi) \ge \log_2 n. \tag{38}$$

The maximum permits zero masses. No zero-mass prior is needed by Algorithm 19.

Proposition 11.2 (Depth-scaled multiplicative weights)**.** *For any sequence of proper oracle trees, let π_t be the normalized weights before round t. The uniform mixture of the generated trees satisfies*

$$R(\mu^{\mathrm{unif}}) \le (1 + \eta)\frac{1}{K}\sum_{t=1}^{K} c_{\pi_t}(T_t) + \frac{2L \ln n}{\eta K}. \tag{39}$$

Proposition 11.3 (Exact rational construction and sparse optimal mixing)**.** *Every iterative prior is strictly positive and rational, with encoding length polynomial in n, K and the encoding length of η. The LP* (37) *is polynomial in size and has an optimal basic solution supported on at most*

n trees. Its value is at most $R(\mu^{\mathrm{unif}})$. Its probabilities have polynomial encoding length and admit exact sampling with an expected polynomial number of fair random bits. Every sampled query run is correct and uses at most $n-1$ supplied tests.

Proof strategy.

1. View hypotheses and legal trees as the two sides of a finite game. Its hardest prior lower-bounds every randomized strategy.
2. Increase the weight of hypotheses with large current depths. The potential comparison bounds each hypothesis's average depth by the oracle's average weighted cost.
3. Optimize the generated mixture by a small LP. The entropy lower bound makes the regret term a multiplicative tolerance; a basic solution leaves at most n trees to sample.

Theorem 11.4 (Transfer to randomized per-hypothesis depth)**.** *For every rational $0 < \varepsilon \le 1$, Algorithm 19, with the calibration at the end of this section, satisfies*

$$R(\mu) < (\alpha + \varepsilon)R^* \qquad (n \ge 2).$$

It makes $O(n/\varepsilon^2)$ calls to the expected-depth oracle, uses only positive rational priors, and returns a mixture of at most n legal trees. If the oracle is polynomial-time in its rational input, preprocessing is polynomial in the input size and $1/\varepsilon$. A singleton is identified without a query.

Proof. The comparison-game proposition and the oracle guarantee give $c_{\pi_t}(T_t) \le \alpha R^*$ in every round. Apply the regret proposition and use $\ln n \le (\ln 2)R^* < R^*$ to obtain

$$R(\mu^{\mathrm{unif}}) < \left[(1+\eta)\alpha + \frac{2L}{\eta K}\right] R^*.$$

The final calibration makes the bracket at most $\alpha + \varepsilon$. The compression proposition transfers this bound to the optimized mixture and supplies positivity, legality, and bit complexity. The chosen K gives the stated number of calls. □

11.2 Proofs of the supporting propositions

11.2.1 The hardest prior is the dual comparison object

Claim 11.1 (Random seeds reduce to a finite proper-tree mixture)**.** *Any zero-error randomized algorithm terminating almost surely on each hypothesis induces a mixture over $\mathcal{T}$ with no larger depths.*

Proof. Condition on a complete random seed. Since $\mathcal{H}$ is finite, outside a probability-zero union of events the resulting deterministic policy terminates correctly on every hypothesis. The union of its n finite paths is a finite identification tree. Delete unreachable branches and contract queries constant on their current states. Every retained query splits a state properly, so the resulting tree has n leaves, $n-1$ internal nodes, and maximum depth at most L. The finite supplied query family admits only finitely many such labeled trees. Pushing the seed distribution onto these trees gives the required mixture. Conversely, sampling any such mixture gives a legal zero-error strategy. □

Claim 11.2 (Finite LP duality identifies the hardest prior)**.** *The finite-tree mixture optimum equals* $\max_{\pi \in \Delta(\mathcal{H})} \min_{T \in \mathcal{T}} c_\pi(T)$.

Proof. Write the full mixture LP with one variable μ_T for each $T \in \mathcal{T}$, the depth inequalities, and normalization $\sum_T \mu_T = 1$. Its dual is

$$\begin{aligned} \text{maximize} \quad & \lambda, \\ \text{subject to} \quad & \lambda \le \sum_h \pi_h d_T(h) \qquad (T \in \mathcal{T}), \\ & \pi_h \ge 0, \qquad \sum_h \pi_h = 1. \end{aligned}$$

Here one may leave the primal variable r unrestricted: for $n \ge 2$, every depth is at least one, so its inequalities already force $r \ge 1$. The coefficient of r in the Lagrangian forces the dual normalization. The finite primal is feasible and bounded, and finite LP duality gives equality of its optimum and this dual optimum. For fixed π, the largest feasible λ is exactly $\min_T c_\pi(T)$. The dual's zero masses affect only the comparison object; the algorithm's priors will remain positive. □

Claim 11.3 (The uniform prior provides a logarithmic lower bound)**.** *Every identifying binary tree has uniform-prior expected depth at least* $\log_2 n$*.*

Proof. Its leaf depths satisfy Kraft's inequality. By convexity of 2^{-x},

$$2^{-\frac{1}{n}\sum_h d_T(h)} \le \frac{1}{n}\sum_h 2^{-d_T(h)} \le \frac{1}{n}.$$

Taking logarithms gives the claim. Applying the dual identity to the uniform prior proves (38). □

11.2.2 A weight potential bounds every hypothesis's depth

Claim 11.4 (Individual weight growth is paid by total weight growth)**.** *For the scaled multiplicative weights and* $g_{h,t} = d_{T_t}(h)/L \in [0,1]$*,*

$$\ln(1+\eta)\sum_{t=1}^{K} g_{h,t} \le \ln n + \eta \sum_{t=1}^{K}\sum_{g} \pi_{g,t} g_{g,t} \quad \textit{for every } h.$$

Proof. Write $\widehat{w}_{h,t} = w_{h,t}/(qL)^{t-1}$ for the integer weights with their common factor removed. This leaves their normalized prior unchanged and makes their update $1+\eta g_{h,t}$. Concavity on $[0,1]$ gives $\ln(1+\eta x) \ge x\ln(1+\eta)$, so each final weight obeys

$$\ln \widehat{w}_{h,K+1} \ge \ln(1+\eta)\sum_t g_{h,t}.$$

For their sum $\widehat{W}_t$, initially $\widehat{W}_1 = n$ and

$$\frac{\widehat{W}_{t+1}}{\widehat{W}_t} = 1 + \eta \sum_h \pi_{h,t} g_{h,t}.$$

Using $\ln(1+x) \le x$ bounds its final logarithm by the right-hand side in the claim. Since $\widehat{w}_{h,K+1} \le \widehat{W}_{K+1}$, the two inequalities give the result. □

Claim 11.5 (The potential comparison gives the stated regret)**.** *The previous claim implies* (39)*.*

Proof. Multiply by $L/[K\ln(1+\eta)]$ and maximize over h. This yields

$$R(\mu^{\text{unif}}) \le \frac{\eta}{\ln(1+\eta)}\frac{1}{K}\sum_t c_{\pi_t}(T_t) + \frac{L\ln n}{K\ln(1+\eta)}.$$

The integral for $\ln(1+\eta)$ gives $\ln(1+\eta) \ge \eta/(1+\eta) \ge \eta/2$ when $0 < \eta \le 1$. Substituting these two bounds proves the regret proposition. □

11.2.3 Integer updates and a basic LP solution give a finite strategy

Claim 11.6 (All oracle inputs have strictly positive rational weights)**.** *The updates* (36) *preserve positivity and have polynomial encoding length.*

Proof. The initial weights are one. Every update factor is a positive integer at most $(p+q)L$. After K rounds, each weight therefore has $O(K\log((p+q)L)+1)$ bits. Normalization adds at most $\lceil \log_2 n \rceil$ bits to the denominator. All priors are strictly positive, so every call meets the oracle's hypothesis. The generated proper trees have $O(nK)$ total nodes; their depth matrix consists of integers at most L. □

Claim 11.7 (An optimal basic mixture uses at most n trees)**.** *The LP* (37) *has an optimal basic solution with at most n positive tree probabilities and polynomial-bit rational coefficients.*

Proof. Its uniform mixture is feasible. Replacing every depth inequality by

$$\sum_t \mu_t d_{T_t}(h) + s_h - r = 0, \qquad s_h \ge 0,$$

gives standard form with $n+1$ equalities and nonnegative variables μ_t, s_h, r. Because $n \ge 2$, every depth is at least one and normalization forces $r \ge 1$. Thus r is positive. A basic solution has at most $n+1$ positive variables, leaving at most n for tree probabilities. Dependent equalities can only reduce this count. A finite optimal basic solution exists, and its integer basis coefficients have absolute value at most L. Cramer's rule and Hadamard's determinant bound give polynomial encoding length for its probabilities. Rational LP algorithms find an optimal basic solution in polynomial time [5]. Since the uniform mixture is feasible, the optimized value cannot exceed $R(\mu^{\text{unif}})$. □

Claim 11.8 (The compressed mixture is exactly sampleable)**.** *The optimized rational mixture gives a legal zero-error execution with at most L queries and expected polynomial sampling cost.*

Proof. Take a common denominator D for its at most n probabilities; its encoding length is polynomial. Draw a uniform integer from $\{0,\dots,D-1\}$ by fair bits and rejection, then use the cumulative integer probability intervals. If $D = 1$, no random bits are needed. Otherwise each attempt uses $\lceil \log_2 D \rceil$ bits and is accepted with probability greater than $1/2$, so the expected number of attempts is less than two. The sampled tree identifies every hypothesis and has depth at most L. This completes the compression proposition. □

The master LP also supplies an upper bound on R^*, while

$$\max_t \frac{c_{\pi_t}(T_t)}{\alpha} \le R^*$$

supplies a lower bound by (35) and (38). Their ratio is an instance-specific certificate. One may stop early when this certificate meets the desired factor; the fixed-round guarantee requires no such event.

11.3 Calibration and the strongest resulting guarantees

Parameter calibration.
For a fixed rational oracle coefficient $\alpha \geq 1$ and target $0 < \varepsilon \leq 1$, choose

$$\eta = \frac{\varepsilon}{2\alpha}, \qquad K = \left\lceil \frac{8\alpha(n-1)}{\varepsilon^2} \right\rceil. \tag{40}$$

Then $\alpha\eta = \varepsilon/2$ and $2L/(\eta K) \leq \varepsilon/2$. These are rational choices; no logarithm is computed by the weight updates.

The current expected-depth certificate in Section 6.3 gives the rational valid oracle coefficient $\alpha = 5.18297551$. Choosing $\varepsilon = 1/4000000$ in the single mixture algorithm yields

$$R(\mu) < 5.18297576\, R^* < 5.182976\, R^*.$$

This is the best polynomial factor established here for the randomized per-hypothesis objective. The small tolerance makes the iteration constant large; the parameterized theorem and the optimized master LP allow more practical tolerance choices.

Corollary 11.5 (Quasipolynomial randomized per-hypothesis approximation)**.** *For every fixed rational $\varepsilon > 0$, the same LP-compressed mixture algorithm has a supplied-query quasipolynomial $(2+\varepsilon)$ approximation for R^*. For $0 < \varepsilon \leq 1$, its time is $(mn)^{O(\varepsilon^{-1}\log n)}$ times a polynomial in input bit length, and its mixture is supported on at most n trees.*

Proof. Use the expected-depth oracle of Theorem 7.4 with coefficient $\alpha = 2 + \varepsilon/2$, and apply the transfer theorem with tolerance $\varepsilon/2$. The factor is strictly below $2 + \varepsilon$ for $n \geq 2$. There are $\lceil 32\alpha(n-1)/\varepsilon^2 \rceil$ oracle calls, whose positive rational priors have polynomial encoding length. These calls and the master LP preserve the oracle's stated quasipolynomial running-time form. For $\varepsilon > 1$, use $\varepsilon = 1$; a singleton again returns its leaf. □

12 Allowing hypotheses on supplied lines

Algorithm 20 implements the reduction below. In this section a supplied line returns one of three answers: negative, zero, or positive. The *zero*, or *on-line*, answer means the hypothesis lies on the queried line, and its answer class is $S^0_\ell = \{h \in S : f_\ell(h) = 0\}$. A *ternary decision tree* has an outgoing edge for each possible answer; a proper such tree retains only nonempty outcomes. Leaf depths and the two cost objectives are defined exactly as in the binary model. Use the two virtual binary predicates

$$b_1(h) = \mathbf{1}[f_\ell(h) < 0], \qquad b_2(h) = \mathbf{1}[f_\ell(h) \leq 0].$$

Their pair of bits distinguishes the three outcomes. Requesting either bit is implemented by querying the original line once and storing its full sign. Both bits are then known. For analysis we count each requested virtual bit as a separate binary query; the actual number of supplied-line queries can only be smaller.

These binary labelings have planar off-boundary representations. Any query whose answer is constant on the hypothesis set can be discarded. For every remaining query, choose ϵ_ℓ to be half the minimum nonzero $|f_\ell(h)|$ on the finite hypothesis set. This rational value is computable in polynomial time and has polynomial encoding length. Then b_1 is represented by $f_\ell(x) < -\epsilon_\ell$ and b_2 by $f_\ell(x) < \epsilon_\ell$. Neither shifted boundary contains a hypothesis. The shifts are used only for preprocessing; their answers are obtained from the original supplied line.

Symbol	Meaning
$f_\ell(h)$	Affine value whose sign is the answer for hypothesis h.
$b_1(h),\ b_2(h)$	Virtual bits for $f_\ell(h) < 0$ and $f_\ell(h) \le 0$; together they encode the ternary sign.
ϵ_ℓ	Positive shift smaller than every nonzero $\lvert f_\ell(h)\rvert$ for this finite state.

Algorithm 20: Transfer a binary algorithm to on-line outcomes

Input: Finite hypotheses, supplied ternary lines, binary tree-construction routine B.
Output: An executable tree using the original supplied lines.

1 **foreach** *nonconstant supplied line* ℓ **do**
2 $\quad \epsilon_\ell \leftarrow \frac{1}{2} \min_{h : f_\ell(h) \neq 0} \lvert f_\ell(h)\rvert$
3 $\quad$ Represent virtual bits $b_1 = \mathbf{1}[f_\ell(h) < 0]$ and $b_2 = \mathbf{1}[f_\ell(h) \le 0]$ by the two shifted off-boundary lines
4 Construct a virtual binary tree with B
5 Execute a requested virtual bit by querying its original line, caching the entire ternary sign; answer further requests from that cache
6 **return** the contracted original-line tree

Lines constant on the whole hypothesis set are discarded before taking the minimum nonzero affine value. Proposition 12.1 proves that the approximation ratio increases by at most two.

Proposition 12.1 (Boundary reduction)**.** *An α-approximation for either binary objective gives a 2α-approximation for the corresponding three-answer objective.*

Proof. Simulate every query of an optimal original tree by its two virtual bits. Each hypothesis uses at most twice as many binary queries, so the virtual optimum is at most twice the original optimum for either objective. Conversely, execute a constructed virtual tree by original line queries and cached signs. Its actual depth is no larger than its virtual depth on every hypothesis. The two comparisons prove the factor-two transfer. □

Corollary 12.2 (Constant approximations with on-line outcomes)**.** *With on-line answers, expected depth has a polynomial-time approximation factor below* 10.365952 *for arbitrary positive priors, and deterministic maximum depth has a factor below* 8.929931*.*

Proof. Apply Proposition 12.1 to Theorems 6.5 and 8.1. □

13 Strong NP-completeness

The approximation algorithms exploit planar structure, but exact optimization remains hard. We first state the decision and source problems and the graph terminology used by both reductions.

Definition 13.1 (Complexity and reductions)**.** A *polynomial-time many-one reduction* maps instances of one decision problem to another in polynomial time, so an input is a yes-instance if and only if its image is a yes-instance.

A decision problem is in *NP* if every yes-instance has a certificate of polynomial encoding length whose validity is checkable in polynomial time, and no certificate is accepted on a no-instance. It is *NP-hard* if every problem in NP admits such a reduction to it, and *NP-complete* if it is both NP-hard and in NP. A numerical problem is *strongly NP-hard* if hardness persists when its integer data are bounded by a fixed polynomial in the combinatorial instance size; it is *strongly NP-complete* if it is also in NP. For rational data, our restricted instances explicitly bound both numerators and denominators.

Definition 13.2 (3-Partition)**.** The input consists of positive integers $a_1, \ldots, a_m, B$, where $m = 3p$ for a positive integer p, satisfying

$$s = \sum_{i=1}^{m} a_i = pB, \qquad B/4 < a_i < B/2.$$

The question is whether $[m]$ can be partitioned into p groups $I_1, \ldots, I_p$ with $|I_b| = 3$ and $\sum_{i \in I_b} a_i = B$ for every b.

Symbol	Meaning
$m = 3p,\ [m]$	Number of source items and their index set.
$a_i,\ B$	Item size i and target sum of each bin.
$s = pB = \sum_i a_i$	Total item size and total capacity of the p bins.
I_b	Index set of the three items assigned to source bin b.

Both reductions use the strong NP-completeness of this problem [6]. We use the NP-hard restriction in which B is polynomially bounded in m. The case $p = 1$ is a trivial yes-instance, so excluding it preserves NP-hardness. In this restriction each a_i is polynomially bounded, so the explicit repetitions in our constructions remain polynomial in the source size. A partition into p groups of sum B then necessarily has three items per group: two items sum to less than B, while four items sum to more than B.

Definition 13.3 (Paths and matchings)**.** A *path* is an undirected graph with distinct vertices $v_0, \ldots, v_k$ and edges exactly $\{v_{j-1}, v_j\}$ for $1 \le j \le k$. A *linear forest* is a disjoint union of paths, allowing isolated vertices. In this paper an *odd path* has an odd number of vertices, not an odd number of edges. Odd and even positions refer to local positions numbered from zero along an item path, not to global hypothesis indices.

A *matching* is a set of edges with no shared endpoint. A vertex is *unmatched* if it is incident to no chosen edge. A *maximum matching* has largest possible cardinality; "maximum" here always refers to the number of edges, not their weights. A *perfect matching* leaves no vertex unmatched.

The two reductions share a combinatorial mechanism. They place each item on an odd path and supply singleton and adjacent-pair membership tests. A maximum matching leaves one even-position vertex unmatched; its position chooses the item's bin. Moving that position changes a short part of a histogram. The maximum-depth reduction counts branch heights, while the expected-depth reduction counts pair weights. In both cases these local changes add to the source bin loads.

13.1 The geometric realization and the matching choice

For integer abscissae, the line

$$I_{a,b}: \quad 4y - 4(a+b)x + (2a-1)(2b+1) = 0$$

has value $4(x - a + \frac{1}{2})(x - b - \frac{1}{2})$ on (x, x^2). Its negative answer selects exactly the integer abscissae $a, \ldots, b$. This realizes interval membership using one line. The line

$$P_t: \quad 2y - 4tx + 2t^2 - 1 = 0 \tag{41}$$

has value $2(x-t)^2 - 1$, and its negative answer isolates exactly the point of abscissa t. Both formulas are nonzero at all integer abscissae, so every answer is binary. The tangent $y = 2tx - t^2$ exposes

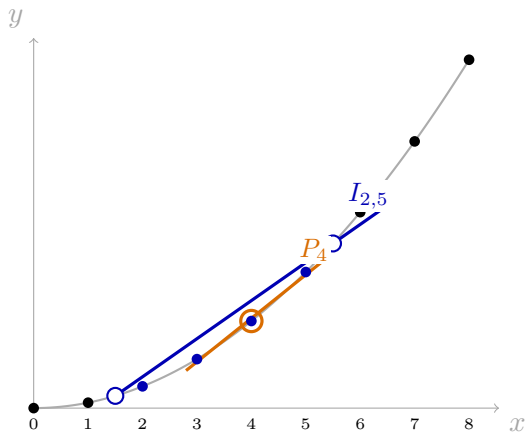


Figure 8: The integer points lie on $y = x^2$. The blue line $I_{2,5}$ has negative answer exactly on the points with $x \in \{2, 3, 4, 5\}$; the orange line P_4 has negative answer only at $x = 4$, marked with an orange ring. The interval endpoints are the half-integer intersections $x = 1.5$ and $x = 5.5$.

the point (t, t^2), since every other parabola point lies strictly above it. The constructed hypotheses are therefore in strictly convex position.

In both reductions, membership sets are the negative answer classes of the displayed lines. Thus a hit in Definition 2.2 is the negative affine sign; the distinguished hypothesis misses these sets. The query itself is unchanged.

These formulas provide the two query types needed by the reduction: an interval query selects a consecutive block, and a private singleton query distinguishes one point from all the others. Putting the points on a parabola gives both query types with integer coefficients while keeping every hypothesis exposed on the convex hull. For example, Figure 8 shows $I_{2,5}$ and P_4 on the points with integer abscissae from 0 to 8.

For maximum depth we also use *clouds* of ordinary points. They make a local identification problem take a chosen number of queries without changing the objective. A cloud's size w_i is its number of points, not a prior weight. Given positive integer cloud sizes w_i, put $Q = 4 \max_i w_i$ and represent cloud i by

$$\{(Qi + j, (Qi + j)^2) : 0 \le j < w_i\}.$$

Its point with $j = 0$ is its *anchor*. Supply the cloud-interval line

$$4y - 4Q(a + b)x + Q^2(2a - 1)(2b + 1) = 0. \tag{42}$$

For $a \ge 1$ its negative answer selects exactly the clouds $a, \dots, b$, while excluding cloud 0. This follows by factoring its value as $4(x - Q(a - \frac{1}{2}))(x - Q(b + \frac{1}{2}))$. Each cloud lies within $Q/4$ of its anchor, so neither boundary, halfway between consecutive anchors, contains a hypothesis. We supply private lines (41) for each nonanchor point, but not for the anchors. An anchor and a point in its own cloud agree on every cloud-interval test and on every other private test. Their separation requires that point's private query. Thus a cloud of size w forces $w - 1$ queries on the anchor's local path, while the anchor itself remains available to represent the cloud in the matching construction.

Clouds are ordered so that consecutive vertices of each path used below have consecutive cloud indices. The only cloud-interval tests are whole-cloud singleton tests and adjacent-pair tests specified by those paths. Every point, including every anchor, must be identified.

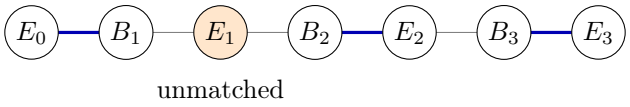


Figure 9: A maximum matching of $E_0, B_1, E_1, B_2, E_2, B_3, E_3$ with E_1 left unmatched. Blue edges are the selected adjacent pairs. In the reduction, the unmatched even-position cloud selects the item's bin; the uniquely forced pairs lie to its left and right.

Lemma 13.4 (Identifying one or two clouds)**.** *For this supplied family, the minimum height on one cloud of size w is $w - 1$. On two clouds of sizes u, v, when their whole-cloud singleton tests are supplied, the minimum height is* $\max\{u, v\}$.

Proof. On a single cloud, old cloud-interval tests are constant and each private test isolates one nonanchor point. The anchor answers the same way as every nonanchor on all the other private tests, so its path must ask all $w - 1$ of them. Querying them sequentially achieves that height.

On two clouds, the anchor of the larger cloud must use all its private tests: each is the only query separating that cloud's anchor from its corresponding nonanchor. It must also use at least one cloud-interval test, since private tests give the same answer at both anchors. The larger cloud therefore forces depth at least $\max\{u, v\}$. First separating the clouds and then performing their private searches attains the bound. □

Lemma 13.5 (One unmatched position)**.** *A maximum matching of the path*

$$E_0, B_1, E_1, \ldots, B_t, E_t$$

leaves exactly one vertex E_k unmatched. It then uses pairs $\{E_{j-1}, B_j\}$ for $j \le k$ and $\{B_j, E_j\}$ for $j > k$.

Proof. The maximum size is t, so exactly one vertex is unmatched. Both sides of that vertex must have even length, forcing it to be an E_k. Each remaining even path has a unique perfect matching: its endpoint must match its only neighbor, and induction fixes the rest. This gives the displayed pairs. □

Figure 9 shows the smallest nontrivial example of the matching pattern. Moving the unmatched even-position cloud changes which segment represents the item's bin choice.

The item paths only organize the supplied membership queries. The reduction uses singleton and adjacent-pair tests directly; no underlying edge-query search tree is part of the instance.

13.2 Maximum depth: bin loads become branch-height counts

Theorem 13.6 (Strong NP-completeness of maximum depth)**.** *Deciding whether* $\mathrm{OPT}_W(\mathcal{H}) \le K$ *is strongly NP-complete, even for integer-coordinate points on a parabola, polynomially bounded integer line coefficients, and $O(n)$ supplied binary line queries.*

Proof strategy.

1. Force the distinguished anchor's path to attain its minimum length. Its first-hit pairs must then be a maximum matching, with one even-position cloud unmatched in each item path.
2. Use local cloud heights and a sorted exit histogram to exclude internal unmatched positions. Each remaining boundary is one bin choice.
3. Moving that boundary transfers the item's size between disjoint histogram windows. The height threshold requires every bin load to be at most B, hence exactly B.

13.2.1 Construction and supporting propositions

Use the restricted 3-Partition instance of Definition 13.2. Let $q = (p-1)(4s+2m)$ be the total matching size and $T = s - B$ the width of a bin window. Choose a cloud offset M, bin levels $\lambda_1, \ldots, \lambda_p$, and high cloud sizes H, C by the calibration in Section 13.2.3; the decision threshold is $K = q + M$. The source information is a_i, B, p; these additional quantities only separate branch-height levels and make the distinguished path tight.

For item i, construct an odd path whose vertices represent clouds. Starting with an even-position cloud of size 2, append $p-1$ segments, indexed by $b = 1, \ldots, p-1$. Each row below appends an odd-position cloud and then the next even-position cloud. Segment b consists of one entry row, a_i repetitions of the four block rows, and one exit row:

part	odd-position size	next even size
entry	C	M
block step 1	λ_b	H
block step 2	$\lambda_b + T$	M
block step 3	$\lambda_{b+1} + T$	H
block step 4	λ_{b+1}	M
exit	C	2

The repeated block is the only part depending on a_i. It transfers a_i units between neighboring histogram windows. There are $4a_i + 2$ odd-position vertices per segment, so the whole item path has $2s_i + 1$ vertices, where $s_i = (p-1)(4a_i + 2)$, and its maximum matching has size s_i. Its p even-position clouds of size 2 are the segment boundaries.

Add one auxiliary cloud of size M and cloud 0 of size 2. Supply each nonzero cloud's whole-cloud singleton test, each adjacent-pair test within an item path, and each nonanchor point's private singleton test. Realize them by (42) and (41).

For boundary choices, let L_b be the total item size assigned to bin b. Put

$$Q_{\mathrm{hi}} = 2(p-1)(s+m), \qquad \beta_1 = q - s, \qquad \beta_b = Q_{\mathrm{hi}} + 2s(p-b) \quad (2 \le b \le p).$$

Here Q_{hi} counts the always-high pairs, and β_b is the reference tail count in bin window b after its bin load is removed. These counts are combinatorial; their levels are calibrated later.

Proposition 13.7 (Tight-spine matching normalization)**.** *Every identifying tree has height at least K. A tree of height at most K can be normalized without increasing height so its first-hit pairs form a maximum matching in every item path, all clouds are intact at their first hit, and the distinguished cloud's private query is last. If $N(t)$ counts exits of local height at least t, sorting those exits by nonincreasing height gives the exact criterion*

$$t + N(t) \le K \tag{43}$$

at every exit-height level.

Proposition 13.8 (Only boundary positions are feasible)**.** *Every matching whose normalized tree fits height K leaves a boundary cloud unmatched in each item path. Its p possible boundaries choose the item's bin.*

Proposition 13.9 (Histogram windows encode bin loads)**.** *At the upper level $t_b = \lambda_b + T$ of each bin window,*

$$N(t_b) = \beta_b + L_b, \qquad t_b + \beta_b = K - B.$$

Consequently the height criterion requires $L_b \le B$ for every bin.

Proposition 13.10 (Balanced loads give a complete tree)**.** *Boundary matchings with $L_b = B$ satisfy every height-profile constraint and yield a complete identifying tree of height K.*

Proposition 13.11 (Polynomial geometric encoding)**.** *The construction has polynomial size on the restricted source input, integer coordinates and coefficients bounded by $O(n^4)$, and $O(n)$ supplied strict binary lines. A height-K identifying tree is a polynomially checkable NP certificate.*

Proof of Theorem 13.6. A source partition yields a height-K tree by Proposition 13.10. Conversely, normalize any height-K tree by Proposition 13.7; Propositions 13.8 and 13.9 give bin loads at most B. Their sum is $s = pB$, so all equal B; the source size bounds force three items per bin. Proposition 13.11 supplies strong polynomial encoding and NP membership. Thus the reduction is exact for arbitrary adaptive trees. □

13.2.2 Technical proofs of the height propositions

Proof of Proposition 13.7.

Claim 13.1 (The distinguished anchor forces maximum matchings)**.** *Every identifying tree has height at least K. If its height is at most K, the first-covered pairs form a maximum matching in every item path, and no nonzero-cloud private test is asked on the anchor path.*

Proof. Follow the anchor of cloud 0 through any complete tree. It needs the private test for the other point of cloud 0. Private tests of other clouds do not distinguish their anchors from this anchor. It therefore also needs cloud queries covering the $2q + m$ item-path anchors and the auxiliary anchor.

Classify each anchor by its first hitting cloud query along this all-miss path. A query first covers one anchor or two adjacent anchors. The first-covered pairs are disjoint, hence form a matching of size at most $\sum_i s_i = q$. There are $2q + m + 1$ nonzero anchors to cover. If g first-covered groups are pairs, the remaining groups are singletons, so the path uses $(2q + m + 1) - g$ cloud queries. Since $g \le q$, this is at least $q + m + 1$. It must also ask the private query separating the two points of cloud 0, giving the lower bound $q + m + 2 = K$.

If a tree has height at most K, equality holds. No private test of a nonzero cloud can occur on this path, and its pairs form a maximum matching in every item path. Lemma 13.5 therefore leaves exactly one even-position cloud unmatched per item. □

At this equality, all clouds are intact when first encountered: none of their private points was removed earlier. A hit exit contains one full cloud or two full clouds. Replace any query first covering just one cloud by its supplied whole-cloud singleton test. Put cloud 0's private query last; this advances the other exits and leaves its two points within depth K. Attach the optimal local codes of Lemma 13.4.

Claim 13.2 (Sorting exit heights gives the complete depth test)**.** *After the cloud-query groups are sorted by nonincreasing local height, every target path has depth at most K exactly when $t+N(t) \le K$ at every exit-height level t.*

Proof. The cloud queries now act on disjoint singletons and pairs. Their exit heights are $w-1$ for a singleton and $\max\{u, v\}$ for a pair. Include the final private query as an exit of height zero. Sorting these heights in nonincreasing order cannot hurt: exchanging consecutive heights $r < s$ replaces $\max\{j + r, j + 1 + s\}$ by $\max\{j + s, j + 1 + r\}$.

Write h_j for the local height of the exit at spine position j. The *tail histogram* of exit heights is $N(t) = |\{j : h_j \ge t\}|$, the number of exits of height at least t. In sorted order, the last such exit occurs at position $N(t)$. An exit at position j with local height h_j has total depth $j + h_j$.

Because the heights are sorted, the maximum of these sums is exactly the maximum of $t + N(t)$ over exit-height levels t: an exit of height t occurs no later than position $N(t)$, and the last exit counted by $N(t)$ has height at least t. Thus all target paths fit within K exactly when (43) holds at every exit-height level t. This criterion is simply the maximum of "spine position plus remaining local depth." □

□

Proof of Proposition 13.8. Every matched pair has exit height at least M, because each odd cloud has size at least $\lambda_1 = M$. The auxiliary singleton has exit height $M - 1$. An unmatched internal even vertex has size M or H, and hence exit height at least $M - 1$. At $t = M - 1$, these give $N(t) \geq q + 1 +$ number of such internal unmatched vertices. Since $K = q + M$, (43) excludes all of them. The unmatched vertex in each item path therefore chooses one of its p boundary positions, interpreted as a bin. □

Proof of Proposition 13.9.

Claim 13.3 (A segment transfers load only between its neighboring windows)**.** *Its two boundary matchings differ by the window transfer of item i.*

Proof. To see the transfer, compare the two matchings on segment b. When its unmatched vertex is before the segment, a repeated block has heights

$$H,\ \lambda_b + T,\ H,\ \lambda_{b+1}.$$

When that vertex is after the segment, the same block has heights

$$\lambda_b,\ H,\ \lambda_{b+1} + T,\ H.$$

The two connector pairs have height C in either case. With a_i repetitions, the change in $N(t)$ is exactly

$$-a_i \mathbf{1}_{(\lambda_b, \lambda_b + T]}(t) + a_i \mathbf{1}_{(\lambda_{b+1}, \lambda_{b+1} + T]}(t).$$

The windows are disjoint because $T < s$ and consecutive λ_b differ by at least s. Moving across several segments cancels the intermediate transfers: an item assigned to bin b removes a_i from window 1 and adds a_i to window b. Figure 10 illustrates this change relative to the reference matching. □

Claim 13.4 (Reference tail counts turn transfers into bin loads)**.** *At $t_b = \lambda_b + T$, the resulting count is $N(t_b) = \beta_b + L_b$.*

Proof. In the reference matching assigning all items to bin 1, Q_{hi} pairs have height H or C. For each segment b, the other pairs include s of height $\lambda_b + T$ and s of height λ_{b+1}. At thresholds in the first window, the reference tail count is $q = \beta_1 + s$. For a later window b, the reference tail count is $Q_{\mathrm{hi}} + 2s(p - b) = \beta_b$: the Q_{hi} high pairs always count, and each later segment contributes $2s$ lower pairs. At the upper window level $t_b = \lambda_b + T$, the transfer formula therefore gives

$$N(t_b) = \beta_b + L_b, \qquad t_b + \beta_b = K - B.$$

The profile condition at this level is $t_b + N(t_b) \leq K$, or equivalently $L_b \leq B$. Thus (43) requires $L_b \leq B$ for every bin. Since $\sum_b L_b = s = pB$, all these inequalities are equalities and yield a source 3-partition. □

□

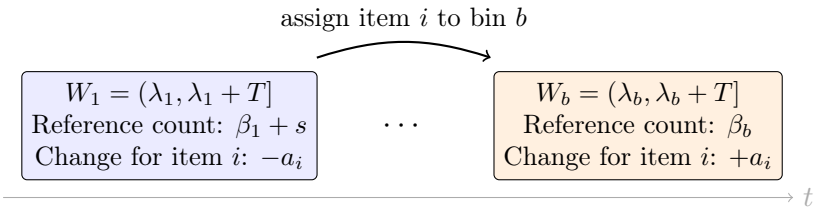


Figure 10: Relative to the reference matching, assigning one item of size a_i from bin 1 to bin b removes a_i from the first histogram window and adds a_i to window b; intermediate windows are unchanged. Superposing the item transfers gives $N(t) = \beta_j + L_j$ throughout window $W_j = (\lambda_j, \lambda_j + T]$.

Proof of Proposition 13.10. Choose the boundary matchings of a source partition, so $L_b = B$. The window upper levels satisfy (43) with equality. For completeness, the remaining levels have the following counts:

t	$N(t)$	$t + N(t)$
0	K	K
1	$K - 1$	K
$M - 1$	$q + 1$	K
λ_b	$\beta_b + s$	K
H	Q_{hi}	K
C	$2m(p-1)$	$K - 2s(p-1) + 1 \le K$

These are all possible exit heights, together with the already checked $\lambda_b + T$. The count $N(t)$ is constant between successive exit heights, so $t + N(t)$ increases there and its maximum occurs at a listed exit height. Sort the corresponding queries, attach their exact one- or two-cloud codes, and finish with cloud 0's private query. This constructs a complete tree of height K. □

Proof of Proposition 13.11. There are $2q + m + 2$ clouds, each of size at most $C = O(ps + m)$. Their total number of hypotheses is polynomial in the restricted source input. The realization has integer coordinates and line coefficients bounded by $O(n^4)$, with $O(n)$ supplied lines. Every proper identifying tree has $n-1$ internal nodes; its answers and height can be checked in polynomial time. This proves membership in NP and strong NP-hardness with polynomially bounded numerical data. □

13.2.3 Calibration of the height construction

Parameter calibration.

Choose

$$M = m + 2, \qquad \lambda_1 = M,$$
$$\lambda_b = M + (2b-3)s \quad (2 \le b \le p), \qquad H = M + 2s(p-1), \qquad C = H + 1.$$

The width $T = s - B < s$ makes the windows disjoint. All block levels are below H, and the connectors have height C. These choices give $t_b + \beta_b = K - B$ and make the minimum distinguished-path length $q + m + 2$ exactly $K = q + M$.

13.3 Expected depth: an exact matching formula

The expected-depth reduction needs no clouds. Each nonzero hypothesis is one path vertex or one isolated vertex, and every query isolates either a singleton or an adjacent pair. The combinatorial

choice is still a matching, but its objective depends on how the resulting pairs are ordered.

For a finite positive weight vector z, keeping repeated values as distinct entries, define the *weighted completion sum* by

$$\mathcal{C}(z) = \sum_{j=1}^{|z|} j z_{(j)}, \qquad z_{(1)} \geq \cdots \geq z_{(|z|)}.$$

It equals the sum of the individual weights plus the smaller weight for each unordered pair of entries: in decreasing order, an earlier entry delays a later entry by one. Thus concatenating vectors u, v adds the cross contribution $\sum_{i,j} \min\{u_i, v_j\}$. Equivalently, if $h_z(t) = |\{j : z_j \geq t\}|$, then

$$\mathcal{C}(z) = \int_0^\infty \frac{h_z(t)(h_z(t)+1)}{2}\, dt. \tag{44}$$

At height t, the first $h_z(t)$ ranks contribute $1 + \cdots + h_z(t)$; integrating those contributions proves the identity.

Lemma 13.12 (Exact singleton/pair formula)**.** *Consider a* linear forest*, meaning a disjoint union of paths (isolated vertices are allowed), with vertices $1, \ldots, N$ ordered component by component. Represent them and a distinguished hypothesis by (i, i^2), $0 \leq i \leq N$. Supply $I_{i,i}$ for each nonzero vertex and $I_{i,i+1}$ for each forest edge. Give the distinguished hypothesis weight $\omega_0 > 0$ and the others positive weights ω_v. For a matching $\mathcal{M}$, let $J(\mathcal{M})$ be the vector containing mass $\omega_u + \omega_v$ for each matched pair and ω_v for each unmatched vertex, retaining multiplicities when masses coincide. The optimal unnormalized expected depth is*

$$\min_{\mathcal{M}} \left\{ \omega_0(N - |\mathcal{M}|) + \mathcal{C}(J(\mathcal{M})) + \sum_{\{u,v\} \in \mathcal{M}} (\omega_u + \omega_v) \right\}. \tag{45}$$

If $\omega_0 > N \sum_{v>0} \omega_v$, an optimal matching in this formula has maximum cardinality.

Proof strategy.
Read any tree from the distinguished all-miss spine: its disjoint first-hit pairs form a matching. Then sort the group masses; adjacent exchanges minimize first-cover cost. A sufficiently heavy distinguished point forces maximum cardinality before any other cost matters.

Proof.

Claim 13.5 (First-hit groups are an attainable matching invoice)**.** *The spine contributes one query per singleton or matched pair, and only a pair requires an extra local query.*

Proof. Follow hypothesis 0, which misses every supplied membership query. The hypotheses first covered together are disjoint singletons or adjacent pairs. The pairs form a matching $\mathcal{M}$, and the spine has $N - |\mathcal{M}|$ queries. A pair's hit branch requires exactly one additional query; a singleton branch requires none. This gives the first and third terms of (45).

For the first-cover term, suppose masses $a < b$ occupy consecutive positions $j, j+1$. Their contribution is $ja + (j+1)b$ in that order and $jb + (j+1)a$ after swapping, a decrease of $b - a$. Repeated exchanges show that nonincreasing mass is optimal. Every such ordering is executable: query the selected pairs and the singleton tests for the unmatched vertices in that order. The groups are disjoint, so each query's hit branch exits only to its own group. This proves the formula both as a lower bound on arbitrary trees and as an attainable value for each matching. □

Claim 13.6 (The distinguished weight forces maximum cardinality)**.** *Under the stated dominance condition, minimizing its depth is the first objective.*

Proof. Every proper complete tree has depth at most N, so the total weighted depth of all nonzero hypotheses is at most $N \sum_{v>0} \omega_v$. One additional query on hypothesis 0's path outweighs this entire budget under the stated inequality. An optimal tree must consequently minimize that path's length, which means choosing a maximum-cardinality matching. □

□

13.4 Expected depth: bin loads become a quadratic penalty

Theorem 13.13 (Strong NP-completeness of expected depth)**.** *Given supplied binary lines, strictly positive integer hypothesis weights bounded by a fixed polynomial in n, and a rational threshold, deciding the optimal expected depth is strongly NP-complete. This holds for $\mathcal{H} = \{(i, i^2) : 0 \le i < n\}$, $O(n)$ integer-coefficient lines, and negative answer classes of size at most two.*

Proof strategy.

1. Use the exact matching invoice and a heavy distinguished point to normalize the optimum to one unmatched even vertex per item.
2. Pair-mass histograms encode bin loads. Integrating their tail counts produces a quadratic load penalty and an unwanted linear term.
3. Tiny, ordered boundary-weight adjustments cancel that linear term. Medium-weight isolated vertices exclude internal choices, and scale separation makes rounding smaller than a unit imbalance.

13.4.1 Construction and supporting propositions

Let $q = 2s(p-1)$ be the total matching size and define the reference bin counts

$$\beta_1 = q - s, \qquad \beta_b = 2s(p-b) \quad (2 \le b \le p).$$

Take an isolated-vertex count r, an even window width V, and bin levels λ_b from Section 13.4.3. The r isolated vertices have weight $Z = V/2$. To cancel the bin linear term, define

$$t_i = q + r + m - i, \quad \delta_{ib} = \left\lfloor \frac{V(\beta_1 - \beta_b)a_i}{t_i} \right\rfloor,$$
$$D = \max_{i,b} \delta_{ib}, \qquad A_i = 1 + (2D+1)i.$$

Here t_i will be the exact coefficient of unmatched item i; A_i fixes the unmatched-weight order, and δ_{ib} is its small bin-dependent correction. Since $\beta_1 - \beta_b = (2b-3)s > 0$ for $b \ge 2$, all corrections are nonnegative and $\delta_{i1} = 0$.

For item i, form an odd path

$$E_{i0},\, B_{i1},\, E_{i1},\, \ldots,\, B_{is_i},\, E_{is_i}, \qquad s_i = 2a_i(p-1),$$

where each displayed symbol also denotes that vertex's weight. First define unperturbed even weights

$$E^0_{ij} = \begin{cases} A_i, & j \equiv 0 \pmod{2a_i}, \\ A_i + V, & j \text{ odd}, \\ A_i + 2V, & \text{otherwise}. \end{cases}$$

Each consecutive difference is $+V$ or $-V$. Between two boundary positions there are $2a_i$ steps, of which a_i go up and a_i go down. For $1 \le j \le s_i$ let $b = \lceil j/(2a_i) \rceil$ and put

$$B_{ij} = \begin{cases} \lambda_b - E^0_{i,j-1}, & E^0_{ij} - E^0_{i,j-1} = V, \\ \lambda_{b+1} - E^0_{ij}, & E^0_{ij} - E^0_{i,j-1} = -V. \end{cases}$$

Finally set $E_{i,2a_i(b-1)} = A_i + \delta_{ib}$ at each of the p boundaries and leave all other even weights unperturbed. The odd weights do not change when this adjustment is made.

Add r isolated forest vertices of weight Z. The nonzero forest has $N = 2q+m+r$ vertices, with maximum matching size q. Let P be the total weight of its item-path vertices, $W = P + rZ$, and give hypothesis 0 weight $\omega_0 = (N+1)W + 1$. Use precisely the lines of Lemma 13.12. Normalize the weights only after the construction.

Use $E_+ = q\sum_{i,b} \delta_{ib}$ for the possible pair-mass perturbation and $E_- = \sum_i (t_i - 1)$ for the rounding loss. Write G for the minimum raw weighted cost for a fixed maximum matching in Lemma 13.12, $U = (U_1, \dots, U_m)$ for its unmatched item weights, and z for its q pair masses. Boundary choice b_i assigns item i to that bin, whose load is $L_b = \sum_{i:b_i=b} a_i$.

Proposition 13.14 (Separated scales and maximum-matching normalization)**.** *All weights are positive. Every pair mass exceeds Z and every unmatched item weight; boundary weights are below Z and ordered by item index. The optimum uses a maximum-cardinality matching, and the calibration ensures $E_+ + E_- < V$.*

Proposition 13.15 (The matching-dependent invoice)**.**

$$G = G_{\text{fix}} + \mathcal{C}(z) + \mathcal{C}(U) + (q-1)\sum_i U_i + r\sum_i \min\{U_i, Z\}, \tag{46}$$

where

$$G_{\text{fix}} = \omega_0(q+m+r) + \frac{Zr(r+1)}{2} + qrZ + P.$$

Proposition 13.16 (Pair windows give a quadratic load identity)**.** *Let b_i be the bin chosen by item i and $L_b = \sum_{i:b_i=b} a_i$. Let z^0 be the unperturbed pair-mass vector for these choices, and let C_{ref} be the completion sum of the reference vector in which every item chooses bin* 1*. Then*

$$\mathcal{C}(z^0) = C_{\text{ref}} + V\left[\sum_b \beta_b L_b - \beta_1 s + \frac{\sum_b L_b^2 - s^2}{2}\right]. \tag{47}$$

Proposition 13.17 (Boundary corrections cancel the linear term)**.** *For every boundary matching, there is an error e with $-E_- \le e \le E_+$ such that*

$$G = G_{\text{fix}} + \sum_i t_i A_i + C_{\text{ref}} + \frac{V}{2}\left(\sum_b L_b^2 - s^2\right) + e. \tag{48}$$

Define the raw threshold

$$k_0 = G_{\text{fix}} + \sum_i t_i A_i + C_{\text{ref}} + \frac{V}{2}(pB^2 - s^2), \qquad k = k_0 + E_+.$$

A boundary matching has $G \le k$ exactly when every bin load is B.

Proposition 13.18 (Internal choices are excluded; encoding is strong)**.** *Every maximum matching with an internal unmatched even vertex has $G > k$. The construction and its normalized threshold k/Ω, where $\Omega = \omega_0 + W$, have polynomially bounded numerical data and $O(n)$ supplied lines; a proper tree is a polynomial NP certificate.*

Proof of Theorem 13.13. A source partition selects boundary choices of raw cost at most k by Proposition 13.17. Conversely, Proposition 13.14 reduces the optimum to maximum matchings. Proposition 13.18 excludes internal unmatched vertices, and Proposition 13.17 forces every bin load to be B. This is a 3-partition. Normalizing divides both objective and threshold by Ω; the encoding proposition gives strong hardness and NP membership, including arbitrary adaptive trees. □

13.4.2 Technical proofs of the expected-depth propositions

Proof of Proposition 13.14. The scale estimates verified in Section 13.4.3 are used here only to compare the three weight classes. Every even weight is positive. By the first bound in (49), $A_i + \delta_{ib} < V/100$, so $A_i + 2V < 2.01V$ and every unmatched even weight is below $3V$. For an upward step, the two possible pair masses are λ_b and $\lambda_b + V$; for a downward step they are $\lambda_{b+1} + V$ and λ_{b+1}. Since $\lambda_b \geq 12V$ and $\lambda_{b+1} \geq 14V$, all these masses are at least $12V$. Boundary adjustments are nonnegative and can only increase a matched pair's mass. The odd weights are also positive: each is either $\lambda_b - E^0_{i,j-1}$ or $\lambda_{b+1} - E^0_{ij}$, and every unperturbed even weight is less than $2.01V$. Hence every pair mass exceeds $Z = V/2$ and every unmatched even weight. Boundary weights are below Z, and they increase strictly with the item index: $A_{i+1} - (A_i + D) = D + 1 > 0$. Finally, $E_+ + E_- < V$ follows from the last two calibration bounds. Lemma 13.12 restricts the optimum to maximum matchings. By Lemma 13.5, each item then leaves one even vertex unmatched. □

Proof of Proposition 13.15. To derive this expression, split the completion sum into the three groups z, U, and the r isolated weights Z. Since every pair mass is larger than the other weights, their cross terms are qrZ and $q\sum_i U_i$. The remaining cross term is $r\sum_i \min\{U_i, Z\}$. Identifying the pairs adds their total weight $P - \sum_i U_i$, which changes the coefficient q to $q - 1$. The distinguished path contributes $\omega_0(q + m + r)$. Everything not depending on the matching is collected in G_{fix}. □

Proof of Proposition 13.16.

Claim 13.7 (Boundary movement transfers pair masses between windows)**.** *The pair-mass tail count in window b is $\beta_b + L_b$, and it is unchanged outside the windows.*

Proof. Unmatching boundary $2a_i(b-1)$ assigns item i to bin b. Temporarily omit the boundary perturbations and denote the resulting pair-mass vector by z^0. For an upward step in segment b, pairing its odd vertex to the even vertex on the left gives mass λ_b, while pairing right gives $\lambda_b + V$. For a downward step, those masses are $\lambda_{b+1} + V$ and λ_{b+1}. Moving the unmatched vertex across a whole segment consequently changes the pair-mass histogram by

$$-a_i \mathbf{1}_{(\lambda_b, \lambda_b + V]} + a_i \mathbf{1}_{(\lambda_{b+1}, \lambda_{b+1} + V]}.$$

There are exactly a_i steps of each type. As in the maximum-depth construction, transfers across consecutive segments cancel except at the starting and chosen bins. In the reference matching, all odd vertices are paired to their right. Segment b contributes s masses $\lambda_b + V$ and s masses λ_{b+1} after summing over items. For thresholds t in the first window, the reference tail count is $q = \beta_1 + s$; for later window b it is $2s(p - b) = \beta_b$. Assigning items according to loads L_b changes the tail count throughout window b to $\beta_b + L_b$: the first window loses $s - L_1$ pair masses from its reference

count q, while each later window gains L_b. Outside these disjoint windows the tail histogram is unchanged. □

Claim 13.8 (Integrating the tail counts yields the quadratic term)**.** *The triangular histogram formula gives* (47)*.*

Proof. Let C_{ref} be the completion sum of the unperturbed pair masses when every first boundary is unmatched. It is explicitly computable by sorting those q masses. Within a window the integrand of (44) is the triangular polynomial $h(h+1)/2$. Substituting $h = \beta_b + L_b$ and subtracting the reference loads $(s, 0, \dots, 0)$ gives the following expression. For a baseline count β_b, the change in the triangular polynomial is $\beta_b L_b + (L_b^2 + L_b)/2$. The reference has load s in the first window, so its change is $\beta_1 s + (s^2 + s)/2$. Subtracting this reference change cancels the linear $L_b/2$ terms because $\sum_b L_b = s$. The remaining expression has a useful quadratic term, but also the unwanted linear term $\sum_b \beta_b L_b$. □

□

Proof of Proposition 13.17.

Claim 13.9 (Pair perturbations change completion cost by at most E_+)**.** *The increase is nonnegative and at most $q \sum_{i,b} \delta_{ib}$.*

Proof. Boundary perturbations increase pair masses by at most the total added boundary weight. For a vector z of q pair masses, $\mathcal{C}(z) = \min_\sigma \sum_{j=1}^{q} j z_{\sigma(j)}$, by the adjacent-swap ordering argument in Lemma 13.12. If coordinates increase by a total amount Δ, this representation shows that $\mathcal{C}$ is nondecreasing and increases by at most $q\Delta$: evaluate the new vector in an ordering optimal for the old one. Here $\Delta \le \sum_{i,b} \delta_{ib}$, so the increase is at most $q \sum_{i,b} \delta_{ib} = E_+$. □

Claim 13.10 (The unmatched ranks cancel the unwanted bin linear term)**.** *The boundary cost is* (48) *with $-E_- \le e \le E_+$.*

Proof. All boundary unmatched weights are below Z, and their order is the fixed decreasing item order. Indeed, $A_{i+1} - (A_i + D) = D + 1 > 0$, so even the largest perturbation of item i leaves it below the unperturbed weight of item $i+1$. Thus their terms in (46) are $\sum_i t_i (A_i + \delta_{ib_i})$. Indeed, item i has rank $m - i + 1$ in $\mathcal{C}(U)$, plus coefficient $q - 1 + r$ from the other terms, giving $t_i = q + r + m - i$. The floor definition gives

$$-E_- \le \sum_i t_i \delta_{ib_i} - V \sum_i (\beta_1 - \beta_{b_i}) a_i \le 0.$$

For each item, the difference is minus the remainder when the integer $V(\beta_1 - \beta_{b_i})a_i$ is divided by t_i; its magnitude is therefore at most $t_i - 1$. Its intended contribution $V \sum_i (\beta_1 - \beta_{b_i}) a_i$ cancels the unwanted linear term in (47). Each floor loses an integer between zero and $t_i - 1$. Combining that loss with the pair-mass increase gives the claimed error interval. □

Claim 13.11 (An unbalanced integral load costs more than the error budget)**.** *The threshold k is met exactly when all loads equal B.*

Proof. Because $\sum_b L_b = pB$,

$$\sum_b L_b^2 - pB^2 = \sum_b (L_b - B)^2.$$

A source partition makes this sum zero and gives weighted depth at most k. Otherwise the integral deviations sum to zero and include at least one positive and one negative entry. Their squares sum to at least two. The weighted depth is then at least $k_0 + V - E_-$. The scale bounds give $V - E_- > 1999V/2000 > V/1000 \geq E_+$, so this depth exceeds $k = k_0 + E_+$. □

□

Proof of Proposition 13.18.

Claim 13.12 (The isolated-vertex penalty excludes every internal choice)**.** *An internal unmatched even vertex makes the raw invoice exceed k.*

Proof. The preceding calculation concerned boundary choices. To complete soundness, a maximum matching leaving an internal even vertex unmatched must also exceed k. Such a vertex has weight $A_i + V$ or $A_i + 2V$, exceeding Z. Every other unmatched weight is at least its A_i. Its interactions with the r isolated vertices contribute rZ instead of at least rA_i. The other matching-dependent terms in (46) are nonnegative, so

$$G \geq G_{\mathrm{fix}} + r \sum_i A_i + r\big(Z - \max_i A_i\big).$$

On the other hand, $pB^2 - s^2 \leq 0$ gives

$$k \leq G_{\mathrm{fix}} + r \sum_i A_i + C_{\mathrm{ref}} + (q+m) \sum_i A_i + E_+.$$

The reference masses are at most $(11+2p)V$, so $C_{\mathrm{ref}} \leq (p+6)(q+1)^2 V$. Using $A_i < V/100$, the last three terms are less than $(p+8)(q+m+1)^2 V$. But $Z - \max_i A_i > V/3$ and $r \geq 1000(p+10)(q+m+1)^2$. The lower bound for an internal unmatched vertex is therefore strictly larger than k. □

Claim 13.13 (The reduction has polynomial numerical data and certificates)**.** *The normalized instance establishes strong hardness rather than only weak numerical hardness.*

Proof. Normalize the positive weights by their total Ω and use the expected-depth threshold k/Ω. The construction has polynomial size on the restricted source input. With $n = N + 1$, we have $V = O(n^2)$, all nonzero vertex weights are $O(n^3)$, and $\omega_0 = O(n^5)$. Coordinates and supplied-line coefficients are $O(n^2)$ and there are $O(n)$ queries. Membership in NP follows by evaluating a proper complete tree with $n - 1$ internal nodes. Both the normalized priors and the threshold have polynomially bounded integer numerators and denominators. This establishes strong NP-completeness with strictly positive polynomially bounded hypothesis weights. □

□

13.4.3 Calibration of the expected-depth construction

The semantic quantities q, β_b, t_i, L_b describe matchings and bin loads. The following slack choices only separate their weight scales; they are not additional source data.

Parameter calibration.
With $a_{\max} = \max_i a_i$, choose

$$r = 1000\big[(p+10)(q+m+1)^2 + (qmp+2m+1)\beta_1 a_{\max} + 1\big],$$
$$V = 2000m(r+q+m+1), \qquad \lambda_b = (10+2b)V \quad (1 \le b \le p).$$

These polynomial integers make V even and give

$$\max_{i,b}(A_i + \delta_{ib}) < V/100, \qquad E_+ \le V/1000, \qquad E_- < V/2000. \tag{49}$$

In particular $E_+ + E_- < V$; a load imbalance cannot be hidden by rounding or pair perturbation.

Indeed, $t_i \ge r$ gives $D \le V\beta_1 a_{\max}/r$. Consequently,

$$E_+ \le qmpD \le V/1000, \qquad \max_{i,b}(A_i + \delta_{ib}) \le (2m+1)D + m + 1 \le V/1000 + V/2000 < V/100.$$

The bound $m+1 \le V/2000$ follows directly from $V = 2000m(r+q+m+1)$. Also $t_i - 1 < r+q+m+1$, giving $E_- < m(r+q+m+1) = V/2000$.

The boundary-weight bound makes $Z - \max_i A_i > V/3$, and the first term of r gives $r/3 > (p+8)(q+m+1)^2$. These are exactly the inequalities used to exclude internal unmatched vertices. Both terms are polynomial in the restricted source data: the first enforces the internal-choice penalty, and the second controls the adjustment error. No optimization or irrational comparison is required by the reduction.

14 Uniform expected-depth hardness

Integer weights can be replaced by uniformly weighted clouds, but private queries might appear anywhere in an adaptive tree. The central step below pushes those queries away from the distinguished spine. It gives an exact identity: the cloud optimum is the weighted optimum plus one computable internal-cloud cost. Raw uniform cost means the sum of leaf depths, before division by the number of hypotheses.

14.1 Exact cloud replacement: normalization before accounting

14.1.1 The construction

Consider the weighted singleton/pair linear forest instance of Lemma 13.12, with vertices $0, 1, \dots, N$ and positive integer weights $w_0, \dots, w_N$. Vertex 0 is distinguished. Order the nonzero vertices along each path and component by component, so every edge joins consecutive indices. Choose the cloud spacing Q by the final calibration below and define

$$n = \sum_{i=0}^{N} w_i, \qquad C_i = \{(Qi + j, (Qi+j)^2) : 0 \le j < w_i\}.$$

The point with $j = 0$ is the anchor of cloud C_i. For every nonzero vertex supply its whole-cloud singleton query; for every linear forest edge supply its whole-cloud pair query. For consecutive cloud indices $a, \dots, b$, these are the negative answers of

$$4y - 4Q(a+b)x + Q^2(2a-1)(2b+1) = 0. \tag{50}$$

The value on the parabola factors as $4(x - Q(a - \frac{1}{2}))(x - Q(b + \frac{1}{2}))$. Every cloud has width less than $Q/4$, so the negative answer selects exactly those clouds and excludes cloud 0.

For every nonanchor point of abscissa t, including those of cloud 0, supply the private line

$$2y - 4tx + 2t^2 - 1 = 0.$$

Its value on (x, x^2) is $2(x - t)^2 - 1$; its negative answer isolates precisely that point. No private line is supplied for an anchor. All answers are strict and binary. The whole-cloud singleton queries separate the anchors, and the private lines distinguish every nonanchor, so the family identifies every hypothesis. All hypotheses have uniform prior $1/n$.

Put

$$D(w) = \frac{(w-1)(w+2)}{2}, \qquad \Delta = \sum_{i=0}^{N} D(w_i). \tag{51}$$

Algorithm 21 gives the construction and the threshold conversion.

Algorithm 21: Uniform cloud replacement of a weighted linear forest instance

Input: Vertices $0, \dots, N$ with each path in order, component by component, positive integer weights w_i, and a raw weighted threshold k

Output: Supplied-line instance with uniform prior and threshold $(k + \Delta)/n$

```
Q ← 4 max_i w_i; n ← Σ_i w_i; H ← ∅; L ← ∅
for i = 0, ..., N do
    for j = 0, ..., w_i − 1 do
        t ← Qi + j; add (t, t^2) to H
        if j > 0 then
            Add 2y − 4tx + 2t^2 − 1 = 0 to L
For each i > 0, add (50) with (a, b) = (i, i) to L
For each linear forest edge {i, i + 1}, add (50) with (a, b) = (i, i + 1) to L
Δ ← Σ_{i=0}^{N} (w_i − 1)(w_i + 2)/2
return (H, L), prior 1/n, and threshold (k + Δ)/n
```

Theorem 14.1 (Exact uniform cloud transfer)**.** *Let* $\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}}$ *be the minimum raw weighted cost of the singleton/pair linear forest instance. For its uniform cloud replacement,*

$$n\,\mathrm{OPT}_{\mathrm{E}}^{\mathrm{cloud}} = \mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} + \Delta. \tag{52}$$

Equivalently, if $J(\mathcal{M})$ *has the pair and singleton masses from Lemma 13.12, the raw uniform optimum is*

$$\Delta + \min_{\mathcal{M}} \left\{ w_0(N - |\mathcal{M}|) + \mathcal{C}(J(\mathcal{M})) + \sum_{\{u,v\} \in \mathcal{M}} (w_u + w_v) \right\}.$$

Proof strategy.

1. Rotate a private singleton past the next query. Equal point weights make its one-unit depth increase no larger than the savings on the opposite, nonempty answer branch.
2. Push all external private queries off the distinguished anchor's spine and move its own private chain to the end. Every first-hit cloud is then intact, so the first-hit pairs form a matching.
3. Sum exact one- and two-cloud local costs, then sort group sizes. The result is the weighted matching invoice plus the fixed offset Δ; the same construction attains it.

14.1.2 Three supporting statements

Lemma 14.2 (Uniform singleton rotation)**.** *Suppose a proper identifying tree on a uniformly weighted state S first asks a supplied query isolating $x \in S$, and the other child first asks a query r that is proper on $S \setminus \{x\}$. These two queries can be exchanged, placing the singleton query in the answer branch of r containing x, without increasing the raw cost.*

Lemma 14.3 (Local uniform cloud costs)**.** *The optimal raw local cost on one intact cloud of size w is $D(w)$. On two intact clouds of sizes u, v, it is*

$$D(u) + D(v) + u + v.$$

Proposition 14.4 (A normalized matching spine)**.** *An optimal uniform cloud tree can be chosen so that its distinguished anchor's path first asks only whole-cloud queries, with every newly hit cloud intact, and then asks only cloud-0 private queries. The first-hit pairs form a matching $\mathcal{M}$. For this matching, the best raw cost is*

$$\Delta + w_0(N - |\mathcal{M}|) + \mathcal{C}(J(\mathcal{M})) + \sum_{\{u,v\} \in \mathcal{M}} (w_u + w_v).$$

Every matching has a tree attaining this invoice.

Proof of Theorem 14.1. Lemma 14.2 permits the normalization in Proposition 14.4 without increasing cost. Lemma 14.3 supplies its exact local offset. Minimize the resulting invoice over matchings and apply Lemma 13.12. This gives $\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{cloud}} = \mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} + \Delta$. Dividing by n proves the displayed expected-depth identity. □

14.1.3 Technical proofs of the supporting statements

Proof of Lemma 14.2. Let A, B be the two nonempty answer classes of r on $S \setminus \{x\}$, and choose their names so that x has the answer of A. In the original tree, x has relative depth one and every hypothesis in $A \cup B$ passes through both queries. Ask r first, put the private singleton query in its $A \cup \{x\}$ branch, and attach the original subtrees on A and B. The depth of x increases by one, the depth of every point of B decreases by one, and all other depths are unchanged. The change in raw cost is $1 - |B| \leq 0$. The resulting queries are proper and are members of the original supplied family. □

If the two queries isolate different singleton points, the same rotation has zero cost. The lemma concerns equal weights; it does not assert the corresponding inequality for unequal priors.

Proof of Lemma 14.3.

Claim 14.1 (A single cloud forces a private chain)**.** *Its raw local cost is $D(w)$, including zero cost when $w = 1$.*

Proof. Within one cloud, every whole-cloud query is constant, and every nonconstant query isolates one of its nonanchors. The only possible proper code is a chain, whose sum of depths is $1 + 2 + \cdots + (w-1) + (w-1) = D(w)$; for $w = 1$ the sum is zero.

□

Claim 14.2 (A whole-cloud separator is optimal on two clouds)**.** *Private-first choices cannot beat $D(u) + D(v) + u + v$.*

Proof. On two clouds a nonconstant whole-cloud query separates the clouds, attaining the displayed formula. Every other nonconstant query isolates one nonanchor. Induct on $u+v$. If a private point of the u cloud is queried first, the remaining two-cloud state has sizes $u-1, v$ and still has its anchors and whole-cloud separator. Using induction, this choice costs

$$u + v + D(u-1) + D(v) + (u-1) + v.$$

Subtracting $D(u) + D(v) + u + v$ gives $v - 1 \geq 0$, because $D(u) - D(u-1) = u$. The analogous difference for a private point of the v cloud is $u - 1 \geq 0$. The base case $u = v = 1$ has raw cost two. When one size is one it has no private point, and the same induction applies to the other cloud. Queries selecting other clouds are constant or separate these two whole clouds, so the argument covers the entire supplied family. □

□

Proof of Proposition 14.4.

Claim 14.3 (External private queries can be pushed off the spine)**.** *No nonzero cloud needs to lose a private point before its first whole-cloud hit.*

Proof. Start with a proper optimal uniform tree and follow the anchor of cloud 0. Suppose a private query for a nonanchor x of a nonzero cloud occurs on this path. Its own cloud anchor is still on the path: x agrees with its anchor on every whole-cloud query, and private queries never remove an anchor. Repeatedly apply Lemma 14.2 to move this private query past the next query on the anchor-0 path. The next query is proper on the remaining state; constant queries have already been removed. Private queries commute at zero cost. Eventually a whole-cloud query separates x's anchor from the anchor of cloud 0: otherwise the tree would not identify both anchors. The point x has the answer of its own anchor to that query. After the rotation its private query is in the hit branch, off the anchor-0 path.

Each such operation removes one external private query from that path, introduces none, and moves it through only finitely many existing path queries. Repetition therefore terminates. □

Claim 14.4 (The distinguished private chain can be moved to the end)**.** *All whole-cloud queries can precede cloud* 0*'s private queries.*

Proof. Next commute each cloud-0 private query past all remaining whole-cloud queries. Its point always has the answer of the anchor of cloud 0, so it stays on that path but moves after the last whole-cloud query. This second process also terminates, by decreasing the number of private/whole-cloud inversions. Every rotation is cost nonincreasing.

□

Claim 14.5 (The normalized groups give an attainable matching invoice)**.** *The minimum normalized raw cost is the expression in Theorem 14.1.*

Proof. Consequently all newly hit clouds are intact at their first whole-cloud query: no private query for any of their points has occurred on the path earlier. First-hit groups are whole singleton clouds or whole adjacent pairs, and the pairs form a matching $\mathcal{M}$. A query first hitting just one cloud can be replaced by its supplied whole-cloud singleton query, which has the same split on the current state. There are $N - |\mathcal{M}|$ whole-cloud spine queries. Cloud 0 contributes $w_0(N - |\mathcal{M}|)$ to the spine cost and $D(w_0)$ afterwards. Every other cloud contributes $D(w_i)$ by Lemma 14.3; every pair also contributes $w_u + w_v$. The first-hit positions charge their cloud masses. Sorting these disjoint group

masses nonincreasingly minimizes that contribution, giving $\mathcal{C}(J(\mathcal{M}))$. This is the asserted lower bound.

Conversely, for any matching, query its disjoint pairs and unmatched singleton clouds in decreasing total cloud size. Attach the optimal local codes of Lemma 14.3 and identify cloud 0 by its private chain at the end. This attains the matching invoice plus Δ. Taking the minimum proves equality. □

□

14.2 Uniform-prior strong NP-completeness

Theorem 14.5 (Strong NP-completeness under a uniform prior)**.** *Deciding whether the optimal expected depth is at most a rational threshold is strongly NP-complete under a uniform prior, even for integer-coordinate points on a parabola, polynomially bounded integer line coefficients, and $O(n)$ supplied strict binary lines.*

Proof strategy.

1. Apply the exact cloud replacement to the hard weighted forest instances of Theorem 13.13.
2. Their polynomial integer weights allow explicit replication in polynomial time. Separated parabola clouds give strict binary queries with polynomial coordinates and coefficients.
3. Convert the raw threshold to $(k+\Delta)/n$ by the transfer identity. A proper tree provides a polynomial certificate.

14.2.1 Supporting size and encoding propositions

Proposition 14.6 (Polynomial replication)**.** *For a hard weighted instance with $n_0 = N+1$ hypotheses, the explicit cloud replacement has $n = O(n_0^5)$ points and is constructible in polynomial time on the restricted 3-Partition source instances.*

Proposition 14.7 (Strong geometric encoding)**.** *The replacement has $O(n)$ supplied strict binary lines. Its integer abscissae are $O(n^2)$; its ordinates and line coefficients are $O(n^4)$. The points remain in strictly convex position on a parabola.*

Proposition 14.8 (Threshold conversion and NP certificates)**.** *For the hard weighted instances and thresholds of Theorem 13.13, the rational threshold $(k+\Delta)/n$ has polynomially bounded numerator and denominator. Its yes/no answer agrees with that of the weighted raw threshold k for arbitrary adaptive trees, and the uniform decision problem belongs to NP.*

Proof of Theorem 14.5. Apply Algorithm 21 to the weighted hard instances. Proposition 14.8 and Theorem 14.1 preserve the yes/no answer. Propositions 14.6 and 14.7 give a polynomial reduction with polynomial numerical data. The certificate proposition supplies NP membership, proving strong NP-completeness. □

14.2.2 Technical proofs of the encoding propositions

Proof of Proposition 14.6. The weighted construction has nonzero weights $O(n_0^3)$ and distinguished weight $O(n_0^5)$. Therefore $n = \sum_i w_i = O(n_0^5)$. The restricted source makes n_0 polynomial in its combinatorial size, so all points and queries can be listed explicitly in polynomial time. □

Proof of Proposition 14.7. There are $n - (N + 1)$ private lines and at most $2N$ whole-cloud lines, hence $O(n)$ supplied queries. The calibrated spacing bounds all coordinates and coefficients as verified below. Their factored values establish strict binary answers; the private and whole-cloud queries separate every pair of hypotheses. Distinct parabola points are exposed by their tangents, so the construction is strictly convex. □

Proof of Proposition 14.8. The transfer identity gives $\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} \le k$ if and only if $\mathrm{OPT}_{\mathrm{E}}^{\mathrm{cloud}} \le (k + \Delta)/n$. Both k and Δ are bounded by fixed polynomials in n: k inherits the weighted construction's polynomial bound, while $D(w_i) \le w_i^2$ for $w_i \ge 1$ gives $\Delta \le n^2$. A proper identifying tree has $n - 1$ internal nodes and is a polynomial certificate. Evaluate each supplied answer and sum the leaf depths to verify its rational expected cost in polynomial time. □

14.2.3 Calibration of the cloud spacing

Parameter calibration.
Choose $Q = 4 \max_i w_i$. Every cloud has width less than $Q/4$; the half-anchor boundaries of (50) therefore miss all points and select exactly the stated clouds. Since $Q \le 4n$ and $N \le n$, every abscissa is $O(n^2)$. Squaring it bounds the ordinates by $O(n^4)$, and the formulas for both query types bound all integer coefficients by $O(n^4)$.

Remark 14.9 (Limits of the reduction). The cloud identity preserves exact hardness but adds the same nonnegative internal cost Δ to every matching. An EPTAS for the compressed weighted path family, such as Theorem 9.5, remains an EPTAS for these explicit cloud instances: its additive error is at most $\varepsilon\,\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} \le \varepsilon(\mathrm{OPT}_{\mathrm{raw}}^{\mathrm{weighted}} + \Delta)$. Thus this reduction does not establish constant-factor inapproximability. That question requires a different construction.

15 Conclusion

Objective	Polynomial time	Quasipolynomial time
Maximum depth, binary answers	$2 + \gamma = 4.46496526$	$2 + \varepsilon$
Maximum depth, on-line answers	$4 + 2\gamma = 8.92993052$	$4 + \varepsilon$
Expected depth, binary answers and arbitrary priors	$\rho_{\mathrm{two}} < 5.182976$	$2 + \varepsilon$
Expected depth, on-line answers and arbitrary priors	$2\rho_{\mathrm{two}} < 10.365952$	$4 + \varepsilon$

The common-witness normalization is the structural step shared by both algorithms. In a flat state, it turns each supplied test into an opposite cap and represents that cap by at most two adjacent vertices of a convex hull. For each hypothesis, the positive hull answers form a circular interval. This representation does not enlarge the query family: it supplies an interval view of the tests that were already given.

The two objectives then use that structure differently. For maximum depth, the algorithm schedules discovery around the actual heights of its recursively built children. The comparison path in an optimal tree supplies the deadline credits, while adjacent support blocks control discovery delay; route-specific endpoint searches preserve those credits. Exact Cartesian-tree scheduling with rank subsets handles logarithmically many discovery slots in polynomial time. A global slot cap and the priority fallback support simultaneous small-optimum and universal bounds, yielding factor 4.464966. For expected depth, the algorithm learns each full hull pattern before recursing. Queries

that become unnecessary after restriction pay for discovery, and the entropy still remaining after discovery pays for endpoint information. Exact integer-budget coverage applied to adjacent hull blocks, joint discovery and endpoint accounting, and the zero-pattern waiting time sharpen the polynomial factor. The final algorithm chooses its first two coverage budgets jointly with the geometric shift and the two-endpoint mode. Signed first-cover corrections and a separate all-negative waiting invoice retain both restriction and information credits. In both analyses, the local credits are decreases of quantities that telescope over the recursive tree.

The quasipolynomial algorithms store compatible discovery grades rather than optimizing each deadline separately. Their internal bounds have leading coefficient arbitrarily close to two and an additive count or entropy potential. Exact logarithmic-depth lookahead absorbs that potential and gives the final $(2+\varepsilon)$ ratio. The on-line-answer entries follow from the same factor-two simulation, using the binary guarantee with parameter $\varepsilon/2$. These results have quasipolynomial running time.

For randomized per-hypothesis depth, the adversary chooses the hidden hypothesis before the random tree is sampled. Multiplicative weights constructs trees for a sequence of positive rational priors; a restricted mixture LP then minimizes their largest per-hypothesis expected depth. The transfer preserves the best general expected-depth factors and compresses the strategy to at most n trees. This is a different objective from deterministic maximum depth. The LP value and the oracle's comparison priors also provide an instance-specific approximation certificate.

The small-cap results place additional limits on potential hardness constructions. With a distinguished all-miss hypothesis and fixed cap size, maximum depth is exactly solvable in polynomial time. The weighted singleton/pair path-and-cycle subclass with all singleton tests has an EPTAS for expected depth. Thus the current weighted pair-path reduction cannot yield APX-hardness unless P = NP. Uniform fixed-size caps in convex position with an all-miss hypothesis also admit a PTAS, without requiring singleton tests. Clearing short gaps permits exact optimization of the residual block interleavings; the prefix cost is a small fraction of the full optimum. Constant-factor hardness must therefore escape this additional restricted class.

The hardness reductions show that this geometry does not make exact optimization easy: even points on a parabola with only supplied binary line queries encode strongly NP-complete 3-Partition instances. The expected-depth result holds for both arbitrary positive priors and uniform priors: a uniform cloud lift adds a fixed internal cost to the weighted optimum. Constant-factor inapproximability remains open. Whether the polynomial factors can be lowered or the quasipolynomial factors can be brought below two remains open.

A Fixed certificate for the two initial batches

This table specifies the proof distribution in Theorem 6.5. It is independent of the input; the algorithm simply minimizes its rational score over all candidate schedules. The shift has density $1/(u \ln \beta)$ on $[1, \beta)$. An entry $(q, r; \mathsf{L})$ selects that initial-budget variant and one first-batch saving minorant. Here $r = 0$ means budgets $q, k_2, k_3, \ldots$, and $r > 0$ means $q, r, k_2, k_3, \ldots$. All entries use

$$\mathsf{A}_q = M_q - \delta w q, \qquad \mathsf{C}_q = c_* M_q - d_* \delta w q.$$

A mixture $\theta(q, r; \mathsf{L}) + (1-\theta)(q', r'; \mathsf{L}')$ is conditional on that row's shift range. It compares two actual variants; it never adds two savings from the same schedule.

The twelve rational mixing weights are listed as integer numerators with common denominator 10^9:

i	1	2	3	4	5	6
$10^9\theta_i$	363738471	165441165	467444028	619060535	88721951	689586436
i	7	8	9	10	11	12
$10^9\theta_i$	751593847	874972087	599476351	160123414	234739978	835733626

The shift ranges also split at the integer breakpoints of k_2. Thus each $k_j = \lfloor \beta^j u \rfloor$, $j = 0, 1, 2$, is constant inside each listed range, and the signed correction of (11) is constant there.

Row	Shift range	Variant and saving minorant
1	$[1, 13/\beta^2)$	$(3, 0; \mathsf{C}_3)$
2	$[13/\beta^2, 14/\beta^2)$	$\theta_1(3, 0; \mathsf{C}_3) + (1-\theta_1)(4, 0; \mathsf{A}_4)$
3	$[14/\beta^2, 4/\beta)$	$(4, 0; \mathsf{A}_4)$
4	$[4/\beta, 15/\beta^2)$	$(4, 0; \mathsf{A}_4)$
5	$[15/\beta^2, 16/\beta^2)$	$(4, 0; \mathsf{A}_4)$
6	$[16/\beta^2, 17/\beta^2)$	$\theta_2(1, 5; \mathsf{C}_1) + (1-\theta_2)(4, 0; \mathsf{A}_4)$
7	$[17/\beta^2, 5/\beta)$	$(1, 5; \mathsf{C}_1)$
8	$[5/\beta, 18/\beta^2)$	$(1, 5; \mathsf{C}_1)$
9	$[18/\beta^2, 19/\beta^2)$	$(1, 5; \mathsf{C}_1)$
10	$[19/\beta^2, 20/\beta^2)$	$(1, 5; \mathsf{C}_1)$
11	$[20/\beta^2, 21/\beta^2)$	$\theta_3(1, 5; \mathsf{C}_1) + (1-\theta_3)(1, 6; \mathsf{C}_1)$
12	$[21/\beta^2, 6/\beta)$	$(1, 6; \mathsf{C}_1)$
13	$[6/\beta, 22/\beta^2)$	$(1, 6; \mathsf{C}_1)$
14	$[22/\beta^2, 23/\beta^2)$	$(1, 6; \mathsf{C}_1)$
15	$[23/\beta^2, 24/\beta^2)$	$(1, 6; \mathsf{C}_1)$
16	$[24/\beta^2, 7/\beta)$	$(1, 7; \mathsf{C}_1)$
17	$[7/\beta, 2)$	$\theta_4(1, 6; \mathsf{C}_1) + (1-\theta_4)(1, 7; \mathsf{C}_1)$
18	$[2, 25/\beta^2)$	$\theta_5(1, 6; \mathsf{C}_1) + (1-\theta_5)(2, 7; \mathsf{C}_2)$
19	$[25/\beta^2, 26/\beta^2)$	$(2, 7; \mathsf{C}_2)$
20	$[26/\beta^2, 27/\beta^2)$	$(2, 7; \mathsf{C}_2)$
21	$[27/\beta^2, 28/\beta^2)$	$\theta_6(2, 7; \mathsf{C}_2) + (1-\theta_6)(2, 8; \mathsf{C}_2)$

Row	Shift range	Variant and saving minorant
22	$[28/\beta^2, 8/\beta)$	$(2, 8; \mathsf{C}_2)$
23	$[8/\beta, 29/\beta^2)$	$(2, 8; \mathsf{C}_2)$
24	$[29/\beta^2, 30/\beta^2)$	$(2, 8; \mathsf{C}_2)$
25	$[30/\beta^2, 31/\beta^2)$	$(2, 8; \mathsf{C}_2)$
26	$[31/\beta^2, 9/\beta)$	$(2, 9; \mathsf{C}_2)$
27	$[9/\beta, 32/\beta^2)$	$\theta_7(2, 8; \mathsf{C}_2) + (1-\theta_7)(2, 9; \mathsf{C}_2)$
28	$[32/\beta^2, 33/\beta^2)$	$(2, 9; \mathsf{C}_2)$
29	$[33/\beta^2, 34/\beta^2)$	$(2, 9; \mathsf{C}_2)$
30	$[34/\beta^2, 35/\beta^2)$	$\theta_8(2, 9; \mathsf{C}_2) + (1-\theta_8)(2, 10; \mathsf{C}_2)$
31	$[35/\beta^2, 10/\beta)$	$(2, 10; \mathsf{C}_2)$
32	$[10/\beta, 36/\beta^2)$	$\theta_9(2, 10; \mathsf{C}_2) + (1-\theta_9)(3, 10; \mathsf{C}_3)$
33	$[36/\beta^2, 37/\beta^2)$	$(3, 10; \mathsf{C}_3)$
34	$[37/\beta^2, 3)$	$(3, 10; \mathsf{C}_3)$
35	$[3, 38/\beta^2)$	$(3, 10; \mathsf{C}_3)$
36	$[38/\beta^2, 11/\beta)$	$\theta_{10}(3, 10; \mathsf{C}_3) + (1-\theta_{10})(3, 11; \mathsf{C}_3)$
37	$[11/\beta, 39/\beta^2)$	$(3, 10; \mathsf{C}_3)$
38	$[39/\beta^2, 40/\beta^2)$	$(3, 11; \mathsf{C}_3)$
39	$[40/\beta^2, 41/\beta^2)$	$(3, 11; \mathsf{C}_3)$
40	$[41/\beta^2, 42/\beta^2)$	$(3, 11; \mathsf{C}_3)$
41	$[42/\beta^2, 12/\beta)$	$\theta_{11}(3, 11; \mathsf{C}_3) + (1-\theta_{11})(3, 12; \mathsf{C}_3)$
42	$[12/\beta, 43/\beta^2)$	$(3, 12; \mathsf{C}_3)$
43	$[43/\beta^2, \beta)$	$\theta_{12}(3, 12; \mathsf{C}_3) + (1-\theta_{12})(3, 13; \mathsf{C}_3)$

For each row $[l, h)$ and option of conditional weight v, compute $P = \ln(h/l)/\ln\beta$. Starting with $a_t = e_\beta(t)$ and $\mu = 0$, add $Pv[d(u,t) + s\mathbf{1}_{\{t\le q\}}]$ to a_t and $Pv\rho$ to μ, where the saving minorant is $sM_q - \rho\delta w$. The main text gives the complete finite-rank, zero-length, and tail checks. The file `expected-two-phase-check.py`, using the same rational data in `expected-two-phase-certificate.json`, reproduces them with outward logarithm enclosures.

References

[1] Uriel Feige, László Lovász, and Prasad Tetali. *Approximating Min-sum Set Cover*. Algorithmica, 2004. Primary journal manuscript: `https://tetali.math.gatech.edu/PUBLIS/mssc_final.pdf`. The generic greedy guarantee is mentioned only as background; the interval-specific ordering used here is proved in Lemma 5.5.

[2] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*, 2nd edition. Wiley-Interscience, 2006. Chapters 2 and 5 cover the entropy chain rule, the log-sum inequality, Kraft's inequality, and binary prefix coding.

[3] Jiří Matoušek. *Lectures on Discrete Geometry*. Graduate Texts in Mathematics 212, Springer, 2002. Standard convex-hull terminology and the planar form of Helly's theorem.

[4] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*, 3rd edition. Springer, 2008. Planar convex-hull algorithms and their running times.

[5] L. G. Khachiyan. A polynomial algorithm in linear programming. *Soviet Mathematics Doklady*, 20:191–194, 1979.

[6] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. Problem SP15 supplies strong NP-completeness of 3-Partition, including the restriction $B/4 < a_i < B/2$.

[7] Sariel Har-Peled and Mira Lee. Weighted geometric set cover problems revisited. *Journal of Computational Geometry*, 3(1):65–85, 2012. Exact minimum-weight planar halfplane cover: `https://doi.org/10.20382/jocg.v3i1a4`.

[8] Yurii Nesterov and Arkadii Nemirovskii. *Interior-Point Polynomial Algorithms in Convex Programming*. SIAM, 1994. Polynomial-time convex quadratic optimization; author lecture notes: `https://www2.isye.gatech.edu/~nemirovs/Lect_IPM.pdf`.

[9] Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of Computing*, 8(6):121–164, 2012. `https://theoryofcomputing.org/articles/v008a006/v008a006.pdf`.

[10] Yoav Freund and Robert E. Schapire. Adaptive game playing using multiplicative weights. *Games and Economic Behavior*, 29:79–103, 1999. Author manuscript: `https://cseweb.ucsd.edu/~yfreund/papers/games_long.pdf`.